

11. live / sim 共通 MPC 本体

solar_ws0129-main

2026-07-29

Contents

対象

項目	内容
ファイル	mpc_solarcar/mpc_node.py
ソース SHA-256	fd107b7606b4eab77abdeb47d7d6c0fc8a151a56dba13fab2a277a538f4ed56a
種別	Python
区分	runtime core
役割	forecast、route、vehicle telemetry、maps を使って上位速度計画と下位追従指令を出す ROS2 ノード。
起動文脈	sim/live/live_wifi で中心に動く単一障害点に近いノード。

このファイルを読む理由

forecast、route、vehicle telemetry、maps を使って上位速度計画と下位追従指令を出す ROS2 ノード。

- SolarCarModel を直接生成する。
- 1 Hz upper timer と lower timer 群を並列 callback group で回す。
- calibration topic で内部係数を上書きする。

呼び出し関係

どこから呼ばれるか

- live_launch.py
- solarcar_sim.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/model.py
- mpc_solarcar/upper_cost.py
- mpc_solarcar/estimator.py

同一リポジトリ内の主要依存

- mpc_solarcar/estimator.py
- mpc_solarcar/forecast_grid.py
- mpc_solarcar/model.py
- mpc_solarcar/path_utils.py
- mpc_solarcar/route_utils.py
- mpc_solarcar/schedule_utils.py
- mpc_solarcar/signal_utils.py
- mpc_solarcar/upper_cost.py

- mpc_solarcar/upper_horizon.py
- mpc_solarcar/upper_policy.py
- mpc_solarcar/upper_solver.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from .upper_solver import hybrid_bounded_minimize

class MPCNode(Node):
    """
    MPC node with two modes:
    - Default: solarcar MPC (forecast-driven)
    - Passo mode: fuel-minimizing advisory MPC
    """

    def __init__(self):
        super().__init__('mpc_node')
        self.params_cfg = {}
        # A full-race upper solve can take longer than the 1 Hz control period.
        # Keep telemetry and lower control responsive while that solve runs.
        self.telemetry_callback_group = MutuallyExclusiveCallbackGroup()
        self.upper_callback_group = MutuallyExclusiveCallbackGroup()
        self.lower_callback_group = MutuallyExclusiveCallbackGroup()
        self.command_callback_group = MutuallyExclusiveCallbackGroup()
        self.declare_parameter('passo_mode', False)
        self.passo_mode = bool(self.get_parameter('passo_mode').value)
        if self.passo_mode:
```

主要構造の読み取り

主要クラスは MPCNode。主要関数は z_next_for, quad_penalty, cost, expand_ctrl, build_balance_seed, integrate_stationary_duration, step_wait, apply_control_stop_at。ROS パラメータ宣言は 151 件。ROS I/O は publisher 14 件、subscription 19 件。

主要クラス・関数

行	種別	名前	補足
42	class	MPCNode	bases: Node
49	function	__init__	args: self
66	function	_load_stops	args: self, stop_yaml
76	function	_load_forecast_file	args: self, path
102	function	_apply_params_yaml	args: self, path
172	function	_maybe_reload_forecast	args: self

192	function	_on_s_km_solar	args: self, msg
208	function	_on_speed_solar	args: self, msg
216	function	_on_soc_solar	args: self, msg
225	function	_on_tb_solar	args: self, msg
234	function	_on_i_solar	args: self, msg
242	function	_on_v_solar	args: self, msg
250	function	_measured_distance_km	args: self
256	function	_sync_measured_state	args: self
270	function	_on_calib_solar_gain	args: self, msg
276	function	_on_calib_drive_ power_gain	args: self, msg
284	function	_on_calib_aux_power	args: self, msg
291	function	_init_solar	args: self
780	function	_current_bin_index	args: self
830	function	_horizon_data	args: self, k0
872	function	_forecast_at_time	args: self, t_utc, s_km, drive, control_stop
948	function	_sample_plan_segments	args: self, dt_sample
959	function	_route_value	args: self, s_km, field, default
970	function	_route_average	args: self, s0_km, s1_km, field, default
979	function	_speed_limit_at	args: self, s_km, default_kmh
987	function	_soc_guard_speed	args: self, v_kmh, s_km, d0
999	function	z_next_for	args: v_kmh_local
1045	function	_control_stop_catalog	args: self
1065	function	_update_live_control_ stop_hold	args: self, s_km, v_kmh
1115	function	_dwell_penalty	args: self, s_km, v_ms
1127	function	_mpc_solve_solar	args: self, data
1164	function	quad_penalty	args: x, cap
1171	function	cost	args: v
1291	function	_mpc_solve_solar_ distance	args: self, t0_utc, s0_km, x0
1379	function	expand_ctrl	args: u_vec
1382	function	build_balance_seed	
1483	function	integrate_stationary_ duration	args: t_utc, z, Tb, s_km, dt_wait
1538	function	step_wait	args: t_utc, z, Tb, s_km
1547	function	apply_control_stop_at	args: t_utc, z, Tb, s_km
1565	function	cost	args: u_vec
1816	function	_publish_upper_plan	args: self
1823	function	_publish_lower_plan	args: self, v_seq_ms
1830	function	_publish_plan_path	args: self, data
1858	function	_publish_metrics	args: self, d0, v_exec_kmh, s_for_profile
1897	function	_publish_summary	args: self, v_exec_kmh
1929	function	_interp_upper_speed	args: self, t_sec
1952	function	_distance_plan_speed	args: self, s_km
1958	function	_tau_max_for_mode	args: self, maps, mode
1965	function	_tau_limits	args: self
1981	function	_traction_force	args: self, tau_nm

1988	function	<code>_pack_from_tau</code>	args: self, v_ms, tau_nm, z, Tb, env
2012	function	<code>_build_lower_ref</code>	args: self, base_time_utc, s_km, d0
2039	function	<code>_lower_rollout</code>	args: self, v0_ms, u_seq, env, z0, Tb0, tau_drive, tau_regen
2072	function	<code>_lower_mpc_solve</code>	args: self, v0_ms, s0_km, z0, Tb0, env, v_ref_seq
2185	function	<code>cost</code>	args: u_vec
2273	function	<code>_step_solar</code>	args: self
2549	function	<code>_step_lower</code>	args: self
2600	function	<code>_publish_lower_command_cycle</code>	args: self
2618	function	<code>_init_passo</code>	args: self
2709	function	<code>_on_s_km</code>	args: self, msg
2712	function	<code>_on_speed</code>	args: self, msg

ファイルを上から読んだときの定義順

行	内容
42	クラス MPCNode を定義する。
2965	関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
2	<code>import math</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L196, L199, L254, L259, L261, L264, L267, L616, ...。
3	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L180, L434。
4	<code>import time</code>	<code>wall/monotonic time</code> に基づく周期制御や <code>freshness</code> 判定を行うため。このファイル内での主な使用位置は L175, L437, L1070, L1915, L2017, L2374, L2398, L2401, ...。
5	<code>from collections import deque</code>	固定長の時系列や遅延キューを効率よく保持するため。このファイル内での主な使用位置は L2676。
6	<code>from datetime import datetime, timezone, timedelta</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L713, L717, L718, L786, L810, L815, L824, L846, ...。
8	<code>import rclpy</code>	ROS 2 Python ノードとして起動・ <code>spin</code> するため。このファイル内での主な使用位置は L2966, L2975。
9	<code>from rclpy.callback_groups import MutuallyExclusiveCallbackGroup</code>	同一ノード内の <code>callback</code> 実行系を分け、 <code>up-callback group</code> と <code>down-callback group</code> を並列に扱うため。このファイル内での主な使用位置は L54, L55, L56, L57。
10	<code>from rclpy.executors import MultiThreadedExecutor</code>	複数 <code>callback group</code> を並列実行する <code>executor</code> を使うため。このファイル内での主な使用位置は L2968。
11	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L42。

12	<code>fromrcipy. parameterimportParameter</code>	<code>rcipy.parameter</code> から <code>Parameter</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L166, L2750, L2752, L2754, L2756。
14	<code>importyaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L70, L105, L2740。
15	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L77, L79, L98, L100, L451, L460, L1904。
16	<code>importnumpyasnp</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L221, L230, L265, L268, L272, L278, L542, L545, ...。
18	<code>fromstd_msgs.msgimportBool, Float32,Float32MultiArray, String</code>	<code>Float32</code> や <code>String</code> など軽量メッセージで <code>planner / vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L192, L208, L216, L225, L234, L242, L270, L276, ...。
19	<code>fromnav_msgs.msgimportPath</code>	将来軌跡を <code>dashboard</code> や <code>logger</code> へ出すため。このファイル内での主な使用位置は L741, L1833, L2701, L2880。
20	<code>fromgeometry_msgs. msgimportPoseStamped</code>	<code>Path</code> を構成する <code>waypoint pose</code> を組み立てるため。このファイル内での主な使用位置は L1842, L1851, L2886。
22	<code>fromscipy. optimizeimportminimize</code>	連続最適化や MHE の逆推定を解くため。このファイル内での主な使用位置は L1278, L2251, L2874。
24	<code>from. modelimportSolarCarModel, Params</code>	車体物理・電気モデル本体から <code>SolarCarModel</code> 、 <code>Params</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/model.py</code> 。このファイル内での主な使用位置は L509, L513。
25	<code>from.path_ utilsimportresolve_path</code>	ROS share / 相対パス解決から <code>resolve_path</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/path_utils.py</code> 。このファイル内での主な使用位置は L135, L423, L424, L442, L450, L459, L470, L483, ...。
26	<code>from.route_ utilsimportaverage_profile, interpolate_profile</code>	<code>route_utils.py</code> から <code>average_profile</code> 、 <code>interpolate_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/route_utils.py</code> 。このファイル内での主な使用位置は L963, L974, L983。
27	<code>from.schedule_ utilsimportDriveSchedule</code>	<code>schedule_utils.py</code> から <code>DriveSchedule</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/schedule_utils.py</code> 。このファイル内での主な使用位置は L484。
28	<code>from. estimatorimportBatteryMHE, MheInput,MheMeas</code>	<code>Battery MHE</code> から <code>BatteryMHE</code> 、 <code>MheInput</code> 、 <code>MheMeas</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/estimator.py</code> 。このファイル内での主な使用位置は L724, L2493, L2504。
29	<code>from.forecast_ gridimportbuild_forecast_ grid_payload,interp_ forecast_grid</code>	<code>forecast_grid.py</code> から <code>build_forecast_grid_payload</code> 、 <code>interp_forecast_grid</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/forecast_grid.py</code> 。このファイル内での主な使用位置は L96, L885, L886, L887, L890, L891, L896, L898, ...。

30	<code>from.signal_utilsimportRobustScalarFilter,finite_float,fresh_enough,slew_limit</code>	signal_utils.py から RobustScalarFilter, finite_float, fresh_enough, slew_limit を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/signal_utils.py。このファイル内での主な使用位置は L195, L252, L253, L258, L263, L266, L636, L643, ...。
31	<code>from.upper_costimportload_upper_cost_config,upper_stage_cost,upper_terminal_cost,quad_penalty</code>	上位 MPC 目的関数から load_upper_cost_config, upper_stage_cost, upper_terminal_cost, quad_penalty を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_cost.py。このファイル内での主な使用位置は L590, L1227, L1231, L1236, L1238, L1242, L1250, L1251, ...。
32	<code>from.upper_horizonimportbuild_upper_distance_horizon,plan_segment_index</code>	上位 MPC 距離メッシュ生成から build_upper_distance_horizon, plan_segment_index を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_horizon.py。このファイル内での主な使用位置は L1293, L1953。
33	<code>from.upper_policyimportabsolute_control_distances,interpolate_upper_policy,load_upper_policy_csv,shift_upper_policy_warm_start</code>	上位速度計画の補間と warm start から absolute_control_distances, interpolate_upper_policy, load_upper_policy_csv, shift_upper_policy_warm_start を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_policy.py。このファイル内での主な使用位置は L471, L1335, L1345, L1355。
39	<code>from.upper_solverimporthybrid_bounded_minimize</code>	上位探索ソルバから hybrid_bounded_minimize を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/upper_solver.py。このファイル内での主な使用位置は L1719。

このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体を作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

先頭アンダースコア、__init__、名前付けの作法

‘self._step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 '(N)」、候補集団なら '(population, N)となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscription などを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC 内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executor の実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった elite 候補から分布を更新する反復的な確率最適化である。このリポジトリの `upper_solver.py` は各制御点速度を独立正規分布で生成し、上下限へ clip する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$

$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left(\text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで $\mathcal{E}_g$ は cost が小さい上位 K 候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 `hybrid_bounded_minimize` は、`deterministic seed` を評価し、上位候補を `L-BFGS-B` で局所 refine し、設定と seed 間不一致に応じて CEM を実行し、最後に再度局所 refine する。したがって CEM 単独ではなく hybrid solver である。

CEM で落とした候補を永久保存しないこと自体は通常的最適化として自然だが、off-nominal 状態からの再利用には別の policy library 設計が必要である。状態を無制限に全組合せ保存する代わりに、SoC、進捗、時刻、予報誤差、停止状態などの scenario を設計し、近傍 policy を検索して MPC で再最適化する。

根拠資料

- Rubinstein and Kroese: [The Cross-Entropy Method](#)
- SciPy 公式: [scipy.optimize.minimize](#)

warm start は何を保存し、どう効くか

warm start は前回または offline 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 library の一員を与える。

$$u_i^{0,\text{new}} = \text{interp} \left(s_i^{\text{new}}; s_j^{\text{old}}, u_j^{*,\text{old}} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して shift する。初期状態や天候が変われば cost 評価は新条件で行われるので、warm start が不適切でも他 seed、CEM、安全 fallback が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 mean または候補 pool、receding horizon なら前回解の時間・距離 shift として現れる。どの位置に渡しているかを呼出引数まで追う。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- Rubinstein and Kroese: [The Cross-Entropy Method](#)

車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 power は概ね ‘ $P_{\text{mech}} = F_{\text{total}} \cdot v + P_{\text{inertia}}$ ’ で、駆動時と回生時で異なる効率 map を通して DC 側 power へ変換する。map は速度・torque などを軸にした実測または同定 table であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{\text{oc}}(z, T) - I R_{\text{total}}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T) I \Delta t}{Q_{\text{eff}}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

上位 MPC と下位 MPC の役割

上位層は長い距離または時間を見て、エネルギー、到着、停止、速度制限、終端 SoC を考えた速度計画を出す。下位層は短い周期で実測速度を見て、上位速度へ追従する駆動・回生入力を求める。

予報・route・SoC -> 上位MPC -> 将来速度列 -> 下位MPC -> throttle/回生/drive mode -> driverまたはvehicle -> 新しいtelemetry

上位解が遅い間も下位出力を止めないこと、古い計画を安全に保持すること、上位と下位で単位と時刻基準を一致させることが実装上重要である。

launch Action、Node Action、実行可能名、remapping

‘launch_ros.actions.Node(...)’は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

‘DeclareLaunchArgument’は launch 実行時に受け取る入力欄を宣言し、‘LaunchConfiguration’はその値を後で解決する substitution を表す。‘perform(context)’は実行時 context から確定文字列を取り出す。

launch の ‘name’は Node 名 override、‘parameters’は起動時 parameter、‘remappings’は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- ROS 2 Humble 公式: [rqt_graph](#)
- ROS 2 Humble 公式: [Recording and playing back data](#)
- ROS 2 公式: [Executors](#)

mpc_node.py 専用の統合解説

起動機構と MPC 内部計算を一つの時間軸で接続する。

起動経路を大本から一本につなぐ

```
SolarSim.ps1 -> scripts/solar_control.sh -> ROS 2 launch -> launch_ros.actions.Node -> setup.pyのconsole_scripts -> mpc_solarcar.mpc_node:main -> rclpy.init -> MPCNode() -> __init__ -> _init_solar -> Executor.add_node -> spin
```

この対応は推測ではない。setup.py の console_scripts には ‘mpc_node = mpc_solarcar.mpc_node:main’ が登録され、live_launch.py と solarcar_sim.launch.py は executable として mpc_node を指定する。launch が main という名前を探索するのではない。

launch 側の Node はプロセス起動指示、rclpy.node.Node は基底クラス、MPCNode はこのリポジトリが定義した派生クラス、‘node = MPCNode()’ の node は実行中メモリに作られたインスタンス、‘/mpc_node’ は ROS graph 上の名前である。

main と spin の前後で実行方式が変わる

```
def main():
    rclpy.init()
    node = MPCNode()
    executor = MultiThreadedExecutor(num_threads=4)
    executor.add_node(node)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()
```

spin までは通常の Python として上から一度だけ進む。spin 後は Executor の event loop へ入り、timer と受信が ready になった時に callback が呼ばれる。callback 終了後は main の次行ではなく Executor へ戻る。

finally は Ctrl+C、launch 停止、例外などで spin を抜けた後の後始末である。Executor、Node、rclpy context の順に明示終了する。

MPCNode.__init__ と self の正確な意味

‘MPCNode()’ を評価すると、新しいインスタンスが作られ、その参照が self として ‘MPCNode.__init__’ へ入る。‘super().__init__(‘mpc_node’)’ が基底 Node を初期化して初めて ROS Node 機能を持つ。

‘self.z’、‘self.Tb’、‘self.model’ は同じ MPCNode インスタンスに保存される状態である。callback が別時刻に呼ばれても同じ属性を参照する。‘z’ のような局所変数はその関数呼出し内だけである。

先頭アンダースコア付きメソッドは内部実装という慣習、‘__init__’ は Python 特殊メソッドである。‘@Class’ という一般構文はなく、このファイルの MPCNode 自体には dataclass decorator は使われていない。

四つの Callback Group と共有状態

telemetry、upper、lower、command の四つを別 MutuallyExclusiveCallbackGroup へ置く。同じ group 内は同時実行しないが、別 group は四 worker で並行実行し得る。長い上位 solve 中にも telemetry 受信と下位出力を続ける意図である。

別 group は self.z、self.Tb、self.v_now、self.last_data など共有する。コードは上位 solve 後に ‘_sync_measured_state()’ を再実行して長時間 solve 中に受けた最新値を反映するが、明示 lock ですべてを原子的 snapshot 化しているわけではない。

Python の GIL は複数文にまたがる状態整合性を保証しない。診断時は callback 開始終了時刻、plan ID、telemetry timestamp を併記し、どの状態 snapshot で解いたか確認する。

時間基準上位 MPC

‘_mpc_solve_solar(data)’ は将来速度列を最適化変数とし、各予測 step で慣性 power、electrical balance、SoC、温度、距離、制約 penalty を順に計算する。

$$P_{\text{inertia},k} = \frac{\frac{1}{2}m(v_k^2 - v_{k-1}^2)}{\Delta t}, \quad s_{k+1} = s_k + \frac{v_k \Delta t}{1000}$$

‘model.electrical_balance’ が電流、電圧、pack power、損失を返し、‘model.soc_step’ が SoC を進める。温度更新と cost 蓄積は呼出側にもある。最後に SciPy L-BFGS-B へ cost、初期値、bounds を渡す。

距離基準上位 MPC と CEM

‘_mpc_solve_solar_distance’ は route を距離区間へ分け、制御点速度 u を区間速度へ線形補間する。停止座標を mesh edge へ追加し、到着、dwell 発電、再出発を同じ座標で評価する。

初期候補は、前回 plan の距離 shift、offline initial policy の補間、現在速度一定の順で選ぶ。別途 balance seed を作り、‘hybrid_bounded_minimize’ へ渡す。

solver は seed 評価、上位 seed の L-BFGS-B、条件付き CEM、再 L-BFGS-B を行う。CEM の各世代では mean 候補を 1 個必ず含め、残りを正規乱数で生成し、elite 平均と標準偏差で次世代分布を更新する。

finite grid 全列挙の証明は宣言 grid 上に限る。CEM や局所候補を含む連続領域の大域最適性を意味しないため、solve_info の ‘discrete_global_proof’、‘finite_library_global_proof’、‘certificate_scope’ を区別する。

warm start と off-nominal 状態

前回速度列を新しい絶対距離制御点へ補間するため、予定より遅い・速い場合でも過去区間を除いた残り計画を初期値にできる。これは plan を固定せず、現在 SoC、温度、天候で cost を再評価する。

offline CEM で理想軌道だけを残すと大きな逸脱時の seed 品質は落ちる。対策は全連続状態の全組合せ保存ではなく、SoC 偏差、進捗偏差、時刻、予報 scale、停止継続、温度などの scenario library を設計し、近傍 policy と physics seed を併用して live MPC で修正することである。

この現行ノードは前回 plan、initial_upper_policy、balance seed、generic seed、CEM を併用するが、多次元 scenario library 検索を完全実装したものではない。この境界を資料上で保証と将来拡張に分ける。

下位 MPC と指令継続

‘_build_lower_ref’ が上位 plan から短期参照速度列を作り、‘_lower_mpc_solve’ が駆動・回生入力 u を求める。逆動力学 seed を必ず作り、設定した場合だけ L-BFGS-B で refine する。

上位 solve 中は option により下位 refine を省略し、決定論的入力を使う。‘_publish_lower_command_cycle’ は独立 timer から保存済み指令を出すため、optimizer 完了を待たず出力を継続する。

ただし docstring は ‘1 Hz output path’ と書く一方、実際の timer 周期は ‘1/lower_rate_hz’ で、既定 lower_rate_hz が 5 なら 5 Hz である。説明では実装値を正とし、この不一致を既知の文書上問題として記す。

実測同期、freshness、MHE、fallback

各 telemetry callback は受信時刻と filter 済み値を保存する。‘_sync_measured_state‘は timeout 内の値だけを状態へ反映し、古い速度は NaN に戻す。距離には大きな後退値を捨てる guard がある。

MHE 有効時は観測可能な SoC、温度、電流、電圧を窓へ push して状態を推定する。無効時は車両モデルで状態を進めるが、新鮮な SoC または温度実測がある項目はモデル上書きを避ける。

solver 失敗時は warm-start plan または決定論的入力へ fallback する。さらに schedule、速度制限、SoC guard、control stop hold を optimizer 後段で強制するため、soft penalty だけに安全を依存しない。

実機で使う診断順

```
ros2 node list
ros2 node info /mpc_node
ros2 topic info -v /vehicle/speed_kmh
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /vehicle/speed_kmh
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

Node が無い場合は launch・console script・process、topic が無い場合は publisher、周期が遅い場合は通信または Executor、値が古い場合は timestamp/freshness、指令が 0 なら schedule/stop/SoC guard、solver 失敗なら status と log、という順で範囲を狭める。

空転試験では距離を進めない入力 source を使い、実 GPS と replay publisher を同時に接続しない。bag へ telemetry、upper/lower 指令、status を同時記録し、停止後に同じ profile と source revision で再生する。

現行改修の設計意図と残る注意点

現行コードのコメントと差分から、長時間 full-race solve 中の応答維持、2 次元 forecast 補間、実測 freshness、runtime profile override、offline policy warm start、control stop の正確な mesh 分割、決定論的 lower fallback、solve 時間 log を目的とした改修を確認できる。

一方、異なる Callback Group 間の共有状態 snapshot、広い ‘except Exception‘、command の二経路 publish、1 Hz という docstring と実 timer の不一致は、利用者が挙動を理解するうえで注意が必要である。これらは機能説明と保証範囲を分けて記載する。

Git index 上では mpc_node.py が競合未解決状態であるため、本資料は 2026-07-29 時点のワークツリー内容を根拠とし、merge 完了後は manifest の source hash を更新して再生成する必要がある。

関数・クラスを上から順に解説

L42 クラス MPCNode

- 行範囲: L42–L2962
- 所属: module 直下
- 定義: MPCNode(bases=Node)
- docstring: MPC node with two modes: - Default: solarcar MPC (forecast-driven) - Passo mode: fuel-minimizing advisory MPC
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `docstring` または説明文字列を置く。
2. 関数 `__init__` を定義する。
3. 関数 `_load_stops` を定義する。
4. 関数 `_load_forecast_file` を定義する。
5. 関数 `_apply_params_yaml` を定義する。
6. 関数 `_maybe_reload_forecast` を定義する。
7. 関数 `_on_s_km_solar` を定義する。
8. 関数 `_on_speed_solar` を定義する。
9. 関数 `_on_soc_solar` を定義する。
10. 関数 `_on_tb_solar` を定義する。
11. 関数 `_on_i_solar` を定義する。
12. 関数 `_on_v_solar` を定義する。

代表コード断片

```
class MPCNode(Node):
    """
    MPC node with two modes:
    - Default: solarcar MPC (forecast-driven)
    - Passo mode: fuel-minimizing advisory MPC
    """

    def __init__(self):
        super().__init__('mpc_node')
        self.params_cfg = {}
        # A full-race upper solve can take longer than the 1 Hz control period.
        # Keep telemetry and lower control responsive while that solve runs.
        self.telemetry_callback_group = MutuallyExclusiveCallbackGroup()
        self.upper_callback_group = MutuallyExclusiveCallbackGroup()
        self.lower_callback_group = MutuallyExclusiveCallbackGroup()
        self.command_callback_group = MutuallyExclusiveCallbackGroup()
```

```

        self.declare_parameter('passo_mode', False)
        self.passo_mode = bool(self.get_parameter('passo_mode').value)
        if self.passo_mode:
            self._init_passo()
        else:
            self._init_solar()

# ----- common helpers -----
def _load_stops(self, stop_yaml: str):
    self.stops = []
    try:
        with open(stop_yaml, 'r', encoding='utf-8') as f:
            y = yaml.safe_load(f) or {}
            self.stops = y.get('stops', [])
            self.get_logger().info(f'Loaded {len(self.stops)} stop points from {
stop_yaml}')
    except Exception:
        self.get_logger().info('No stop_points.yaml provided. Running without dwell
constraints.')

    def _load_forecast_file(self, path: str):
...

```

L49 関数 MPCNode.__init__

- 行範囲: L49–L63
- 所属: MPCNode
- 定義: __init__(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: MutuallyExclusiveCallbackGroup, __init__, _init_passo, _init_solar, bool, declare_parameter, get_parameter, super
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self._init_passo, self._init_solar, self.declare_parameter, self.get_parameter, self.passo_mode
- 更新する主なインスタンス属性: self.command_callback_group, self.lower_callback_group, self.params_cfg, self.passo_mode, self.telemetry_callback_group, self.upper_callback_group
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. super().__init__(...) を実行する。
2. self.params_cfg に {} の結果を代入する。

3. self.telemetry_callback_group に MutuallyExclusiveCallbackGroup() の結果を代入する。
4. self.upper_callback_group に MutuallyExclusiveCallbackGroup() の結果を代入する。
5. self.lower_callback_group に MutuallyExclusiveCallbackGroup() の結果を代入する。
6. self.command_callback_group に MutuallyExclusiveCallbackGroup() の結果を代入する。
7. self.declare_parameter(...) を実行する。
8. self.passo_mode に bool(self.get_parameter('passo_mode').value) の結果を代入する。
9. 条件 self.passo_mode を判定し、真なら内部処理を行う。
10. self._init_passo(...) を実行する。
11. 上の条件が偽の場合:
12. self._init_solar(...) を実行する。

代表コード断片

```
def __init__(self):
    super().__init__('mpc_node')
    self.params_cfg = {}
    # A full-race upper solve can take longer than the 1 Hz control period.
    # Keep telemetry and lower control responsive while that solve runs.
    self.telemetry_callback_group = MutuallyExclusiveCallbackGroup()
    self.upper_callback_group = MutuallyExclusiveCallbackGroup()
    self.lower_callback_group = MutuallyExclusiveCallbackGroup()
    self.command_callback_group = MutuallyExclusiveCallbackGroup()
    self.declare_parameter('passo_mode', False)
    self.passo_mode = bool(self.get_parameter('passo_mode').value)
    if self.passo_mode:
        self._init_passo()
    else:
        self._init_solar()
```

L66 関数 MPCNode._load_stops

- 行範囲: L66–L74
- 所属: MPCNode
- 定義: _load_stops(self, stop_yaml:str)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: get, get_logger, info, len, open, safe_load
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: f, y
- 読み取る主なインスタンス属性: self.get_logger, self.stops
- 更新する主なインスタンス属性: self.stops
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. self.stops に [] の結果を代入する。
2. 例外処理を伴う try ブロックを実行する。
3. with 文で open(stop_yaml, 'r', encoding='utf-8') を管理しながら処理する。
4. y に yaml.safe_load(f) or {} の結果を代入する。
5. self.stops に y.get('stops', []) の結果を代入する。
6. self.get_logger().info(...) を実行する。
7. Exception を捕捉した場合:
8. self.get_logger().info(...) を実行する。

代表コード断片

```
def _load_stops(self, stop_yaml: str):
    self.stops = []
    try:
        with open(stop_yaml, 'r', encoding='utf-8') as f:
            y = yaml.safe_load(f) or {}
            self.stops = y.get('stops', [])
            self.get_logger().info(f'Loaded {len(self.stops)} stop points from {
stop_yaml}')
    except Exception:
        self.get_logger().info('No stop_points.yaml provided. Running without dwell
constraints.')
```

L76 関数 MPCNode._load_forecast_file

- 行範囲: L76–L100
- 所属: MPCNode
- 定義: _load_forecast_file(self, path: str)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: Index, all, build_forecast_grid_payload, drop_duplicates, dropna, get_logger, getattr, isna, read_csv, sort_values, str, to_datetime
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: t, tzname
- 読み取る主なインスタンス属性: self.df, self.get_logger
- 更新する主なインスタンス属性: self.df, self.forecast_grid, self.forecast_times
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 5、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. self.df に pd.read_csv(path) の結果を代入する。
2. 条件 'time' in self.df.columns を判定し、真なら内部処理を行う。
3. t に pd.to_datetime(self.df['time'], format='mixed', errors='coerce') の結果を代入する。
4. tzname に str(getattr(self, 'forecast_time_tz', 'UTC') or 'UTC') の結果を代入する。
5. 条件 t.dt.tz is None を判定し、真なら内部処理を行う。
6. 条件 tzname.upper() == 'UTC' を判定し、真なら内部処理を行う。
7. t に t.dt.tz_localize('UTC') の結果を代入する。
8. 上の条件が偽の場合:
9. 例外処理を伴う try ブロックを実行する。
10. 上の条件が偽の場合:
11. t に t.dt.tz_convert('UTC') の結果を代入する。
12. self.df['time'] に t の結果を代入する。
13. 条件 self.df['time'].isna().all() を判定し、真なら内部処理を行う。
14. self.get_logger().warn(...) を実行する。
15. 上の条件が偽の場合:
16. self.get_logger().warn(...) を実行する。
17. self.forecast_grid に build_forecast_grid_payload(self.df) の結果を代入する。
18. 条件 'time' in self.df.columns and (not self.df['time'].isna().all()) を判定し、真なら内部処理を行う。
19. self.forecast_times に pd.Index(self.df['time'].dropna().drop_duplicates().sort_values()) の結果を代入する。
20. 上の条件が偽の場合:
21. self.forecast_times に pd.Index([]) の結果を代入する。

代表コード断片

```
def _load_forecast_file(self, path: str):
    self.df = pd.read_csv(path)
    if 'time' in self.df.columns:
        t = pd.to_datetime(self.df['time'], format='mixed', errors='coerce')
        tzname = str(getattr(self, 'forecast_time_tz', 'UTC') or 'UTC')
        if t.dt.tz is None:
            if tzname.upper() == 'UTC':
                t = t.dt.tz_localize('UTC')
            else:
                try:
                    t = t.dt.tz_localize(tzname, ambiguous='NaT', nonexistent='NaT').dt.
tz_convert('UTC')
                except Exception:
```

```

        t = t.dt.tz_localize('UTC')
    else:
        t = t.dt.tz_convert('UTC')
        self.df['time'] = t
        if self.df['time'].isna().all():
            self.get_logger().warn("forecast 'time' column could not be parsed; falling
back to index bins.")
        else:
            self.get_logger().warn("forecast CSV has no 'time' column; falling back to index
bins.")
            self.forecast_grid = build_forecast_grid_payload(self.df)
            if 'time' in self.df.columns and not self.df['time'].isna().all():
                self.forecast_times = pd.Index(self.df['time'].dropna().drop_duplicates().
sort_values())
            else:
                self.forecast_times = pd.Index([])

```

L102 関数 MPCNode._apply_params_yaml

- 行範囲: L102–L170
- 所属: MPCNode
- 定義: `_apply_params_yaml(self, path:str)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Parameter`, `append`, `dict`, `float`, `get`, `get_logger`, `get_parameter`, `has_parameter`, `hasattr`, `info`, `isinstance`, `items`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `cfg`, `f`, `k`, `key`, `model_cfg`, `motor_type`, `mpc_cfg`, `mt`, `params`, `runtime_mode`, `runtime_overrides`, `runtime_section`, `v`, `v_str`, `val`
- 読み取る主なインスタンス属性: `self.get_logger`, `self.get_parameter`, `self.has_parameter`, `self.model`, `self.set_parameters`
- 更新する主なインスタンス属性: `self.params_cfg`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 15、ループ 2、`try` 6

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `with` 文で `open(path, 'r', encoding='utf-8')` を管理しながら処理する。
3. `cfg` に `yaml.safe_load(f)` or `{}` の結果を代入する。
4. `Exception` を捕捉した場合:

5. `self.get_logger().warn(...)` を実行する。
6. を返す。
7. `self.params_cfg` に `cfg if isinstance(cfg, dict) else {}` の結果を代入する。
8. `model_cfg` に `cfg.get('model', cfg) if isinstance(cfg, dict) else {}` の結果を代入する。
9. 条件 `isinstance(model_cfg, dict)` を判定し、真なら内部処理を行う。
10. `motor_type` に `None` の結果を代入する。
11. `model_cfg.items()` を順に走査し、各要素を `(k, v)` に入れて処理する。
12. 条件 `hasattr(self.model.p, k)` を判定し、真なら内部処理を行う。
13. 例外処理を伴う `try` ブロックを実行する。
14. 条件 `k == 'motor_type'` を判定し、真なら内部処理を行う。
15. `motor_type` に `str(v)` の結果を代入する。
16. 条件 `k == 'drive_mode'` を判定し、真なら内部処理を行う。
17. `self.model.drive_mode` に `str(v)` の結果を代入する。
18. 条件 `k == 'drive_mode_tau_margin'` を判定し、真なら内部処理を行う。
19. 例外処理を伴う `try` ブロックを実行する。
20. 条件 `k == 'ocv_soc_map'` を判定し、真なら内部処理を行う。
21. 例外処理を伴う `try` ブロックを実行する。
22. 条件 `motor_type` を判定し、真なら内部処理を行う。
23. `mt` に `motor_type.lower()` の結果を代入する。
24. 条件 `mt in ('inwheel', 'hub')` を判定し、真なら内部処理を行う。
25. 条件 `not 'gear_ratio' in model_cfg` を判定し、真なら内部処理を行う。
26. 条件 `not 'gear_eta' in model_cfg` を判定し、真なら内部処理を行う。
27. `mpc_cfg` に `dict(cfg.get('mpc', {})) or {}` の結果を代入する。
28. `runtime_mode` に `str(self.get_parameter('profile_runtime_mode').value or "").strip()` の結果を代入する。
29. `runtime_section` に `cfg.get(runtime_mode, {})` if `runtime_mode` else `{}` の結果を代入する。
30. `runtime_overrides` に `runtime_section.get('mpc_overrides', {})` if `isinstance(runtime_section, dict)` else `{}` の結果を代入する。
31. 条件 `isinstance(runtime_overrides, dict) and runtime_overrides` を判定し、真なら内部処理を行う。
32. `mpc_cfg.update(...)` を実行する。
33. `self.get_logger().info(...)` を実行する。
34. 条件 `isinstance(mpc_cfg, dict)` を判定し、真なら内部処理を行う。
35. `params` に `[]` の結果を代入する。
36. `mpc_cfg.items()` を順に走査し、各要素を `(key, val)` に入れて処理する。
37. 条件 `isinstance(val, dict) or val is None or (not self.has_parameter(key))` を判定し、真なら内部処理を行う。

38. Continue 文を実行する。
39. 例外処理を伴う try ブロックを実行する。
40. params.append(...) を実行する。
41. (TypeError, ValueError) を捕捉した場合:
42. self.get_logger().warn(...) を実行する。
43. 条件 params を判定し、真なら内部処理を行う。
44. self.set_parameters(...) を実行する。

代表コード断片

```
def _apply_params_yaml(self, path: str):
    try:
        with open(path, 'r', encoding='utf-8') as f:
            cfg = yaml.safe_load(f) or {}
    except Exception as exc:
        self.get_logger().warn(f'params_yaml load failed: {exc}')
    return
self.params_cfg = cfg if isinstance(cfg, dict) else {}
model_cfg = cfg.get('model', cfg) if isinstance(cfg, dict) else {}
if isinstance(model_cfg, dict):
    motor_type = None
    for k, v in model_cfg.items():
        if hasattr(self.model.p, k):
            try:
                setattr(self.model.p, k, float(v))
            except Exception:
                try:
                    setattr(self.model.p, k, v)
                except Exception:
                    pass
        if k == 'motor_type':
            motor_type = str(v)
        if k == 'drive_mode':
            self.model.drive_mode = str(v)
        if k == 'drive_mode_tau_margin':
            try:
                self.model.drive_mode_tau_margin = float(v)
            except Exception:
                pass
    if k == 'ocv_soc_map':
        try:
            v_str = str(v).strip()
            if v_str:
                self.model.load_ocv_map(resolve_path(v_str, 'maps'))
        except Exception:
            ...
```

L172 関数 MPCNode._maybe_reload_forecast

- 行範囲: L172–L190
- 所属: MPCNode
- 定義: _maybe_reload_forecast(self)
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `_load_forecast_file`, `get_logger`, `getmtime`, `info`, `monotonic`, `warn`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `mtime`, `now`
- 読み取る主なインスタンス属性: `self._load_forecast_file`, `self.forecast_mtime`, `self.forecast_path`, `self.forecast_reload_sec`, `self.get_logger`, `self.last_forecast_check`
- 更新する主なインスタンス属性: `self.forecast_mtime`, `self.forecast_reloaded`, `self.last_forecast_check`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 2

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.forecast_reload_sec <= 0` を判定し、真なら内部処理を行う。
2. を返す。
3. `now` に `time.monotonic()` の結果を代入する。
4. 条件 `now - self.last_forecast_check < self.forecast_reload_sec` を判定し、真なら内部処理を行う。
5. を返す。
6. `self.last_forecast_check` に `now` の結果を代入する。
7. 例外処理を伴う `try` ブロックを実行する。
8. `mtime` に `os.path.getmtime(self.forecast_path)` の結果を代入する。
9. `Exception` を捕捉した場合:
10. を返す。
11. 条件 `self.forecast_mtime is None or mtime > self.forecast_mtime` を判定し、真なら内部処理を行う。
12. `self.forecast_mtime` に `mtime` の結果を代入する。
13. 例外処理を伴う `try` ブロックを実行する。
14. `self._load_forecast_file(...)` を実行する。
15. `self.forecast_reloaded` に `True` の結果を代入する。
16. `self.get_logger().info(...)` を実行する。
17. `Exception` を捕捉した場合:
18. `self.get_logger().warn(...)` を実行する。

代表コード断片

```
def _maybe_reload_forecast(self):
    if self.forecast_reload_sec <= 0:
        return
    now = time.monotonic()
    if (now - self.last_forecast_check) < self.forecast_reload_sec:
        return
    self.last_forecast_check = now
    try:
        mtime = os.path.getmtime(self.forecast_path)
    except Exception:
        return
    if self.forecast_mtime is None or mtime > self.forecast_mtime:
        self.forecast_mtime = mtime
    try:
        self._load_forecast_file(self.forecast_path)
        self.forecast_reloaded = True
        self.get_logger().info('Forecast CSV reloaded.')
    except Exception as exc:
        self.get_logger().warn(f'Forecast reload failed: {exc}')
```

L192 関数 MPCNode._on_s_km_solar

- 行範囲: L192–L206
- 所属: MPCNode
- 定義: `_on_s_km_solar(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool`, `finite_float`, `float`, `get_clock`, `get_parameter`, `isfinite`, `now`, `update`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `current`, `now_sec`, `raw`
- 読み取る主なインスタンス属性: `self.distance_meas_filter`, `self.distance_meas_max_backtrack_km`, `self.get_clock`, `self.get_parameter`, `self.s_meas`
- 更新する主なインスタンス属性: `self.s_km`, `self.s_meas`, `self.s_meas_time`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `now_sec` に `self.get_clock().now().nanoseconds / 1000000000.0` の結果を代入する。
3. `raw` に `finite_float(msg.data)` の結果を代入する。

4. 条件 `not math.isfinite(raw)` を判定し、真なら内部処理を行う。
5. を返す。
6. `current` に `float(self.distance_meas_filter.value)` の結果を代入する。
7. 条件 `math.isfinite(current) and raw < current - self.distance_meas_max_backtrack_km` を判定し、真なら内部処理を行う。
8. を返す。
9. `self.s_meas` に `float(self.distance_meas_filter.update(raw, now=now_sec))` の結果を代入する。
10. `self.s_meas_time` に `now_sec` の結果を代入する。
11. 条件 `bool(self.get_parameter('use_measured_s').value)` を判定し、真なら内部処理を行う。
12. `self.s_km` に `float(self.s_meas)` の結果を代入する。
13. Exception を捕捉した場合:
14. Pass 文を実行する。

代表コード断片

```
def _on_s_km_solar(self, msg: Float32):
    try:
        now_sec = self.get_clock().now().nanoseconds / 1e9
        raw = finite_float(msg.data)
        if not math.isfinite(raw):
            return
        current = float(self.distance_meas_filter.value)
        if math.isfinite(current) and raw < current - self.
distance_meas_max_backtrack_km:
            return
        self.s_meas = float(self.distance_meas_filter.update(raw, now=now_sec))
        self.s_meas_time = now_sec
        if bool(self.get_parameter('use_measured_s').value):
            self.s_km = float(self.s_meas)
    except Exception:
        pass
```

L208 関数 MPCNode._on_speed_solar

- 行範囲: L208–L214
- 所属: MPCNode
- 定義: `_on_speed_solar(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `get_clock`, `now`, `update`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `now_sec`
- 読み取る主なインスタンス属性: `self.get_clock`, `self.speed_meas_filter`
- 更新する主なインスタンス属性: `self.v_meas_time`, `self.v_now`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `now_sec` に `self.get_clock().now().nanoseconds / 1000000000.0` の結果を代入する。
3. `self.v_now` に `float(self.speed_meas_filter.update(msg.data, now=now_sec))` の結果を代入する。
4. `self.v_meas_time` に `now_sec` の結果を代入する。
5. `Exception` を捕捉した場合:
6. `Pass` 文を実行する。

代表コード断片

```
def _on_speed_solar(self, msg: Float32):
    try:
        now_sec = self.get_clock().now().nanoseconds / 1e9
        self.v_now = float(self.speed_meas_filter.update(msg.data, now=now_sec))
        self.v_meas_time = now_sec
    except Exception:
        pass
```

L216 関数 `MPCNode._on_soc_solar`

- 行範囲: L216–L223
- 所属: `MPCNode`
- 定義: `_on_soc_solar(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `float`, `get_clock`, `now`, `update`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `now_sec`
- 読み取る主なインスタンス属性: `self.get_clock`, `self.model`, `self.soc_meas_filter`, `self.solar_soc_meas`
- 更新する主なインスタンス属性: `self.soc_meas_time`, `self.solar_soc_meas`, `self.z`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. now_sec に self.get_clock().now().nanoseconds / 1000000000.0 の結果を代入する。
3. self.solar_soc_meas に float(self.soc_meas_filter.update(msg.data, now=now_sec)) の結果を代入する。
4. self.soc_meas_time に now_sec の結果を代入する。
5. self.z に float(np.clip(self.solar_soc_meas, self.model.p.soc_min, self.model.p.soc_max)) の結果を代入する。
6. Exception を捕捉した場合:
7. Pass 文を実行する。

代表コード断片

```
def _on_soc_solar(self, msg: Float32):
    try:
        now_sec = self.get_clock().now().nanoseconds / 1e9
        self.solar_soc_meas = float(self.soc_meas_filter.update(msg.data, now=now_sec))
        self.soc_meas_time = now_sec
        self.z = float(np.clip(self.solar_soc_meas, self.model.p.soc_min, self.model.p.soc_max))
    except Exception:
        pass
```

L225 関数 MPCNode._on_tb_solar

- 行範囲: L225–L232
- 所属: MPCNode
- 定義: _on_tb_solar(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: clip, float, get_clock, now, update
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: now_sec
- 読み取る主なインスタンス属性: self.get_clock, self.model, self.solar_tb_meas, self.tb_meas_filter
- 更新する主なインスタンス属性: self.Tb, self.solar_tb_meas, self.tb_meas_time
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. now_sec に self.get_clock().now().nanoseconds / 1000000000.0 の結果を代入する。
3. self.solar_tb_meas に float(self.tb_meas_filter.update(msg.data, now=now_sec)) の結果を代入する。
4. self.tb_meas_time に now_sec の結果を代入する。
5. self.Tb に float(np.clip(self.solar_tb_meas, self.model.p.T_min, self.model.p.T_max)) の結果を代入する。
6. Exception を捕捉した場合:
7. Pass 文を実行する。

代表コード断片

```
def _on_tb_solar(self, msg: Float32):
    try:
        now_sec = self.get_clock().now().nanoseconds / 1e9
        self.solar_tb_meas = float(self.tb_meas_filter.update(msg.data, now=now_sec))
        self.tb_meas_time = now_sec
        self.Tb = float(np.clip(self.solar_tb_meas, self.model.p.T_min, self.model.p.
T_max))
    except Exception:
        pass
```

L234 関数 MPCNode._on_i_solar

- 行範囲: L234–L240
- 所属: MPCNode
- 定義: _on_i_solar(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, get_clock, now, update
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: now_sec
- 読み取る主なインスタンス属性: self.current_meas_filter, self.get_clock
- 更新する主なインスタンス属性: self.current_meas_time, self.solar_i_meas
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. now_sec に self.get_clock().now().nanoseconds / 1000000000.0 の結果を代入する。
3. self.solar_i_meas に float(self.current_meas_filter.update(msg.data, now=now_sec)) の結果を代入する。
4. self.current_meas_time に now_sec の結果を代入する。
5. Exception を捕捉した場合:
6. Pass 文を実行する。

代表コード断片

```
def _on_i_solar(self, msg: Float32):
    try:
        now_sec = self.get_clock().now().nanoseconds / 1e9
        self.solar_i_meas = float(self.current_meas_filter.update(msg.data, now=now_sec))
    )
    self.current_meas_time = now_sec
    except Exception:
        pass
```

L242 関数 MPCNode._on_v_solar

- 行範囲: L242–L248
- 所属: MPCNode
- 定義: _on_v_solar(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, get_clock, now, update
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: now_sec
- 読み取る主なインスタンス属性: self.get_clock, self.voltage_meas_filter
- 更新する主なインスタンス属性: self.solar_v_meas, self.voltage_meas_time
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. now_sec に self.get_clock().now().nanoseconds / 1000000000.0 の結果を代入する。
3. self.solar_v_meas に float(self.voltage_meas_filter.update(msg.data, now=now_sec)) の結果を代入する。
4. self.voltage_meas_time に now_sec の結果を代入する。
5. Exception を捕捉した場合:
6. Pass 文を実行する。

代表コード断片

```
def _on_v_solar(self, msg: Float32):
    try:
        now_sec = self.get_clock().now().nanoseconds / 1e9
        self.solar_v_meas = float(self.voltage_meas_filter.update(msg.data, now=now_sec))
    )
    self.voltage_meas_time = now_sec
    except Exception:
        pass
```

L250 関数 MPCNode._measured_distance_km

- 行範囲: L250–L254
- 所属: MPCNode
- 定義: _measured_distance_km(self)->float
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: finite_float, fresh_enough, get_clock, now
- 戻り値の要点: math.nan / finite_float(self.s_meas)
- 主なローカル代入名: now_sec
- 読み取る主なインスタンス属性: self.distance_meas_timeout_sec, self.get_clock, self.s_meas, self.s_meas_time
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. now_sec に self.get_clock().now().nanoseconds / 1000000000.0 の結果を代入する。
2. 条件 fresh_enough(self.s_meas_time, now_sec, self.distance_meas_timeout_sec) を判定し、真なら内部処理を行う。
3. finite_float(self.s_meas) を返す。
4. math.nan を返す。

代表コード断片

```
def _measured_distance_km(self) -> float:
    now_sec = self.get_clock().now().nanoseconds / 1e9
    if fresh_enough(self.s_meas_time, now_sec, self.distance_meas_timeout_sec):
        return finite_float(self.s_meas)
    return math.nan
```

L256 関数 MPCNode._sync_measured_state

- 行範囲: L256–L268
- 所属: MPCNode
- 定義: `_sync_measured_state(self)`->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_measured_distance_km`, `bool`, `clip`, `float`, `fresh_enough`, `get_clock`, `get_parameter`, `isfinite`, `now`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `distance_km`, `now_sec`
- 読み取る主なインスタンス属性: `self._measured_distance_km`, `self.battery_meas_timeout_sec`, `self.get_clock`, `self.get_parameter`, `self.model`, `self.soc_meas_time`, `self.solar_soc_meas`, `self.solar_tb_meas`, `self.speed_meas_timeout_sec`, `self.tb_meas_time`, `self.v_meas_time`
- 更新する主なインスタンス属性: `self.Tb`, `self.s_km`, `self.v_now`, `self.z`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `now_sec` に `self.get_clock().now().nanoseconds / 1000000000.0` の結果を代入する。
2. 条件 `not fresh_enough(self.v_meas_time, now_sec, self.speed_meas_timeout_sec)` を判定し、真なら内部処理を行う。
3. `self.v_now` に `math.nan` の結果を代入する。
4. `distance_km` に `self._measured_distance_km()` の結果を代入する。
5. 条件 `bool(self.get_parameter('use_measured_s').value) and math.isfinite(distance_km)` を判定し、真なら内部処理を行う。
6. `self.s_km` に `float(distance_km)` の結果を代入する。
7. 条件 `fresh_enough(self.soc_meas_time, now_sec, self.battery_meas_timeout_sec)` を判定し、真なら内部処理を行う。
8. 条件 `math.isfinite(self.solar_soc_meas)` を判定し、真なら内部処理を行う。

9. self.z に float(np.clip(self.solar_soc_meas, self.model.p.soc_min, self.model.p.soc_max)) の結果を代入する。
10. 条件 fresh_enough(self.tb_meas_time, now_sec, self.battery_meas_timeout_sec) を判定し、真なら内部処理を行う。
11. 条件 math.isfinite(self.solar_tb_meas) を判定し、真なら内部処理を行う。
12. self.Tb に float(np.clip(self.solar_tb_meas, self.model.p.T_min, self.model.p.T_max)) の結果を代入する。

代表コード断片

```
def _sync_measured_state(self) -> None:
    now_sec = self.get_clock().now().nanoseconds / 1e9
    if not fresh_enough(self.v_meas_time, now_sec, self.speed_meas_timeout_sec):
        self.v_now = math.nan
    distance_km = self._measured_distance_km()
    if bool(self.get_parameter('use_measured_s').value) and math.isfinite(distance_km):
        self.s_km = float(distance_km)
    if fresh_enough(self.soc_meas_time, now_sec, self.battery_meas_timeout_sec):
        if math.isfinite(self.solar_soc_meas):
            self.z = float(np.clip(self.solar_soc_meas, self.model.p.soc_min, self.model.p.soc_max))
        if fresh_enough(self.tb_meas_time, now_sec, self.battery_meas_timeout_sec):
            if math.isfinite(self.solar_tb_meas):
                self.Tb = float(np.clip(self.solar_tb_meas, self.model.p.T_min, self.model.p.T_max))
```

L270 関数 MPCNode._on_calib_solar_gain

- 行範囲: L270–L274
- 所属: MPCNode
- 定義: _on_calib_solar_gain(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: clip, float
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.solar_gain
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. self.solar_gain に float(np.clip(float(msg.data), 0.2, 2.5)) の結果を代入する。
3. Exception を捕捉した場合:
4. を返す。

代表コード断片

```
def _on_calib_solar_gain(self, msg: Float32):
    try:
        self.solar_gain = float(np.clip(float(msg.data), 0.2, 2.5))
    except Exception:
        return
```

L276 関数 MPCNode._on_calib_drive_power_gain

- 行範囲: L276–L282
- 所属: MPCNode
- 定義: _on_calib_drive_power_gain(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: clip, float, max
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: gain
- 読み取る主なインスタンス属性: self.base_drive_eff_scale, self.base_regen_eff_scale, self.model
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. gain に float(np.clip(float(msg.data), 0.4, 2.5)) の結果を代入する。
3. self.model.p.drive_eff_scale に float(self.base_drive_eff_scale) / max(gain, 0.001) の結果を代入する。
4. self.model.p.regen_eff_scale に float(self.base_regen_eff_scale) / max(gain, 0.001) の結果を代入する。
5. Exception を捕捉した場合:
6. を返す。

代表コード断片

```
def _on_calib_drive_power_gain(self, msg: Float32):
    try:
        gain = float(np.clip(float(msg.data), 0.4, 2.5))
        self.model.p.drive_eff_scale = float(self.base_drive_eff_scale) / max(gain, 1.0e
-3)
        self.model.p.regen_eff_scale = float(self.base_regen_eff_scale) / max(gain, 1.0e
-3)
    except Exception:
        return
```

L284 関数 MPCNode._on_calib_aux_power

- 行範囲: L284–L288
- 所属: MPCNode
- 定義: `_on_calib_aux_power(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.model`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `self.model.aux_power_override_w` に `max(0.0, float(msg.data))` の結果を代入する。
3. `Exception` を捕捉した場合:
4. を返す。

代表コード断片

```
def _on_calib_aux_power(self, msg: Float32):
    try:
        self.model.aux_power_override_w = max(0.0, float(msg.data))
    except Exception:
        return
```

L291 関数 MPCNode._init_solar

- 行範囲: L291–L778
- 所属: MPCNode
- 定義: `_init_solar(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `BatteryMHE`, `Params`, `RobustScalarFilter`, `SolarCarModel`, `_apply_params_yaml`, `_load_forecast_file`, `_load_stops`, `abs`, `astimezone`, `bool`, `clip`, `create_publisher`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `battery_tau`, `drive_map`, `drive_map_eco`, `drive_map_power`, `drive_schedule_yaml`, `expected_dt`, `fcsv`, `initial_policy_csv`, `maps_dir`, `mppt_map`, `ocv_map`, `panel_map`, `params_yaml`, `regen_map`, `regen_map_eco`, `regen_map_power`, `rint_map`, `route_profile_csv`, `speed_profile_csv`, `start_time_utc`
- 読み取る主なインスタンス属性: `self._apply_params_yaml`, `self._load_forecast_file`, `self._load_stops`, `self._on_calib_a`, `self._on_calib_drive_power_gain`, `self._on_calib_solar_gain`, `self._on_i_solar`, `self._on_s_km_solar`, `self._on_soc_solar`, `self._on_speed_solar`, `self._on_tb_solar`, `self._on_v_solar`, `self._publish_lower_command_cycle`, `self._step_lower`, `self._step_solar`, `self._warned_lower_rate`, `self.base_drive_eff_scale`, `self.command_callback_group`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.drive_schedule`, `self.dt`
- 更新する主なインスタンス属性: `self.Np`, `self.Tb`, `self._forecast_warned_out_of_range`, `self._lower_solve_logged`, `self._lower_upper_busy_logged`, `self._upper_solve_in_progress`, `self._upper_solve_logged`, `self._warned_lower_rate`, `self.active_control_stop_index`, `self.base_drive_eff_scale`, `self.base_regen_eff_scale`, `self.battery_meas_timeout_sec`, `self.completed_control_stops`, `self.control_stop_end_monotonic`, `self.control_stop_hold`, `self.control_stop_position_initial`, `self.control_stop_tilt_fraction`, `self.current_forecast_time_utc`, `self.current_meas_filter`, `self.current_meas_time`, `self.distance_meas_filter`, `self.distance_meas_max_backtrack_km`, `self.distance_meas_timeout_sec`, `self.drive_schedule`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 15、ループ 0、`try` 5

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `self.declare_parameter(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。

10. self.declare_parameter(...) を実行する。
11. self.declare_parameter(...) を実行する。
12. self.declare_parameter(...) を実行する。
13. self.declare_parameter(...) を実行する。
14. self.declare_parameter(...) を実行する。
15. self.declare_parameter(...) を実行する。
16. self.declare_parameter(...) を実行する。
17. self.declare_parameter(...) を実行する。
18. self.declare_parameter(...) を実行する。
19. self.declare_parameter(...) を実行する。
20. self.declare_parameter(...) を実行する。
21. self.declare_parameter(...) を実行する。
22. self.declare_parameter(...) を実行する。
23. self.declare_parameter(...) を実行する。
24. self.declare_parameter(...) を実行する。
25. self.declare_parameter(...) を実行する。
26. self.declare_parameter(...) を実行する。
27. self.declare_parameter(...) を実行する。
28. self.declare_parameter(...) を実行する。
29. self.declare_parameter(...) を実行する。
30. self.declare_parameter(...) を実行する。
31. self.declare_parameter(...) を実行する。
32. self.declare_parameter(...) を実行する。
33. self.declare_parameter(...) を実行する。
34. self.declare_parameter(...) を実行する。
35. self.declare_parameter(...) を実行する。
36. self.declare_parameter(...) を実行する。
37. self.declare_parameter(...) を実行する。
38. self.declare_parameter(...) を実行する。
39. self.declare_parameter(...) を実行する。
40. self.declare_parameter(...) を実行する。
41. self.declare_parameter(...) を実行する。
42. self.declare_parameter(...) を実行する。
43. self.declare_parameter(...) を実行する。

44. self.declare_parameter(...) を実行する。
45. self.declare_parameter(...) を実行する。
46. self.declare_parameter(...) を実行する。
47. self.declare_parameter(...) を実行する。
48. self.declare_parameter(...) を実行する。
49. self.declare_parameter(...) を実行する。
50. self.declare_parameter(...) を実行する。
51. self.declare_parameter(...) を実行する。
52. self.declare_parameter(...) を実行する。
53. self.declare_parameter(...) を実行する。
54. self.declare_parameter(...) を実行する。
55. self.declare_parameter(...) を実行する。
56. self.declare_parameter(...) を実行する。
57. self.declare_parameter(...) を実行する。
58. self.declare_parameter(...) を実行する。
59. self.declare_parameter(...) を実行する。
60. self.declare_parameter(...) を実行する。
61. self.declare_parameter(...) を実行する。
62. self.declare_parameter(...) を実行する。
63. self.declare_parameter(...) を実行する。
64. self.declare_parameter(...) を実行する。
65. self.declare_parameter(...) を実行する。
66. self.declare_parameter(...) を実行する。
67. self.declare_parameter(...) を実行する。
68. self.declare_parameter(...) を実行する。
69. self.declare_parameter(...) を実行する。
70. self.declare_parameter(...) を実行する。
71. self.declare_parameter(...) を実行する。
72. self.declare_parameter(...) を実行する。
73. self.declare_parameter(...) を実行する。
74. self.declare_parameter(...) を実行する。
75. self.declare_parameter(...) を実行する。
76. self.declare_parameter(...) を実行する。
77. self.declare_parameter(...) を実行する。

78. self.declare_parameter(...) を実行する。

79. self.declare_parameter(...) を実行する。

80. self.declare_parameter(...) を実行する。

代表コード断片

```
def _init_solar(self):
    self.declare_parameter('forecast_csv', 'inputs/forecast_10min.csv')
    self.declare_parameter('maps_dir', 'maps')
    self.declare_parameter('drive_eff_map', '')
    self.declare_parameter('regen_eff_map', '')
    self.declare_parameter('rint_map', '')
    self.declare_parameter('drive_map_eco', '')
    self.declare_parameter('drive_map_power', '')
    self.declare_parameter('regen_map_eco', '')
    self.declare_parameter('regen_map_power', '')
    self.declare_parameter('panel_eff_map', '')
    self.declare_parameter('mppt_eff_map', '')
    self.declare_parameter('ocv_soc_map', '')
    self.declare_parameter('dt', 600.0) # 10 min [s]
    self.declare_parameter('horizon_steps', 9)
    self.declare_parameter('v_max_kmh', 110.0)
    self.declare_parameter('terminal_soc_min', 0.10)
    self.declare_parameter('stop_yaml', 'inputs/stop_points.yaml')
    self.declare_parameter('forecast_time_mode', 'auto') # auto/absolute/relative/loop
    self.declare_parameter('forecast_time_tz', 'UTC')
    self.declare_parameter('forecast_start_time_utc', '')
    self.declare_parameter('forecast_time_offset_sec', 0.0)
    self.declare_parameter('forecast_reload_sec', 60.0)
    self.declare_parameter('replan_on_forecast_reload', False)
    self.declare_parameter('params_yaml', '')
    self.declare_parameter('profile_runtime_mode', '')
    self.declare_parameter('route_profile_csv', '')
    self.declare_parameter('speed_profile_csv', '')
    self.declare_parameter('drive_schedule_yaml', '')
    self.declare_parameter('initial_upper_policy_csv', '')
    self.declare_parameter('use_measured_s', True)
    self.declare_parameter('use_measured_speed', True)
    self.declare_parameter('soc0', 0.95)
    self.declare_parameter('Tb0', 30.0)
    self.declare_parameter('s0_kmh', 0.0)
    ...
```

L780 関数 MPCNode._current_bin_index

- 行範囲: L780–L828
- 所属: MPCNode
- 定義: `_current_bin_index(self)->int`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `datetime64`, `get_logger`, `getattr`, `int`, `len`, `lower`, `max`, `now`, `searchsorted`, `str`, `timedelta`
- 戻り値の要点: `int(np.clip(self.k, 0, max(0, bin_count - 1))) / int(idx % bin_count)` if mode == 'loop' else `int(np.clip(idx, 0, bin_count - 1)) / int(np.clip(idx, 0, len(self.forecast_times) - 1)) / 0`
- 主なローカル代入名: `bin_count`, `elapsed`, `has_time`, `idx`, `lookup_time`, `loop_sec`, `mode`, `now`, `t_first`, `t_last`, `t_max`, `t_min`, `t_series`

- 読み取る主なインスタンス属性: `self.df`, `self.dt`, `self.forecast_relative_wall_start`, `self.forecast_start_time`, `self.forecast_time_mode`, `self.forecast_time_offset`, `self.forecast_times`, `self.get_logger`, `self.k`
- 更新する主なインスタンス属性: `self._forecast_warned_out_of_range`, `self.current_forecast_time_utc`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 8、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `mode` に `str(self.forecast_time_mode).lower()` の結果を代入する。
2. `has_time` に `len(getattr(self, 'forecast_times', [])) > 0` の結果を代入する。
3. 条件 `mode == 'auto'` を判定し、真なら内部処理を行う。
4. `mode` に `'absolute' if has_time else 'relative'` の結果を代入する。
5. `now` に `datetime.now(timezone.utc) + timedelta(seconds=self.forecast_time_offset)` の結果を代入する。
6. `lookup_time` に `now` の結果を代入する。
7. 条件 `mode == 'absolute' and has_time` を判定し、真なら内部処理を行う。
8. `t_series` に `self.forecast_times.values` の結果を代入する。
9. `t_min` に `self.forecast_times[0]` の結果を代入する。
10. `t_max` に `self.forecast_times[-1]` の結果を代入する。
11. 条件 `now < t_min or now > t_max` を判定し、真なら内部処理を行う。
12. 条件 `not getattr(self, '_forecast_warned_out_of_range', False)` を判定し、真なら内部処理を行う。
13. `self.get_logger().warn(...)` を実行する。
14. `self._forecast_warned_out_of_range` に `True` の結果を代入する。
15. `mode` に `'relative'` の結果を代入する。
16. 上の条件が偽の場合:
17. `idx` に `int(np.searchsorted(t_series, np.datetime64(now)) - 1)` の結果を代入する。
18. `self.current_forecast_time_utc` に `now` の結果を代入する。
19. `int(np.clip(idx, 0, len(self.forecast_times) - 1))` を返す。
20. 条件 `mode in ('relative', 'loop') or not has_time` を判定し、真なら内部処理を行う。
21. `elapsed` に `(now - self.forecast_relative_wall_start).total_seconds()` の結果を代入する。
22. `elapsed` に `max(0.0, elapsed)` の結果を代入する。
23. `bin_count` に `len(self.forecast_times) if has_time else len(self.df)` の結果を代入する。
24. 条件 `bin_count == 0` を判定し、真なら内部処理を行う。

25. 0 を返す。
26. 条件 `has_time` を判定し、真なら内部処理を行う。
27. `lookup_time` に `self.forecast_start_time + timedelta(seconds=elapsed)` の結果を代入する。
28. 条件 `mode == 'loop' and len(self.forecast_times) > 1` を判定し、真なら内部処理を行う。
29. `t_first` に `self.forecast_times[0].to_pydatetime()` の結果を代入する。
30. `t_last` に `self.forecast_times[-1].to_pydatetime()` の結果を代入する。
31. `loop_sec` に `max(1.0, (t_last - t_first).total_seconds())` の結果を代入する。
32. `lookup_time` に `t_first + timedelta(seconds=(lookup_time - t_first).total_seconds() % loop_sec)` の結果を代入する。
33. `self.current_forecast_time_utc` に `lookup_time` の結果を代入する。
34. `idx` に `int(np.searchsorted(self.forecast_times.values, np.datetime64(lookup_time)) - 1)` の結果を代入する。
35. `int(np.clip(idx, 0, bin_count - 1))` を返す。
36. `idx` に `int(elapsed / max(self.dt, 0.001))` の結果を代入する。
37. `self.current_forecast_time_utc` に `self.forecast_start_time + timedelta(seconds=elapsed)` の結果を代入する。
38. `int(idx % bin_count)` if `mode == 'loop'` else `int(np.clip(idx, 0, bin_count - 1))` を返す。
39. `bin_count` に `len(self.forecast_times)` if `has_time` else `len(self.df)` の結果を代入する。
40. `int(np.clip(self.k, 0, max(0, bin_count - 1)))` を返す。

代表コード断片

```
def _current_bin_index(self) -> int:
    mode = str(self.forecast_time_mode).lower()
    has_time = len(getattr(self, 'forecast_times', [])) > 0
    if mode == 'auto':
        mode = 'absolute' if has_time else 'relative'

    now = datetime.now(timezone.utc) + timedelta(seconds=self.forecast_time_offset)
    lookup_time = now

    if mode == 'absolute' and has_time:
        t_series = self.forecast_times.values
        t_min = self.forecast_times[0]
        t_max = self.forecast_times[-1]
        if (now < t_min) or (now > t_max):
            if not getattr(self, '_forecast_warned_out_of_range', False):
                self.get_logger().warn('Forecast time out of range; switching to
relative indexing.')
                self._forecast_warned_out_of_range = True
                mode = 'relative'
        else:
            idx = int(np.searchsorted(t_series, np.datetime64(now)) - 1)
            self.current_forecast_time_utc = now
            return int(np.clip(idx, 0, len(self.forecast_times) - 1))

    if mode in ('relative', 'loop') or not has_time:
        elapsed = (now - self.forecast_relative_wall_start).total_seconds()
        elapsed = max(0.0, elapsed)
        bin_count = len(self.forecast_times) if has_time else len(self.df)
        if bin_count == 0:
            return 0
```

```

        if has_time:
            lookup_time = self.forecast_start_time + timedelta(seconds=elapsed)
            if mode == 'loop' and len(self.forecast_times) > 1:
                t_first = self.forecast_times[0].to_pydatetime()
                t_last = self.forecast_times[-1].to_pydatetime()
                loop_sec = max(1.0, (t_last - t_first).total_seconds())
...

```

L830 関数 MPCNode._horizon_data

- 行範囲: L830–L870
- 所属: MPCNode
- 定義: `_horizon_data(self, k0:int)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_forecast_at_time`, `append`, `bool`, `clip`, `datetime64`, `float`, `get`, `get_parameter`, `getattr`, `int`, `isfinite`, `len`
- 戻り値の要点: `data / data`
- 主なローカル代入名: `N`, `Np`, `data`, `env`, `has_time`, `i`, `is_drive`, `j`, `limits`, `loop_sec`, `mode`, `row`, `row_match`, `s0_km`, `s_query_km`, `speed_guess_kmh`, `t_first`, `t_last`, `t_utc`
- 読み取る主なインスタンス属性: `self.Np`, `self._forecast_at_time`, `self.current_forecast_time_utc`, `self.df`, `self.drive_schedule`, `self.dt`, `self.forecast_start_time`, `self.forecast_time_mode`, `self.forecast_times`, `self.get_parameter`, `self.s_km`, `self.s_meas`, `self.v_upper_cmd`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `data` に `[]` の結果を代入する。
2. `has_time` に `len(getattr(self, 'forecast_times', [])) > 0` の結果を代入する。
3. `N` に `len(self.forecast_times) if has_time else len(self.df)` の結果を代入する。
4. `mode` に `str(self.forecast_time_mode).lower()` の結果を代入する。
5. 条件 `mode == 'auto'` を判定し、真なら内部処理を行う。
6. `mode` に `'absolute' if has_time else 'relative'` の結果を代入する。
7. 条件 `N <= 0` を判定し、真なら内部処理を行う。
8. `data` を返す。
9. `Np` に `max(1, self.Np)` の結果を代入する。
10. `s0_km` に `float(self.s_km)` の結果を代入する。

11. 条件 `bool(self.get_parameter('use_measured_s').value)` and `np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
12. `s0_km` に `float(self.s_meas)` の結果を代入する。
13. `speed_guess_kmh` に `max(0.0, float(self.v_upper_cmd))` の結果を代入する。
14. `range(Np)` を順に走査し、各要素を `i` に入れて処理する。
15. 条件 `has_time` を判定し、真なら内部処理を行う。
16. `t_utc` に `self.current_forecast_time_utc + timedelta(seconds=self.dt * i)` の結果を代入する。
17. 条件 `mode == 'loop'` and `len(self.forecast_times) > 1` を判定し、真なら内部処理を行う。
18. `t_first` に `self.forecast_times[0].to_pydatetime()` の結果を代入する。
19. `t_last` に `self.forecast_times[-1].to_pydatetime()` の結果を代入する。
20. `loop_sec` に `max(1.0, (t_last - t_first).total_seconds())` の結果を代入する。
21. `t_utc` に `t_first + timedelta(seconds=(t_utc - t_first).total_seconds() % loop_sec)` の結果を代入する。
22. `j` に `int(np.searchsorted(self.forecast_times.values, np.datetime64(t_utc)) - 1)` の結果を代入する。
23. `j` に `int(np.clip(j, 0, N - 1))` の結果を代入する。
24. `row_match` に `self.df.loc[self.df['time'] == self.forecast_times[j]]` の結果を代入する。
25. `row` に `row_match.iloc[0]` if not `row_match.empty` else `self.df.iloc[0]` の結果を代入する。
26. 上の条件が偽の場合:
27. `j` に `(k0 + i) % N` if `mode == 'loop'` else `min(k0 + i, N - 1)` の結果を代入する。
28. `row` に `self.df.iloc[j]` の結果を代入する。
29. `t_utc` に `self.forecast_start_time + timedelta(seconds=self.dt * (k0 + i))` の結果を代入する。
30. `s_query_km` に `s0_km + speed_guess_kmh * self.dt * i / 3600.0` の結果を代入する。
31. `is_drive` に `True` の結果を代入する。
32. 条件 `self.drive_schedule is not None` を判定し、真なら内部処理を行う。
33. `limits` に `self.drive_schedule.speed_limits(t_utc)` の結果を代入する。
34. 条件 `limits is not None` and `limits[1] <= 0.0` を判定し、真なら内部処理を行う。
35. `is_drive` に `False` の結果を代入する。
36. `env` に `self._forecast_at_time(t_utc, s_query_km, drive=is_drive)` の結果を代入する。
37. `env['slope_pct']` に `float(row.get('slope_pct', 0.0))` の結果を代入する。
38. `env['t_utc']` に `t_utc` の結果を代入する。
39. `data.append(...)` を実行する。
40. `data` を返す。

代表コード断片

```
def _horizon_data(self, k0: int):
    data = []
    has_time = len(getattr(self, 'forecast_times', [])) > 0
    N = len(self.forecast_times) if has_time else len(self.df)
    mode = str(self.forecast_time_mode).lower()
    if mode == 'auto':
        mode = 'absolute' if has_time else 'relative'
    if N <= 0:
        return data
    Np = max(1, self.Np)
    s0_km = float(self.s_km)
    if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
        s0_km = float(self.s_meas)
    speed_guess_kmh = max(0.0, float(self.v_upper_cmd))
    for i in range(Np):
        if has_time:
            t_utc = self.current_forecast_time_utc + timedelta(seconds=self.dt * i)
            if mode == 'loop' and len(self.forecast_times) > 1:
                t_first = self.forecast_times[0].to_pydatetime()
                t_last = self.forecast_times[-1].to_pydatetime()
                loop_sec = max(1.0, (t_last - t_first).total_seconds())
                t_utc = t_first + timedelta(seconds=(t_utc - t_first).total_seconds() %
loop_sec)
            j = int(np.searchsorted(self.forecast_times.values, np.datetime64(t_utc)) -
1)
            j = int(np.clip(j, 0, N - 1))
            row_match = self.df.loc[self.df['time'] == self.forecast_times[j]]
            row = row_match.iloc[0] if not row_match.empty else self.df.iloc[0]
        else:
            j = (k0 + i) % N if mode == 'loop' else min(k0 + i, N - 1)
            row = self.df.iloc[j]
            t_utc = self.forecast_start_time + timedelta(seconds=self.dt * (k0 + i))
            s_query_km = s0_km + speed_guess_kmh * self.dt * i / 3600.0
            is_drive = True
            if self.drive_schedule is not None:
                limits = self.drive_schedule.speed_limits(t_utc)
                if limits is not None and limits[1] <= 0.0:
...

```

L872 関数 MPCNode._forecast_at_time

- 行範囲: L872–L946
- 所属: MPCNode
- 定義: `_forecast_at_time(self, t_utc: datetime, s_km: float | None=None, drive: bool=True, control_stop: bool=False) -> dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_route_value`, `all`, `clip`, `datetime64`, `dict`, `float`, `get`, `getattr`, `int`, `interp_forecast_grid`, `isna`, `len`
- 戻り値の要点: `dict(G_poa=(poa_drive + tilt_fraction * (poa_stop_ideal - poa_drive)) * self.solar_gain * gain, Tcell_C=tcell_drive + tilt_fraction * (tcell_stop_ideal - tcell_drive), Tamb_C=float(row.get('Tamb_C', 30.0)), headwind_ms=float(row.get('headwind_ms', 0.0)) if 'headwind_ms' in row else 0.0, panel_stop_mode='drive' if drive else 'control_stop' if control_stop else 'camp_or_strategy', elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0))/dict(G_poa=0.0, Tcell_C=40.0, Tamb_C=30.0, headwind_ms=0.0, elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0))/dict(G_poa=(poa_drive + tilt_fraction * (poa_stop_ideal - poa_drive)) * self.solar_gain * gain,`

gain, Tcell_C=tcell_drive+tilt_fraction*(tcell_stop_ideal-tcell_drive), Tamb_C=tamb, headwind_ms=headwind, panel_stop_mode=stop_mode, elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0))

- 主なローカル代入名: elapsed, gain, ghi, has_time, headwind, idx, poa_drive, poa_stop_ideal, row, stop_mode, t_series, tamb, tcell_drive, tcell_stop_ideal, tilt_fraction
- 読み取る主なインスタンス属性: self._route_value, self.control_stop_tilt_fraction, self.df, self.dt, self.forecast_grid, self.forecast_start_time, self.poa_gain_drive, self.poa_gain_stop, self.solar_gain, self.stop_tilt_fraction
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 4、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 len(self.df) == 0 を判定し、真なら内部処理を行う。
2. dict(G_poa=0.0, Tcell_C=40.0, Tamb_C=30.0, headwind_ms=0.0, elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0)) を返す。
3. 条件 getattr(self, 'forecast_grid', None) を判定し、真なら内部処理を行う。
4. ghi に interp_forecast_grid(self.forecast_grid, 'GHI', t_utc, s_km, 0.0) の結果を代入する。
5. poa_drive に interp_forecast_grid(self.forecast_grid, 'POA_drive', t_utc, s_km, ghi) の結果を代入する。
6. poa_stop_ideal に interp_forecast_grid(self.forecast_grid, 'POA_stop_ideal', t_utc, s_km, poa_drive) の結果を代入する。
7. tamb に interp_forecast_grid(self.forecast_grid, 'Tamb_C', t_utc, s_km, 30.0) の結果を代入する。
8. tcell_drive に interp_forecast_grid(self.forecast_grid, 'Tcell_drive_C', t_utc, s_km, interp_forecast_grid(self.forecast_grid, 'Tcell_C', t_utc, s_km, tamb)) の結果を代入する。
9. tcell_stop_ideal に interp_forecast_grid(self.forecast_grid, 'Tcell_stop_ideal_C', t_utc, s_km, tcell_drive) の結果を代入する。
10. headwind に interp_forecast_grid(self.forecast_grid, 'headwind_ms', t_utc, s_km, 0.0) の結果を代入する。
11. 条件 drive を判定し、真なら内部処理を行う。
12. tilt_fraction に 0.0 の結果を代入する。
13. gain に self.poa_gain_drive の結果を代入する。
14. stop_mode に 'drive' の結果を代入する。
15. 上の条件が偽の場合:
16. tilt_fraction に self.control_stop_tilt_fraction if control_stop else self.stop_tilt_fraction の結果を代入する。
17. gain に self.poa_gain_stop の結果を代入する。
18. stop_mode に 'control_stop' if control_stop else 'camp_or_strategy' の結果を代入する。

19. dict(G_poa=(poa_drive+tilt_fraction*(poa_stop_ideal-poa_drive))*self.solar_gain*gain, Tcell_C=tcell_drive+tilt_fraction*(tcell_stop_ideal-tcell_drive), Tamb_C=tamb, headwind_ms=headwind, panel_stop_mode=stop_mode, elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0)) を返す。
20. has_time に 'time' in self.df.columns and (not self.df['time'].isna().all()) の結果を代入する。
21. 条件 has_time を判定し、真なら内部処理を行う。
22. t_series に self.df['time'].values の結果を代入する。
23. idx に int(np.searchsorted(t_series, np.datetime64(t_utc)) - 1) の結果を代入する。
24. idx に int(np.clip(idx, 0, len(self.df) - 1)) の結果を代入する。
25. 上の条件が偽の場合:
26. elapsed に (t_utc - self.forecast_start_time).total_seconds() の結果を代入する。
27. idx に int(np.clip(elapsed / max(self.dt, 0.001), 0, len(self.df) - 1)) の結果を代入する。
28. row に self.df.iloc[idx] の結果を代入する。
29. gain に self.poa_gain_drive if drive else self.poa_gain_stop の結果を代入する。
30. poa_drive に float(row.get('POA_drive', row.get('GHI', 0.0))) の結果を代入する。
31. poa_stop_ideal に float(row.get('POA_stop_ideal', poa_drive)) の結果を代入する。
32. tilt_fraction に 0.0 if drive else self.control_stop_tilt_fraction if control_stop else self.stop_tilt_fraction の結果を代入する。
33. tcell_drive に float(row.get('Tcell_drive_C', row.get('Tcell_C', 40.0))) の結果を代入する。
34. tcell_stop_ideal に float(row.get('Tcell_stop_ideal_C', tcell_drive)) の結果を代入する。
35. dict(G_poa=(poa_drive+tilt_fraction*(poa_stop_ideal-poa_drive))*self.solar_gain*gain, Tcell_C=tcell_drive+tilt_fraction*(tcell_stop_ideal-tcell_drive), Tamb_C=float(row.get('Tamb_C', 30.0)), headwind_ms=float(row.get('headwind_ms', 0.0)) if 'headwind_ms' in row else 0.0, panel_stop_mode='drive' if drive else 'control_stop' if control_stop else 'camp_or_strategy', elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0)) を返す。

代表コード断片

```
def _forecast_at_time(
    self,
    t_utc: datetime,
    s_km: float | None = None,
    drive: bool = True,
    control_stop: bool = False,
) -> dict:
    if len(self.df) == 0:
        return dict(
            G_poa=0.0, Tcell_C=40.0, Tamb_C=30.0, headwind_ms=0.0,
            elevation_m=self._route_value(float(s_km or 0.0), 'elev_m', 0.0),
        )
    if getattr(self, 'forecast_grid', None):
        ghi = interp_forecast_grid(self.forecast_grid, 'GHI', t_utc, s_km, 0.0)
        poa_drive = interp_forecast_grid(self.forecast_grid, 'POA_drive', t_utc, s_km,
ghi)
        poa_stop_ideal = interp_forecast_grid(
            self.forecast_grid, 'POA_stop_ideal', t_utc, s_km, poa_drive
        )
        tamb = interp_forecast_grid(self.forecast_grid, 'Tamb_C', t_utc, s_km, 30.0)
        tcell_drive = interp_forecast_grid(
            self.forecast_grid,
            'Tcell_drive_C',
```



```

        t_utc,
        s_km,
        interp_forecast_grid(self.forecast_grid, 'Tcell_C', t_utc, s_km, tamb),
    )
    tcell_stop_ideal = interp_forecast_grid(
        self.forecast_grid, 'Tcell_stop_ideal_C', t_utc, s_km, tcell_drive
    )
    headwind = interp_forecast_grid(self.forecast_grid, 'headwind_ms', t_utc, s_km,
0.0)
    if drive:
        tilt_fraction = 0.0
        gain = self.poa_gain_drive
        stop_mode = 'drive'
    else:
...

```

L948 関数 MPCNode._sample_plan_segments

- 行範囲: L948–L957
- 所属: MPCNode
- 定義: `_sample_plan_segments(self, dt_sample:float)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ceil, extend, float, int, max`
- 戻り値の要点: `samples / [] / [float(seg['v_kmh']) for seg in self.v_plan_segments]`
- 主なローカル代入名: `n, samples, seg`
- 読み取る主なインスタンス属性: `self.v_plan_segments`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `not self.v_plan_segments` を判定し、真なら内部処理を行う。
を返す。

2. 条件 `dt_sample <= 0.0` を判定し、真なら内部処理を行う。

`(seg['v_kmh'] ) for seg in self.v_plan_segments]` を返す。

3. `samples` に `[]` の結果を代入する。
4. `self.v_plan_segments` を順に走査し、各要素を `seg` に入れて処理する。
5. `n` に `max(1, int(math.ceil(seg['dt_sec'] / dt_sample)))` の結果を代入する。
6. `samples.extend(...)` を実行する。
7. `samples` を返す。

代表コード断片

```
def _sample_plan_segments(self, dt_sample: float):
    if not self.v_plan_segments:
        return []
    if dt_sample <= 0.0:
        return [float(seg['v_kmh']) for seg in self.v_plan_segments]
    samples = []
    for seg in self.v_plan_segments:
        n = max(1, int(math.ceil(seg['dt_sec'] / dt_sample)))
        samples.extend([float(seg['v_kmh'])] * n)
    return samples
```

L959 関数 MPCNode._route_value

- 行範囲: L959–L968
- 所属: MPCNode
- 定義: `_route_value(self, s_km: float, field: str, default: float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `interpolate_profile`, `isfinite`
- 戻り値の要点: `float(default) / float(val) / float(default) / float(default)`
- 主なローカル代入名: `val`
- 読み取る主なインスタンス属性: `self.route_profile`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.route_profile is None` を判定し、真なら内部処理を行う。
2. `float(default)` を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `val` に `float(interpolate_profile(self.route_profile, s_km, field, default))` の結果を代入する。
5. 条件 `not np.isfinite(val)` を判定し、真なら内部処理を行う。
6. `float(default)` を返す。
7. `float(val)` を返す。
8. `Exception` を捕捉した場合:
9. `float(default)` を返す。

代表コード断片

```
def _route_value(self, s_km: float, field: str, default: float) -> float:
    if self.route_profile is None:
        return float(default)
    try:
        val = float(interpolate_profile(self.route_profile, s_km, field, default))
        if not np.isfinite(val):
            return float(default)
        return float(val)
    except Exception:
        return float(default)
```

L970 関数 MPCNode._route_average

- 行範囲: L970–L977
- 所属: MPCNode
- 定義: `_route_average(self, s0_km:float, s1_km:float, field:str, default:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `average_profile, float, isfinite`
- 戻り値の要点: `float(default) / value if np.isfinite(value) else float(default) / float(default)`
- 主なローカル代入名: `value`
- 読み取る主なインスタンス属性: `self.route_profile`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.route_profile is None` を判定し、真なら内部処理を行う。
2. `float(default)` を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `value` に `float(average_profile(self.route_profile, s0_km, s1_km, field, default))` の結果を代入する。
5. `value if np.isfinite(value) else float(default)` を返す。
6. `Exception` を捕捉した場合:
7. `float(default)` を返す。

代表コード断片

```
def _route_average(self, s0_km: float, s1_km: float, field: str, default: float) -> float:
    if self.route_profile is None:
        return float(default)
    try:
        value = float(average_profile(self.route_profile, s0_km, s1_km, field, default))
        return value if np.isfinite(value) else float(default)
    except Exception:
        return float(default)
```

L979 関数 MPCNode._speed_limit_at

- 行範囲: L979–L985
- 所属: MPCNode
- 定義: `_speed_limit_at(self, s_km: float, default_kmh: float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `interpolate_profile`
- 戻り値の要点: `float(default_kmh) / float(interpolate_profile(self.speed_profile, s_km, 'v_max_kmh', default_kmh)) / float(default_kmh)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.speed_profile`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.speed_profile is None` を判定し、真なら内部処理を行う。
2. `float(default_kmh)` を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `float(interpolate_profile(self.speed_profile, s_km, 'v_max_kmh', default_kmh))` を返す。
5. `Exception` を捕捉した場合:
6. `float(default_kmh)` を返す。

代表コード断片

```
def _speed_limit_at(self, s_km: float, default_kmh: float) -> float:
    if self.speed_profile is None:
        return float(default_kmh)
    try:
        return float(interpolate_profile(self.speed_profile, s_km, 'v_max_kmh',
        default_kmh))
    except Exception:
        return float(default_kmh)
```

L987 関数 MPCNode._soc_guard_speed

- 行範囲: L987–L1043
- 所属: MPCNode
- 定義: `_soc_guard_speed(self, v_kmh: float, s_km: float, d0: dict) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_route_value`, `electrical_balance`, `float`, `get`, `get_parameter`, `lower`, `max`, `range`, `soc_step`, `str`, `z_next_for`
- 戻り値の要点: `float(lo) / self.model.soc_step(self.z, P_pack, self.model.p.dt, current_a=float(out['I']), Tbat_C=self.Tb) / float(lo) / v_kmh`
- 主なローカル代入名: `P_pack`, `_`, `headwind_ms`, `hi`, `lo`, `mid`, `mode`, `out`, `slope_pct`, `soc_guard`, `target`
- 読み取る主なインスタンス属性: `self.Tb`, `self._route_value`, `self.get_parameter`, `self.model`, `self.route_profile`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 8、ループ 2、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `mode` に `str(self.get_parameter('soc_guard_mode').value).lower()` の結果を代入する。
2. `soc_guard` に `float(self.get_parameter('soc_guard_margin').value)` の結果を代入する。
3. `target` に `self.model.p.soc_min + soc_guard` の結果を代入する。
4. `slope_pct` に `d0['slope_pct']` の結果を代入する。
5. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
6. `slope_pct` に `self._route_value(s_km, 'slope_pct', slope_pct)` の結果を代入する。
7. `headwind_ms` に `d0.get('headwind_ms', 0.0)` の結果を代入する。
8. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。

9. headwind_ms に self._route_value(s_km, 'headwind_ms', headwind_ms) の結果を代入する。
10. 関数 z_next_for を定義する。
11. 条件 self.z <= target を判定し、真なら内部処理を行う。
12. 条件 mode == 'stop' を判定し、真なら内部処理を行う。
13. 0.0 を返す。
14. 条件 mode != 'pv_only' を判定し、真なら内部処理を行う。
15. v_kmh を返す。
16. lo に 0.0 の結果を代入する。
17. hi に max(0.0, float(v_kmh)) の結果を代入する。
18. range(20) を順に走査し、各要素を _ に代入して処理する。
19. mid に 0.5 * (lo + hi) の結果を代入する。
20. 条件 z_next_for(mid) < self.z を判定し、真なら内部処理を行う。
21. hi に mid の結果を代入する。
22. 上の条件が偽の場合:
23. lo に mid の結果を代入する。
24. float(lo) を返す。
25. 条件 z_next_for(v_kmh) >= target を判定し、真なら内部処理を行う。
26. v_kmh を返す。
27. lo に 0.0 の結果を代入する。
28. hi に max(0.0, float(v_kmh)) の結果を代入する。
29. range(25) を順に走査し、各要素を _ に代入して処理する。
30. mid に 0.5 * (lo + hi) の結果を代入する。
31. 条件 z_next_for(mid) < target を判定し、真なら内部処理を行う。
32. hi に mid の結果を代入する。
33. 上の条件が偽の場合:
34. lo に mid の結果を代入する。
35. float(lo) を返す。

代表コード断片

```
def _soc_guard_speed(self, v_kmh: float, s_km: float, d0: dict) -> float:
    mode = str(self.get_parameter('soc_guard_mode').value).lower()
    soc_guard = float(self.get_parameter('soc_guard_margin').value)
    target = self.model.p.soc_min + soc_guard

    slope_pct = d0['slope_pct']
    if self.route_profile is not None:
        slope_pct = self._route_value(s_km, 'slope_pct', slope_pct)
    headwind_ms = d0.get('headwind_ms', 0.0)
    if self.route_profile is not None:
        headwind_ms = self._route_value(s_km, 'headwind_ms', headwind_ms)
```

```

def z_next_for(v_kmh_local: float) -> float:
    out = self.model.electrical_balance(
        v_kmh_local / 3.6, slope_pct, self.z, self.Tb,
        d0['G_poa'], d0['Tcell_C'], headwind_ms=headwind_ms,
        ambient_temp_c=d0.get('Tamb_C'),
        elevation_m=self._route_value(s_km, 'elev_m', d0.get('elevation_m', 0.0)),
    )
    P_pack = float(out['P_pack'])
    return self.model.soc_step(
        self.z,
        P_pack,
        self.model.p.dt,
        current_a=float(out['I']),
        Tbat_C=self.Tb,
    )

# If already below target, apply guard mode
if self.z <= target:
    if mode == 'stop':
        return 0.0
    if mode != 'pv_only':
        return v_kmh
    # find max speed such that P_pack <= 0
...

```

L999 関数 MPCNode._soc_guard_speed.z_next_for

- 行範囲: L999–L1013
- 所属: MPCNode._soc_guard_speed
- 定義: z_next_for(v_kmh_local:float)->float
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _route_value, electrical_balance, float, get, soc_step
- 戻り値の要点: self.model.soc_step(self.z, P_pack, self.model.p.dt, current_a=float(out['I']), Tbat_C=self.Tb)
- 主なローカル代入名: P_pack, out
- 読み取る主なインスタンス属性: self.Tb, self._route_value, self.model, self.z
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. out に self.model.electrical_balance(v_kmh_local / 3.6, slope_pct, self.z, self.Tb, d0['G_poa'], d0['Tcell_C'], headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=self._route_value(s_km, 'elev_m', d0.get('elevation_m', 0.0))) の結果を代入する。
2. P_pack に float(out['P_pack']) の結果を代入する。
3. self.model.soc_step(self.z, P_pack, self.model.p.dt, current_a=float(out['I']), Tbat_C=self.Tb) を返す。

代表コード断片

```
def z_next_for(v_kmh_local: float) -> float:
    out = self.model.electrical_balance(
        v_kmh_local / 3.6, slope_pct, self.z, self.Tb,
        d0['G_poa'], d0['Tcell_C'], headwind_ms=headwind_ms,
        ambient_temp_c=d0.get('Tamb_C'),
        elevation_m=self._route_value(s_km, 'elev_m', d0.get('elevation_m', 0.0)),
    )
    P_pack = float(out['P_pack'])
    return self.model.soc_step(
        self.z,
        P_pack,
        self.model.p.dt,
        current_a=float(out['I']),
        Tbat_C=self.Tb,
    )
```

L1045 関数 MPCNode._control_stop_catalog

- 行範囲: L1045–L1063
- 所属: MPCNode
- 定義: `_control_stop_catalog(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `enumerate`, `float`, `get`, `int`, `lower`, `max`, `sorted`, `str`, `strip`
- 戻り値の要点: `sorted(catalog, key=lambda item: item['s_km'])`
- 主なローカル代入名: `catalog`, `dwell_sec`, `explicit`, `kind`, `source_index`, `stop`
- 読み取る主なインスタンス属性: `self.stops`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

上から順の処理

1. `catalog` に `[]` の結果を代入する。
2. `enumerate(self.stops)` を順に走査し、各要素を `(source_index, stop)` に入れて処理する。
3. `kind` に `str(stop.get('kind', '')).strip().lower()` の結果を代入する。
4. `explicit` に `stop.get('is_control_stop', None)` の結果を代入する。
5. 条件 `explicit is False and kind != 'control_stop'` を判定し、真なら内部処理を行う。
6. `Continue` 文を実行する。

7. 条件 `explicit is None and kind in {'trouble_stop', 'unscheduled_stop', 'strategy_stop'}` を判定し、真なら内部処理を行う。
8. `Continue` 文を実行する。
9. `dwell_sec` に `max(0.0, float(stop.get('dwell_sec', stop.get('dwell_s', 0.0))))` の結果を代入する。
10. 条件 `dwell_sec <= 0.0` を判定し、真なら内部処理を行う。
11. `Continue` 文を実行する。
12. `catalog.append(...)` を実行する。
13. `sorted(catalog, key=lambda item: item['s_km'])` を返す。

代表コード断片

```
def _control_stop_catalog(self):
    catalog = []
    for source_index, stop in enumerate(self.stops):
        kind = str(stop.get('kind', '')).strip().lower()
        explicit = stop.get('is_control_stop', None)
        if explicit is False and kind != 'control_stop':
            continue
        if explicit is None and kind in {'trouble_stop', 'unscheduled_stop', 'strategy_stop'}:
            continue
        dwell_sec = max(0.0, float(stop.get('dwell_sec', stop.get('dwell_s', 0.0))))
        if dwell_sec <= 0.0:
            continue
        catalog.append({
            'source_index': int(source_index),
            's_km': float(stop.get('s_km', 0.0)),
            'dwell_sec': dwell_sec,
            'label': str(stop.get('label', f'control_stop_{source_index + 1}')),
        })
    return sorted(catalog, key=lambda item: item['s_km'])
```

L1065 関数 `MPCNode._update_live_control_stop_hold`

- 行範囲: L1065–L1113
- 所属: `MPCNode`
- 定義: `_update_live_control_stop_hold(self, s_km:float, v_kmh:float)->bool`
- docstring: Enforce approach, standstill detection, and timed hold at declared control stops.
- このブロックが直接呼ぶ主な関数・メソッド: `_control_stop_catalog`, `add`, `float`, `get_logger`, `get_parameter`, `getattr`, `info`, `int`, `max`, `monotonic`
- 戻り値の要点: `False / False / True / True`
- 主なローカル代入名: `braking_km`, `catalog`, `decel_ms2`, `distance_to_stop_km`, `now_mono`, `stationary_kmh`, `stop`, `stop_index`, `tolerance_km`, `v_ms`
- 読み取る主なインスタンス属性: `self._control_stop_catalog`, `self.active_control_stop_index`, `self.completed_control_stop_index`, `self.control_stop_end_monotonic`, `self.get_logger`, `self.get_parameter`
- 更新する主なインスタンス属性: `self.active_control_stop_index`, `self.control_stop_end_monotonic`, `self.control_stop_p`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 8、ループ 2、`try` 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. catalog に self._control_stop_catalog() の結果を代入する。
2. 条件 not catalog を判定し、真なら内部処理を行う。
3. False を返す。
4. now_mono に time.monotonic() の結果を代入する。
5. 条件 self.active_control_stop_index is not None を判定し、真なら内部処理を行う。
6. 条件 self.control_stop_end_monotonic is None or now_mono < self.control_stop_end_monotonic を判定し、真なら内部処理を行う。
7. True を返す。
8. self.completed_control_stops.add(...) を実行する。
9. self.get_logger().info(...) を実行する。
10. self.active_control_stop_index に None の結果を代入する。
11. self.control_stop_end_monotonic に None の結果を代入する。
12. tolerance_km に max(0.01, float(self.get_parameter('control_stop_arrival_tolerance_km').value)) の結果を代入する。
13. 条件 not getattr(self, 'control_stop_position_initialized', False) を判定し、真なら内部処理を行う。
14. catalog を順に走査し、各要素を stop に入れて処理する。
15. 条件 stop['s_km'] < float(s_km) - tolerance_km を判定し、真なら内部処理を行う。
16. self.completed_control_stops.add(...) を実行する。
17. self.control_stop_position_initialized に True の結果を代入する。
18. stationary_kmh に max(0.0, float(self.get_parameter('control_stop_stationary_speed_kmh').value)) の結果を代入する。
19. decel_ms2 に max(0.1, float(self.get_parameter('control_stop_brake_decel_kmhps').value) / 3.6) の結果を代入する。
20. v_ms に max(0.0, float(v_kmh)) / 3.6 の結果を代入する。
21. braking_km に v_ms * v_ms / (2.0 * decel_ms2) / 1000.0 の結果を代入する。
22. braking_km を Add で更新する。
23. catalog を順に走査し、各要素を stop に入れて処理する。
24. stop_index に int(stop['source_index']) の結果を代入する。
25. 条件 stop_index in self.completed_control_stops を判定し、真なら内部処理を行う。
26. Continue 文を実行する。

27. distance_to_stop_km に float(stop['s_kmh']) - float(s_kmh) の結果を代入する。
28. 条件 distance_to_stop_km > braking_kmh を判定し、真なら内部処理を行う。
29. False を返す。
30. 条件 distance_to_stop_km <= tolerance_kmh and float(v_kmh) <= stationary_kmh を判定し、真なら内部処理を行う。
31. self.active_control_stop_index に stop_index の結果を代入する。
32. self.control_stop_end_monotonic に now_mono + float(stop['dwell_sec']) の結果を代入する。
33. self.get_logger().info(...) を実行する。
34. True を返す。
35. False を返す。

代表コード断片

```
def _update_live_control_stop_hold(self, s_kmh: float, v_kmh: float) -> bool:
    """Enforce approach, standstill detection, and timed hold at declared control stops.
    """
    catalog = self._control_stop_catalog()
    if not catalog:
        return False
    now_mono = time.monotonic()
    if self.active_control_stop_index is not None:
        if self.control_stop_end_monotonic is None or now_mono < self.
control_stop_end_monotonic:
            return True
        self.completed_control_stops.add(int(self.active_control_stop_index))
        self.get_logger().info('Control-stop dwell completed; speed command released.')
        self.active_control_stop_index = None
        self.control_stop_end_monotonic = None

    tolerance_kmh = max(
        0.01, float(self.get_parameter('control_stop_arrival_tolerance_kmh').value)
    )
    if not getattr(self, 'control_stop_position_initialized', False):
        for stop in catalog:
            if stop['s_kmh'] < float(s_kmh) - tolerance_kmh:
                self.completed_control_stops.add(int(stop['source_index']))
            self.control_stop_position_initialized = True

    stationary_kmh = max(
        0.0, float(self.get_parameter('control_stop_stationary_speed_kmh').value)
    )
    decel_ms2 = max(
        0.1,
        float(self.get_parameter('control_stop_brake_decel_kmhps').value) / 3.6,
    )
    v_ms = max(0.0, float(v_kmh)) / 3.6
    braking_kmh = v_ms * v_ms / (2.0 * decel_ms2) / 1000.0
    braking_kmh += max(0.0, float(self.get_parameter('control_stop_brake_margin_kmh').
value))

    for stop in catalog:
...

```

L1115 関数 MPCNode._dwell_penalty

- 行範囲: L1115–L1125
- 所属: MPCNode
- 定義: `_dwell_penalty(self, s_km:float, v_ms:float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `float`, `get`, `get_parameter`, `max`
- 戻り値の要点: `pen`
- 主なローカル代入名: `dwell_s`, `pen`, `s_stop`, `st`, `vmax_kmh`, `vmax_ms`, `width_km`
- 読み取る主なインスタンス属性: `self.get_parameter`, `self.stops`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `vmax_kmh` に `float(self.get_parameter('v_max_kmh').value)` の結果を代入する。
2. `vmax_ms` に `vmax_kmh / 3.6` の結果を代入する。
3. `pen` に `0.0` の結果を代入する。
4. `self.stops` を順に走査し、各要素を `st` に入れて処理する。
5. `s_stop` に `float(st.get('s_km', 0.0))` の結果を代入する。
6. `dwell_s` に `float(st.get('dwell_sec', st.get('dwell_s', 0.0)))` の結果を代入する。
7. `width_km` に `max(0.05, dwell_s * vmax_ms / 1000.0 * 0.5)` の結果を代入する。
8. 条件 `abs(s_km - s_stop) <= width_km` を判定し、真なら内部処理を行う。
9. `pen` を `Add` で更新する。
10. `pen` を返す。

代表コード断片

```
def _dwell_penalty(self, s_km: float, v_ms: float) -> float:
    vmax_kmh = float(self.get_parameter('v_max_kmh').value)
    vmax_ms = vmax_kmh / 3.6
    pen = 0.0
    for st in self.stops:
        s_stop = float(st.get('s_km', 0.0))
        dwell_s = float(st.get('dwell_sec', st.get('dwell_s', 0.0)))
        width_km = max(0.05, (dwell_s * vmax_ms) / 1000.0 * 0.5)
        if abs(s_km - s_stop) <= width_km:
            pen += 1.0e5 * (v_ms ** 2)
    return pen
```

L1127 関数 MPCNode._mpc_solve_solar

- 行範囲: L1127–L1289
- 所属: MPCNode
- 定義: `_mpc_solve_solar(self, data)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_dwell_penalty`, `_route_value`, `_speed_limit_at`, `abs`, `all`, `array`, `bool`, `clip`, `dict`, `electrical_balance`, `float`, `get`
- 戻り値の要点: $(\text{float}(v_seq_kmh[0]), [\text{float}(v) \text{ for } v \text{ in } v_seq_kmh]) / (\text{self}.v_cmd, [\text{self}.v_cmd]) / x * x / J$
- 主なローカル代入名: `I`, `J`, `Np`, `P_pack`, `Tb`, `Tb_next`, `V`, `aux_power_w`, `bounds`, `d`, `dv`, `dv_max_kmhps`, `dv_max_msps`, `headwind_ms`, `inertial_power_w`, `k`, `lb`, `limits`, `loss_int`, `out`
- 読み取る主なインスタンス属性: `self.Tb`, `self._dwell_penalty`, `self._route_value`, `self._speed_limit_at`, `self.drive_scheduled`, `self.get_logger`, `self.get_parameter`, `self.model`, `self.race_km`, `self.route_profile`, `self.s_km`, `self.s_meas`, `self.soc_band`, `self.soc_day_end_max`, `self.soc_finish_target`, `self.soc_finish_tol`, `self.soc_target`, `self.upper_vmin_kmh`, `self.v_cmd`, `self.v_now`, `self.w_soc_band`, `self.w_soc_day_max`, `self.w_soc_progress`, `self.w_soc_target`
- 更新する主なインスタンス属性: `self.last_upper_solve_ok`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 20、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `p` に `self.model.p` の結果を代入する。
2. `Np` に `len(data)` の結果を代入する。
3. 条件 `Np <= 0` を判定し、真なら内部処理を行う。
4. `(self.v_cmd, [self.v_cmd])` を返す。
5. `v0_guess` に `self.v_cmd / 3.6` の結果を代入する。
6. 条件 `bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now)` を判定し、真なら内部処理を行う。
7. `v0_guess` に `float(self.v_now) / 3.6` の結果を代入する。
8. `x0` に `np.array([v0_guess] * Np, dtype=float)` の結果を代入する。
9. `lb` に `np.zeros(Np, dtype=float)` の結果を代入する。
10. `v_max_kmh` に `float(self.get_parameter('v_max_kmh').value)` の結果を代入する。
11. `v_min_solver` に `max(0.1, float(self.upper_vmin_kmh))` の結果を代入する。
12. `ub` に `np.ones(Np, dtype=float) * (v_max_kmh / 3.6)` の結果を代入する。
13. `term_soc_min` に `float(self.get_parameter('terminal_soc_min').value)` の結果を代入する。

14. `w_dv` に `float(self.get_parameter('w_dv').value)` の結果を代入する。
15. `w_dv_limit` に `float(self.get_parameter('w_dv_limit').value)` の結果を代入する。
16. `dv_max_kmhps` に `float(self.get_parameter('dv_max_kmhps').value)` の結果を代入する。
17. `dv_max_msps` に `dv_max_kmhps / 3.6` の結果を代入する。
18. `w_T` に `float(self.get_parameter('w_T').value)` の結果を代入する。
19. `w_speed_limit` に `float(self.get_parameter('w_speed_limit').value)` の結果を代入する。
20. `w_drive_window` に `float(self.get_parameter('w_drive_window').value)` の結果を代入する。
21. `w_current` に `float(self.get_parameter('w_current').value)` の結果を代入する。
22. `soc_target` に `float(self.soc_target)` の結果を代入する。
23. `soc_band` に `float(self.soc_band)` の結果を代入する。
24. `w_soc_target` に `float(self.w_soc_target)` の結果を代入する。
25. `w_soc_band` に `float(self.w_soc_band)` の結果を代入する。
26. `soc_day_end_max` に `float(self.soc_day_end_max)` の結果を代入する。
27. `w_soc_day_max` に `float(self.w_soc_day_max)` の結果を代入する。
28. `soc_finish_target` に `float(self.soc_finish_target)` の結果を代入する。
29. `soc_finish_tol` に `float(self.soc_finish_tol)` の結果を代入する。
30. `w_soc_progress` に `float(self.w_soc_progress)` の結果を代入する。
31. `w_soc_terminal` に `float(self.w_soc_terminal)` の結果を代入する。
32. `race_km` に `float(self.race_km)` の結果を代入する。
33. `z_start` に `float(self.z)` の結果を代入する。
34. 関数 `quad_penalty` を定義する。
35. 関数 `cost` を定義する。
36. `bounds` に `list(zip(lb, ub))` の結果を代入する。
37. `res` に `minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=150))` の結果を代入する。
38. 条件 `np.all(np.isfinite(res.x))` を判定し、真なら内部処理を行う。
39. `self.last_upper_solve_ok` に `bool(res.success)` の結果を代入する。
40. `v_seq` に `res.x` の結果を代入する。
41. 条件 `not res.success` を判定し、真なら内部処理を行う。
42. `self.get_logger().warn(...)` を実行する。
43. 上の条件が偽の場合:
44. `self.last_upper_solve_ok` に `False` の結果を代入する。
45. `self.get_logger().warn(...)` を実行する。
46. `v_seq` に `x0` の結果を代入する。
47. `v_seq_kmh` に `np.clip(v_seq * 3.6, 0.0, v_max_kmh)` の結果を代入する。
48. `(float(v_seq_kmh[0]), [float(v) for v in v_seq_kmh])` を返す。

代表コード断片

```
def _mpc_solve_solar(self, data):
    p = self.model.p
    Np = len(data)
    if Np <= 0:
        return self.v_cmd, [self.v_cmd]

    v0_guess = self.v_cmd / 3.6
    if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
        v0_guess = float(self.v_now) / 3.6
    x0 = np.array([v0_guess] * Np, dtype=float) # m/s
    lb = np.zeros(Np, dtype=float)
    v_max_kmh = float(self.get_parameter('v_max_kmh').value)
    v_min_solver = max(0.1, float(self.upper_vmin_kmh))
    ub = np.ones(Np, dtype=float) * (v_max_kmh / 3.6)

    term_soc_min = float(self.get_parameter('terminal_soc_min').value)
    w_dv = float(self.get_parameter('w_dv').value)
    w_dv_limit = float(self.get_parameter('w_dv_limit').value)
    dv_max_kmhps = float(self.get_parameter('dv_max_kmhps').value)
    dv_max_msps = dv_max_kmhps / 3.6
    w_T = float(self.get_parameter('w_T').value)
    w_speed_limit = float(self.get_parameter('w_speed_limit').value)
    w_drive_window = float(self.get_parameter('w_drive_window').value)
    w_current = float(self.get_parameter('w_current').value)
    soc_target = float(self.soc_target)
    soc_band = float(self.soc_band)
    w_soc_target = float(self.w_soc_target)
    w_soc_band = float(self.w_soc_band)
    soc_day_end_max = float(self.soc_day_end_max)
    w_soc_day_max = float(self.w_soc_day_max)
    soc_finish_target = float(self.soc_finish_target)
    soc_finish_tol = float(self.soc_finish_tol)
    w_soc_progress = float(self.w_soc_progress)
    w_soc_terminal = float(self.w_soc_terminal)
    race_km = float(self.race_km)

    ...
```

L1164 関数 MPCNode._mpc_solve_solar.quad_penalty

- 行範囲: L1164–L1169
- 所属: MPCNode._mpc_solve_solar
- 定義: quad_penalty(x, cap=1000.0)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: $x * x / 0.0$
- 主なローカル代入名: x
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 $x \leq 0.0$ を判定し、真なら内部処理を行う。
2. 0.0 を返す。
3. 条件 $x > \text{cap}$ を判定し、真なら内部処理を行う。
4. x に cap の結果を代入する。
5. $x * x$ を返す。

代表コード断片

```
def quad_penalty(x, cap=1.0e3):  
    if x <= 0.0:  
        return 0.0  
    if x > cap:  
        x = cap  
    return x * x
```

L1171 関数 MPCNode._mpc_solve_solar.cost

- 行範囲: L1171–L1275
- 所属: MPCNode._mpc_solve_solar
- 定義: `cost(v)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_dwell_penalty`, `_route_value`, `_speed_limit_at`, `abs`, `bool`, `electrical_balance`, `float`, `get`, `get_parameter`, `is_drive_time`, `isfinite`, `max`
- 戻り値の要点: `J`
- 主なローカル代入名: `I`, `J`, `P_pack`, `Tb`, `Tb_next`, `V`, `aux_power_w`, `d`, `dv`, `headwind_ms`, `inertial_power_w`, `k`, `limits`, `loss_int`, `out`, `prog`, `s_km`, `slope_pct`, `soc_line`, `t_next`
- 読み取る主なインスタンス属性: `self.Tb`, `self._dwell_penalty`, `self._route_value`, `self._speed_limit_at`, `self.drive_scheduled`, `self.get_parameter`, `self.model`, `self.route_profile`, `self.s_km`, `self.s_meas`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 14、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `z` に `float(self.z)` の結果を代入する。
2. `Tb` に `float(self.Tb)` の結果を代入する。
3. `s_km` に `float(self.s_km)` の結果を代入する。
4. 条件 `bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
5. `s_km` に `float(self.s_meas)` の結果を代入する。
6. `v_prev` に `float(v0_guess)` の結果を代入する。
7. `J` に `0.0` の結果を代入する。
8. `range(Np)` を順に走査し、各要素を `k` に入れて処理する。
9. `d` に `data[k]` の結果を代入する。
10. `v_k` に `float(v[k])` の結果を代入する。
11. `slope_pct` に `d['slope_pct']` の結果を代入する。
12. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
13. `slope_pct` に `self._route_value(s_km, 'slope_pct', slope_pct)` の結果を代入する。
14. `headwind_ms` に `d.get('headwind_ms', 0.0)` の結果を代入する。
15. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
16. `headwind_ms` に `self._route_value(s_km, 'headwind_ms', headwind_ms)` の結果を代入する。
17. `aux_power_w` に `None` の結果を代入する。
18. 条件 `self.drive_schedule is not None and 't_utc' in d and (not self.drive_schedule.is_drive_time(d['t_utc']))` を判定し、真なら内部処理を行う。
19. `aux_power_w` に `self.model.scheduled_auxiliary_power(is_driving=False, irradiance_wm2=d.get('G_poa', 0.0))` の結果を代入する。
20. `inertial_power_w` に `0.5 * p.m * (v_k * v_k - v_prev * v_prev) / max(p.dt, 1e-09)` の結果を代入する。
21. `out` に `self.model.electrical_balance(v_k, slope_pct, z, Tb, d['G_poa'], d['Tcell_C'], headwind_ms=headwind_ms, aux_power_w=aux_power_w, inertial_power_w=inertial_power_w, ambient_temp_c=d.get('Tamb_C'), elevation_m=d.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)))` の結果を代入する。
22. `I` に `float(out['I'])` の結果を代入する。
23. `V` に `float(out['V'])` の結果を代入する。
24. `P_pack` に `float(out['P_pack'])` の結果を代入する。
25. `loss_int` に `float(out['losses_int'])` の結果を代入する。
26. `z_next` に `self.model.soc_step(z, P_pack, p.dt, current_a=I, Tbat_C=Tb)` の結果を代入する。
27. `Tb_next` に `Tb + p.dt / 1800.0 * (d['Tamb_C'] - Tb) + loss_int * p.dt / 50000.0` の結果を代入する。
28. `s_km` に `s_km + v_k * (p.dt / 1000.0)` の結果を代入する。
29. `J` を Add で更新する。
30. `J` を Add で更新する。

31. 条件 `self.drive_schedule is not None and soc_day_end_max > 0.0 and ('t_utc' in d)` を判定し、真なら内部処理を行う。
32. `t_next` に `d['t_utc'] + timedelta(seconds=p.dt)` の結果を代入する。
33. 条件 `self.drive_schedule.is_drive_time(d['t_utc']) and (not self.drive_schedule.is_drive_time(t_next))` を判定し、真なら内部処理を行う。
34. `J` を `Add` で更新する。
35. 条件 `soc_finish_target > 0.0` を判定し、真なら内部処理を行う。
36. `prog` に `max(0.0, min(1.0, s_km / max(race_km, 1.0)))` の結果を代入する。
37. `soc_line` に `z_start + (soc_finish_target - z_start) * prog` の結果を代入する。
38. 条件 `z_next > soc_line + soc_finish_tol` を判定し、真なら内部処理を行う。
39. `J` を `Add` で更新する。
40. 条件 `z_next < soc_line - soc_finish_tol` を判定し、真なら内部処理を行う。
41. `J` を `Add` で更新する。
42. `dv` に `(v_k - v_prev) / max(p.dt, 0.001)` の結果を代入する。
43. 条件 `dv_max_msps > 0.0` を判定し、真なら内部処理を行う。
44. `J` を `Add` で更新する。
45. 条件 `self.drive_schedule is not None and 't_utc' in d` を判定し、真なら内部処理を行う。
46. `limits` に `self.drive_schedule.speed_limits(d['t_utc'])` の結果を代入する。
47. 条件 `limits is not None` を判定し、真なら内部処理を行う。
48. `(vmin_kmh, vmax_kmh)` に `limits` の結果を代入する。
49. `vmin_ms` に `vmin_kmh / 3.6` の結果を代入する。
50. `vmax_ms` に `vmax_kmh / 3.6` の結果を代入する。
51. `J` を `Add` で更新する。
52. `J` を `Add` で更新する。
53. `vmax_local` に `self._speed_limit_at(s_km, self.get_parameter('v_max_kmh').value)` の結果を代入する。
54. 条件 `vmax_local < self.get_parameter('v_max_kmh').value` を判定し、真なら内部処理を行う。
55. `J` を `Add` で更新する。
56. `J` を `Add` で更新する。
57. `J` を `Add` で更新する。
58. `J` を `Add` で更新する。
59. `J` を `Add` で更新する。
60. `J` を `Add` で更新する。
61. `J` を `Add` で更新する。
62. `J` を `Add` で更新する。
63. `J` を `Add` で更新する。

64. J を Add で更新する。
65. (z, Tb) に (z_next, Tb_next) の結果を代入する。
66. v_prev に v_k の結果を代入する。
67. J を Add で更新する。
68. 条件 soc_finish_target > 0.0 を判定し、真なら内部処理を行う。
69. J を Add で更新する。
70. J を返す。

代表コード断片

```
def cost(v):
    z = float(self.z)
    Tb = float(self.Tb)
    s_km = float(self.s_km)
    if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
        s_km = float(self.s_meas)
    v_prev = float(v0_guess)
    J = 0.0

    for k in range(Np):
        d = data[k]
        v_k = float(v[k])
        slope_pct = d['slope_pct']
        if self.route_profile is not None:
            slope_pct = self._route_value(s_km, 'slope_pct', slope_pct)
        headwind_ms = d.get('headwind_ms', 0.0)
        if self.route_profile is not None:
            headwind_ms = self._route_value(s_km, 'headwind_ms', headwind_ms)
        aux_power_w = None
        if self.drive_schedule is not None and 't_utc' in d and not self.
drive_schedule.is_drive_time(d['t_utc']):
            aux_power_w = self.model.scheduled_auxiliary_power(
                is_driving=False,
                irradiance_wm2=d.get('G_poa', 0.0),
            )
        inertial_power_w = 0.5 * p.m * (v_k * v_k - v_prev * v_prev) / max(p.dt, 1.0
e-9)
        out = self.model.electrical_balance(
            v_k,
            slope_pct,
            z,
            Tb,
            d['G_poa'],
            d['Tcell_C'],
            headwind_ms=headwind_ms,
            aux_power_w=aux_power_w,
            inertial_power_w=inertial_power_w,
        )
    ...
```

L1291 関数 MPCNode._mpc_solve_solar_distance

- 行範囲: L1291–L1814
- 所属: MPCNode
- 定義: _mpc_solve_solar_distance(self, t0_utc:datetime, s0_km:float, x0=None)

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_control_stop_catalog`, `_forecast_at_time`, `_route_average`, `_route_value`, `_speed_limit_at`, `abs`, `absolute_control_distances`, `append`, `apply_control_stop_at`, `array`, `asarray`, `bool`
- 戻り値の要点: `(v0_kmh, segments, u_seq) / (self.v_cmd, [{ 'v_kmh': float(self.v_cmd), 'dt_sec': float(p.dt) }], [float(self.v_cmd)]) / (1.0 - alpha) * u_vec[idx] + alpha * u_vec[idx_next] / np.array(u_seed, dtype=float)`
- 主なローカル代入名: `I`, `J`, `Nc`, `Np`, `P_mech_wheel`, `P_pack`, `P_pv`, `Tb`, `Tb_after`, `Tb_next`, `Tb_seed`, `V`, `_`, `alpha`, `base_edges`, `best_score`, `best_v`, `bounds`, `candidate_grid`, `constraint`
- 読み取る主なインスタンス属性: `self.Tb`, `self._control_stop_catalog`, `self._forecast_at_time`, `self._route_average`, `self._route_value`, `self._speed_limit_at`, `self.drive_schedule`, `self.get_logger`, `self.get_parameter`, `self.initial_upper_policy`, `self.last_upper_solve_ok`, `self.model`, `self.race_km`, `self.soc_day_end_max`, `self.soc_day_end_target`, `self.soc_finish_target`, `self.soc_finish_tol`, `self.upper_adaptive_growth`, `self.upper_adaptive_max_ds_km`, `self.upper_adaptive_min_ds_km`, `self.upper_cost_cfg`, `self.upper_ctrl_km`, `self.upper_day_end_soc_min`, `self.upper_ds_km`
- 更新する主なインスタンス属性: `self.last_upper_solve_ok`, `self.upper_plan_ctrl_s`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 25、ループ 7、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `p` に `self.model.p` の結果を代入する。
2. `horizon` に `build_upper_distance_horizon(mode=self.upper_horizon_mode, s0_km=s0_km, race_km=self.race_km, ds_km=self.upper_ds_km, horizon_km=self.upper_horizon_km, max_steps=self.upper_max_steps, ctrl_km=self.upper_ctrl_km, adaptive_min_ds_km=self.upper_adaptive_min_ds_km, adaptive_max_ds_km=self.upper_adaptive_max_ds_km, adaptive_growth=self.upper_adaptive_growth)` の結果を代入する。
3. `ds_seq` に `np.array(horizon.ds_seq_km, dtype=float)` の結果を代入する。
4. `seg_s` に `np.array(horizon.seg_s_km, dtype=float)` の結果を代入する。
5. `Np` に `int(len(ds_seq))` の結果を代入する。
6. 条件 `Np <= 0` を判定し、真なら内部処理を行う。
7. `(self.v_cmd, [{ 'v_kmh': float(self.v_cmd), 'dt_sec': float(p.dt) }], [float(self.v_cmd)])` を返す。
8. `control_stops` に `self._control_stop_catalog()` の結果を代入する。
9. `horizon_end_km` に `float(s0_km + np.sum(ds_seq))` の結果を代入する。
10. `extra_edges` に `[float(stop['s_km']) for stop in control_stops if float(s0_km) + 1e-09 < float(stop['s_km']) < horizon_end_km - 1e-09]` の結果を代入する。
11. 条件 `extra_edges` を判定し、真なら内部処理を行う。
12. `base_edges` に `np.concatenate([float(s0_km)], float(s0_km) + np.cumsum(ds_seq)))` の結果を代入する。
13. `route_edges` に `np.array(sorted(set(np.round(np.concatenate((base_edges, extra_edges)), 9))), dtype=float)` の結果を代入する。

14. `ds_seq` に `np.diff(route_edges)` の結果を代入する。
15. `seg_s` に `route_edges[:-1] - float(s0_km)` の結果を代入する。
16. `Np` に `int(len(ds_seq))` の結果を代入する。
17. `v_max_kmh` に `float(self.get_parameter('v_max_kmh').value)` の結果を代入する。
18. `v_min_solver` に `max(0.1, float(self.upper_vmin_kmh))` の結果を代入する。
19. `v0` に `float(self.v_cmd)` の結果を代入する。
20. 条件 `bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now)` を判定し、真なら内部処理を行う。
21. `v0` に `float(self.v_now)` の結果を代入する。
22. `ctrl_s` に `np.array(horizon.ctrl_s_km, dtype=float)` の結果を代入する。
23. `ctrl_s_abs` に `absolute_control_distances(s0_km, ctrl_s)` の結果を代入する。
24. `Nc` に `int(len(ctrl_s))` の結果を代入する。
25. `previous_ctrl_s` に `getattr(self, 'upper_plan_ctrl_s', None)` の結果を代入する。
26. 条件 `x0 is not None and previous_ctrl_s is not None and (len(x0) == len(previous_ctrl_s)) and (len(previous_ctrl_s) >= 1)` を判定し、真なら内部処理を行う。
27. `x0` に `shift_upper_policy_warm_start(previous_ctrl_s, x0, ctrl_s_abs, minimum_speed_kmh=v_min_solver, maximum_speed_kmh=v_max_kmh)` の結果を代入する。
28. 上の条件が偽の場合:
29. 条件 `x0 is not None and len(x0) == Nc` を判定し、真なら内部処理を行う。
30. `x0` に `np.array(x0, dtype=float)` の結果を代入する。
31. 上の条件が偽の場合:
32. 条件 `self.initial_upper_policy is not None` を判定し、真なら内部処理を行う。
33. `x0` に `interpolate_upper_policy(self.initial_upper_policy, ctrl_s_abs, minimum_speed_kmh=v_min_solver, maximum_speed_kmh=v_max_kmh)` の結果を代入する。
34. 上の条件が偽の場合:
35. `x0` に `np.full(Nc, float(np.clip(v0, v_min_solver, v_max_kmh)), dtype=float)` の結果を代入する。
36. `x0` に `np.clip(np.asarray(x0, dtype=float), v_min_solver, v_max_kmh)` の結果を代入する。
37. `self.upper_plan_ctrl_s` に `ctrl_s_abs.copy()` の結果を代入する。
38. `bounds` に `[(v_min_solver, v_max_kmh)] * Nc` の結果を代入する。
39. `idx` に `np.searchsorted(ctrl_s, seg_s, side='right') - 1` の結果を代入する。
40. `idx` に `np.clip(idx, 0, Nc - 1)` の結果を代入する。
41. `idx_next` に `np.clip(idx + 1, 0, Nc - 1)` の結果を代入する。
42. `denom` に `np.maximum(ctrl_s[idx_next] - ctrl_s[idx], 1e-06)` の結果を代入する。
43. `alpha` に `(seg_s - ctrl_s[idx]) / denom` の結果を代入する。
44. 関数 `expand_ctrl` を定義する。

45. 関数 `build_balance_seed` を定義する。
46. `w_dv_limit` に `float(self.get_parameter('w_dv_limit').value)` の結果を代入する。
47. `dv_max_kmhps` に `float(self.get_parameter('dv_max_kmhps').value)` の結果を代入する。
48. `term_soc_min` に `float(self.get_parameter('terminal_soc_min').value)` の結果を代入する。
49. `day_end_soc_min` に `float(self.upper_day_end_soc_min)` の結果を代入する。
50. `soc_day_end_max` に `float(self.soc_day_end_max)` の結果を代入する。
51. `soc_finish_target` に `float(self.soc_finish_target)` の結果を代入する。
52. `soc_day_end_tol` に `float(self.get_parameter('soc_day_end_tol').value)` の結果を代入する。
53. 関数 `integrate_stationary_duration` を定義する。
54. 関数 `step_wait` を定義する。
55. 関数 `apply_control_stop_at` を定義する。
56. 関数 `cost` を定義する。
57. `structured_seeds` に `[('balance_seed', build_balance_seed())]` の結果を代入する。
58. `global_search_enabled` に `bool(self.get_parameter('upper_global_search_enabled').value)` の結果を代入する。
59. `global_search_mode` に `str(self.get_parameter('upper_global_search_mode').value or 'auto').strip().lower()` の結果を代入する。
60. `(u_seq, solve_info)` に `hybrid_bounded_minimize(cost, x0, bounds, maxiter=int(self.upper_max_iter), structured_seeds=structured_seeds, cem_enabled=global_search_enabled, cem_mode=global_search_mode, cem_generations=cem_population=int(self.get_parameter('upper_cem_population').value), cem_elite=int(self.get_parameter('upper_cem_elite').value), local_refine_topk=int(self.get_parameter('upper_local_refine_topk').value), seed_library_mode=str(self.get_parameter('upper_seed_library_mode').value), rng_seed=int(max(0.0, round(s0_km * 10.0))), shgo_samples=int(self.get_parameter('upper_shgo_samples').value), shgo_iters=int(self.get_parameter('upper_shgo_iters').value), cert_grid_levels=int(self.get_parameter('upper_cert_grid_levels').value), cert_max_evaluations=int(self.get_parameter('upper_cert_max_evaluations').value), cert_workers=int(self.get_parameter('upper_cert_workers').value))` の結果を代入する。
61. `self.last_upper_solve_ok` に `bool(solve_info.get('success', False))` の結果を代入する。
62. 条件 `not self.last_upper_solve_ok` を判定し、真なら内部処理を行う。
63. `self.get_logger().warn(...)` を実行する。
64. `v_seq` に `expand_ctrl(u_seq)` の結果を代入する。
65. `segments` に `[]` の結果を代入する。
66. `t_utc` に `t0_utc` の結果を代入する。
67. `s_km` に `float(s0_km)` の結果を代入する。
68. `z` に `float(self.z)` の結果を代入する。
69. `Tb` に `float(self.Tb)` の結果を代入する。
70. `v_prev_seg` に `float(v0)` の結果を代入する。
71. `enumerate(v_seq)` を順に走査し、各要素を `(idx_seg, v_k)` に入れて処理する。
72. `(t_utc, z, Tb, schedule_wait, _)` に `step_wait(t_utc, z, Tb, s_km)` の結果を代入する。

73. 条件 `schedule_wait > 1e-09` を判定し、真なら内部処理を行う。
74. `v_prev_seg` に 0.0 の結果を代入する。
75. `ds_step_km` に `float(ds_seq[idx_seg])` の結果を代入する。
76. `v_k` に `float(np.clip(v_k, 0.0, v_max_kmh))` の結果を代入する。
77. `vmax_local` に `self._speed_limit_at(s_km, v_max_kmh)` の結果を代入する。
78. 条件 `vmax_local >= v_min_solver` を判定し、真なら内部処理を行う。
79. `v_k` に `max(v_min_solver, min(v_k, vmax_local))` の結果を代入する。
80. 上の条件が偽の場合:

代表コード断片

```
def _mpc_solve_solar_distance(self, t0_utc: datetime, s0_km: float, x0=None):
    p = self.model.p
    horizon = build_upper_distance_horizon(
        mode=self.upper_horizon_mode,
        s0_km=s0_km,
        race_km=self.race_km,
        ds_km=self.upper_ds_km,
        horizon_km=self.upper_horizon_km,
        max_steps=self.upper_max_steps,
        ctrl_km=self.upper_ctrl_km,
        adaptive_min_ds_km=self.upper_adaptive_min_ds_km,
        adaptive_max_ds_km=self.upper_adaptive_max_ds_km,
        adaptive_growth=self.upper_adaptive_growth,
    )
    ds_seq = np.array(horizon.ds_seq_km, dtype=float)
    seg_s = np.array(horizon.seg_s_km, dtype=float)
    Np = int(len(ds_seq))
    if Np <= 0:
        return self.v_cmd, [{ 'v_kmh': float(self.v_cmd), 'dt_sec': float(p.dt) }], [float(
            self.v_cmd)]

    # Split the integration mesh at every declared control stop so arrival
    # time, dwell charging, and the next departure are evaluated exactly at
    # the stop coordinate rather than at the end of an arbitrary route bin.
    control_stops = self._control_stop_catalog()
    horizon_end_km = float(s0_km + np.sum(ds_seq))
    extra_edges = [
        float(stop['s_km'])
        for stop in control_stops
        if float(s0_km) + 1.0e-9 < float(stop['s_km']) < horizon_end_km - 1.0e-9
    ]
    if extra_edges:
        base_edges = np.concatenate(([float(s0_km)], float(s0_km) + np.cumsum(ds_seq)))
        route_edges = np.array(sorted(set(np.round(np.concatenate((base_edges,
            extra_edges)), 9))), dtype=float)
        ds_seq = np.diff(route_edges)
        seg_s = route_edges[:-1] - float(s0_km)
    ...
```

L1379 関数 MPCNode._mpc_solve_solar_distance.expand_ctrl

- 行範囲: L1379–L1380
- 所属: MPCNode._mpc_solve_solar_distance

- 定義: `expand_ctrl(u_vec)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: $(1.0 - \alpha) * u_vec[idx] + \alpha * u_vec[idx_next]$
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. $(1.0 - \alpha) * u_vec[idx] + \alpha * u_vec[idx_next]$ を返す。

代表コード断片

```
def expand_ctrl(u_vec):
    return (1.0 - alpha) * u_vec[idx] + alpha * u_vec[idx_next]
```

L1382 関数 `MPCNode._mpc_solve_solar_distance.build_balance_seed`

- 行範囲: L1382–L1473
- 所属: `MPCNode._mpc_solve_solar_distance`
- 定義: `build_balance_seed()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_forecast_at_time`, `_route_average`, `_route_value`, `_speed_limit_at`, `abs`, `apply_control_stop_at`, `array`, `clip`, `electrical_balance`, `float`, `get`, `int`
- 戻り値の要点: `np.array(u_seed, dtype=float)`
- 主なローカル代入名: `Tb_seed`, `_`, `best_score`, `best_v`, `candidate_grid`, `control_wait`, `ds_ctrl`, `ds_leg_km`, `dt_seed`, `edge_s_km`, `env`, `env_leg`, `headwind_leg`, `headwind_ms`, `idx_ctrl`, `limits`, `out`, `p_pack`, `s_seed`, `score`
- 読み取る主なインスタンス属性: `self.Tb`, `self._forecast_at_time`, `self._route_average`, `self._route_value`, `self._speed_limit_at`, `self.drive_schedule`, `self.model`, `self.upper_ds_km`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 3、try 0

この定義を読むための Python 構文

- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `u_seed` に `np.array(x0, dtype=float)` の結果を代入する。
2. `z_seed` に `float(self.z)` の結果を代入する。
3. `Tb_seed` に `float(self.Tb)` の結果を代入する。
4. `s_seed` に `float(s0_km)` の結果を代入する。
5. `t_seed` に `t0_utc` の結果を代入する。
6. `v_prev_seed` に `float(v0)` の結果を代入する。
7. `range(Nc)` を順に走査し、各要素を `idx_ctrl` に入れて処理する。
8. (`t_seed`, `z_seed`, `Tb_seed`, `_`, `_`) に `step_wait(t_seed, z_seed, Tb_seed, s_seed)` の結果を代入する。
9. `vmax_local` に `self._speed_limit_at(s_seed, v_max_kmh)` の結果を代入する。
10. `vmin_local` に `v_min_solver` の結果を代入する。
11. 条件 `self.drive_schedule is not None` を判定し、真なら内部処理を行う。
12. `limits` に `self.drive_schedule.speed_limits(t_seed)` の結果を代入する。
13. 条件 `limits is not None` を判定し、真なら内部処理を行う。
14. `vmin_local` に `max(vmin_local, float(limits[0]))` の結果を代入する。
15. `vmax_local` に `min(vmax_local, float(limits[1]))` の結果を代入する。
16. 条件 `vmax_local < vmin_local` を判定し、真なら内部処理を行う。
17. `u_seed[idx_ctrl]` に `max(0.0, vmax_local)` の結果を代入する。
18. `Continue` 文を実行する。
19. `candidate_grid` に `np.linspace(vmin_local, vmax_local, num=max(5, min(13, int((vmax_local - vmin_local) / 4.0) + 2)))` の結果を代入する。
20. `best_v` に `float(np.clip(v_prev_seed, vmin_local, vmax_local))` の結果を代入する。
21. `best_score` に `float('inf')` の結果を代入する。
22. `env` に `self._forecast_at_time(t_seed, s_seed, drive=True)` の結果を代入する。
23. `slope_pct` に `self._route_value(s_seed, 'slope_pct', 0.0)` の結果を代入する。
24. `headwind_ms` に `self._route_value(s_seed, 'headwind_ms', env.get('headwind_ms', 0.0))` の結果を代入する。
25. `candidate_grid` を順に走査し、各要素を `v_test` に入れて処理する。
26. `out` に `self.model.electrical_balance(v_test / 3.6, slope_pct, z_seed, Tb_seed, env['G_poa'], env['Tcell_C'], headwind_ms=headwind_ms, ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', self._route_value('elev_m', 0.0)))` の結果を代入する。
27. `p_pack` に `float(out['P_pack'])` の結果を代入する。
28. `score` に `abs(p_pack) + 0.35 * (float(v_test) - float(v_prev_seed)) ** 2` の結果を代入する。
29. 条件 `score < best_score` を判定し、真なら内部処理を行う。
30. `best_score` に `score` の結果を代入する。
31. `best_v` に `float(v_test)` の結果を代入する。

32. `u_seed[idx_ctrl]` に `best_v` の結果を代入する。
33. 条件 `idx_ctrl + 1 < len(ctrl_s)` を判定し、真なら内部処理を行う。
34. `ds_ctrl` に `max(0.0, float(ctrl_s[idx_ctrl + 1] - ctrl_s[idx_ctrl]))` の結果を代入する。
35. 上の条件が偽の場合:
36. `ds_ctrl` に `max(float(ds_seq[-1]), float(self.upper_ds_km))` の結果を代入する。
37. `target_s_km` に `s_seed + ds_ctrl` の結果を代入する。
38. `stop_edges` に `[float(stop['s_km']) for stop in control_stops if s_seed + 1e-09 < float(stop['s_km']) <= target_s_km + 1e-09]` の結果を代入する。
39. `seed_edges` に `sorted(set([*stop_edges, float(target_s_km)]))` の結果を代入する。
40. `stopped_in_chunk` に `False` の結果を代入する。
41. `seed_edges` を順に走査し、各要素を `edge_s_km` に入れて処理する。
42. `ds_leg_km` に `max(0.0, float(edge_s_km) - s_seed)` の結果を代入する。
43. 条件 `ds_leg_km > 1e-09` を判定し、真なら内部処理を行う。
44. `dt_seed` に `ds_leg_km / max(best_v, 0.001) * 3600.0` の結果を代入する。
45. `env_leg` に `self._forecast_at_time(t_seed, s_seed, drive=True)` の結果を代入する。
46. `slope_leg` に `self._route_average(s_seed, edge_s_km, 'slope_pct', 0.0)` の結果を代入する。
47. `headwind_leg` に `self._route_value(s_seed, 'headwind_ms', env_leg.get('headwind_ms', 0.0))` の結果を代入する。
48. `out` に `self.model.electrical_balance(best_v / 3.6, slope_leg, z_seed, Tb_seed, env_leg['G_poa'], env_leg['Tcell_C'], headwind_ms=headwind_leg, ambient_temp_c=env_leg.get('Tamb_C'), elevation_m=env_leg.get('elevation_m', self._route_value(s_seed, 'elev_m', 0.0)))` の結果を代入する。
49. `z_seed` に `float(np.clip(self.model.soc_step(z_seed, float(out['P_pack']), dt_seed, current_a=float(out['I']), Tbat_C=Tb_seed, p.soc_min, p.soc_max))` の結果を代入する。
50. `Tb_seed` に `float(np.clip(Tb_seed + dt_seed / 1800.0 * (env_leg['Tamb_C'] - Tb_seed) + float(out['losses_int']) * dt_seed / 50000.0, p.T_min, p.T_max))` の結果を代入する。
51. `t_seed` を `Add` で更新する。
52. `s_seed` に `float(edge_s_km)` の結果を代入する。
53. `(t_seed, z_seed, Tb_seed, control_wait, _)` に `apply_control_stop_at(t_seed, z_seed, Tb_seed, s_seed)` の結果を代入する。
54. `stopped_in_chunk` に `stopped_in_chunk or control_wait > 1e-09` の結果を代入する。
55. `v_prev_seed` に `0.0 if stopped_in_chunk else best_v` の結果を代入する。
56. `np.array(u_seed, dtype=float)` を返す。

代表コード断片

```
def build_balance_seed():
    u_seed = np.array(x0, dtype=float)
    z_seed = float(self.z)
    Tb_seed = float(self.Tb)
    s_seed = float(s0_km)
    t_seed = t0_utc
    v_prev_seed = float(v0)
    for idx_ctrl in range(Nc):
        t_seed, z_seed, Tb_seed, _, _ = step_wait(t_seed, z_seed, Tb_seed, s_seed)
        vmax_local = self._speed_limit_at(s_seed, v_max_kmh)
        vmin_local = v_min_solver
        if self.drive_schedule is not None:
            limits = self.drive_schedule.speed_limits(t_seed)
            if limits is not None:
                vmin_local = max(vmin_local, float(limits[0]))
                vmax_local = min(vmax_local, float(limits[1]))
        if vmax_local < vmin_local:
            u_seed[idx_ctrl] = max(0.0, vmax_local)
            continue
        candidate_grid = np.linspace(vmin_local, vmax_local, num=max(5, min(13, int
((vmax_local - vmin_local) / 4.0) + 2)))
        best_v = float(np.clip(v_prev_seed, vmin_local, vmax_local))
        best_score = float("inf")
        env = self._forecast_at_time(t_seed, s_seed, drive=True)
        slope_pct = self._route_value(s_seed, 'slope_pct', 0.0)
        headwind_ms = self._route_value(s_seed, 'headwind_ms', env.get('headwind_ms'
, 0.0))
        for v_test in candidate_grid:
            out = self.model.electrical_balance(
                v_test / 3.6, slope_pct, z_seed, Tb_seed, env['G_poa'], env['Tcell_C
'],
                headwind_ms=headwind_ms, ambient_temp_c=env.get('Tamb_C'),
                elevation_m=env.get('elevation_m', self._route_value(s_seed, 'elev_m
', 0.0)),
            )
            p_pack = float(out['P_pack'])
            score = abs(p_pack) + 0.35 * ((float(v_test) - float(v_prev_seed)) ** 2)
            if score < best_score:
                best_score = score
    ...
```

L1483 関数 MPCNode._mpc_solve_solar_distance.integrate_stationary_duration

- 行範囲: L1483–L1536
- 所属: MPCNode._mpc_solve_solar_distance
- 定義: integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, *, control_stop)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _forecast_at_time, _route_value, clip, electrical_balance, float, get, max, min, quad_penalty, scheduled_auxiliary_power, soc_step, timedelta
- 戻り値の要点: (t_cursor, float(z), float(Tb), dt_wait, float(wait_cost)) / (t_utc, float(z), float(Tb), 0.0, 0.0)
- 主なローカル代入名: I, P_pack, Tb, V, constraint, dt_local, dt_sample, dt_wait, env, headwind_ms, loss_int, out, remaining, slope_pct, t_cursor, wait_cost, z
- 読み取る主なインスタンス属性: self._forecast_at_time, self._route_value, self.model, self.upper_cost_cfg

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `dt_wait` に `max(0.0, float(dt_wait))` の結果を代入する。
2. 条件 `dt_wait <= 0.0` を判定し、真なら内部処理を行う。
3. `(t_utc, float(z), float(Tb), 0.0, 0.0)` を返す。
4. `slope_pct` に `self._route_value(s_km, 'slope_pct', 0.0)` の結果を代入する。
5. `t_cursor` に `t_utc` の結果を代入する。
6. `remaining` に `dt_wait` の結果を代入する。
7. `dt_sample` に `min(600.0, max(60.0, float(p.dt)))` の結果を代入する。
8. `wait_cost` に `float(self.upper_cost_cfg.w_wait) * dt_wait` の結果を代入する。
9. 条件 `remaining > 1e-09` が成り立つ間くり返す。
10. `dt_local` に `min(dt_sample, remaining)` の結果を代入する。
11. `env` に `self._forecast_at_time(t_cursor, s_km, drive=False, control_stop=control_stop)` の結果を代入する。
12. `headwind_ms` に `self._route_value(s_km, 'headwind_ms', env.get('headwind_ms', 0.0))` の結果を代入する。
13. `out` に `self.model.electrical_balance(0.0, slope_pct, z, Tb, env['G_poa'], env['Tcell_C'], headwind_ms=headwind_ms, aux_power_w=self.model.scheduled_auxiliary_power(is_driving=False, irradiance_wm2=env['G_poa']), ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0)))` の結果を代入する。
14. `P_pack` に `float(out['P_pack'])` の結果を代入する。
15. `loss_int` に `float(out['losses_int'])` の結果を代入する。
16. `I` に `float(out['I'])` の結果を代入する。
17. `V` に `float(out['V'])` の結果を代入する。
18. `z` に `float(np.clip(self.model.soc_step(z, P_pack, dt_local, current_a=I, Tbat_C=Tb), p.soc_min, p.soc_max))` の結果を代入する。
19. `Tb` に `float(np.clip(Tb + dt_local / 1800.0 * (env['Tamb_C'] - Tb) + loss_int * dt_local / 50000.0, p.T_min, p.T_max))` の結果を代入する。
20. `constraint` に `float(self.upper_cost_cfg.constraint_penalty)` の結果を代入する。
21. `wait_cost` を `Add` で更新する。
22. `t_cursor` を `Add` で更新する。
23. `remaining` を `Sub` で更新する。
24. `(t_cursor, float(z), float(Tb), dt_wait, float(wait_cost))` を返す。

代表コード断片

```
def integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, *, control_stop):
    dt_wait = max(0.0, float(dt_wait))
    if dt_wait <= 0.0:
        return t_utc, float(z), float(Tb), 0.0, 0.0
    slope_pct = self._route_value(s_km, 'slope_pct', 0.0)
    t_cursor = t_utc
    remaining = dt_wait
    dt_sample = min(600.0, max(60.0, float(p.dt)))
    wait_cost = float(self.upper_cost_cfg.w_wait) * dt_wait
    while remaining > 1.0e-9:
        dt_local = min(dt_sample, remaining)
        env = self._forecast_at_time(
            t_cursor, s_km, drive=False, control_stop=control_stop
        )
        headwind_ms = self._route_value(s_km, 'headwind_ms', env.get('headwind_ms',
0.0))
        out = self.model.electrical_balance(
            0.0,
            slope_pct,
            z,
            Tb,
            env['G_poa'],
            env['Tcell_C'],
            headwind_ms=headwind_ms,
            aux_power_w=self.model.scheduled_auxiliary_power(
                is_driving=False,
                irradiance_wm2=env['G_poa'],
            ),
            ambient_temp_c=env.get('Tamb_C'),
            elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m',
0.0)),
        )
        P_pack = float(out['P_pack'])
        loss_int = float(out['losses_int'])
        I = float(out['I'])
        V = float(out['V'])
        z = float(np.clip(self.model.soc_step(
...

```

L1538 関数 MPCNode._mpc_solve_solar_distance.step_wait

- 行範囲: L1538–L1545
- 所属: MPCNode._mpc_solve_solar_distance
- 定義: `step_wait(t_utc, z, Tb, s_km)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `integrate_stationary_duration`, `is_drive_time`, `max`, `next_drive_start`, `total_seconds`
- 戻り値の要点: `integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, control_stop=False) / (t_utc, float(z), float(Tb), 0.0, 0.0)`
- 主なローカル代入名: `dt_wait`, `t_start`
- 読み取る主なインスタンス属性: `self.drive_schedule`
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `self.drive_schedule is None or self.drive_schedule.is_drive_time(t_utc)` を判定し、真なら内部処理を行う。
2. `(t_utc, float(z), float(Tb), 0.0, 0.0)` を返す。
3. `t_start` に `self.drive_schedule.next_drive_start(t_utc)` の結果を代入する。
4. `dt_wait` に `max(0.0, (t_start - t_utc).total_seconds())` の結果を代入する。
5. `integrate_stationary_duration(t_utc, z, Tb, s_km, dt_wait, control_stop=False)` を返す。

代表コード断片

```
def step_wait(t_utc, z, Tb, s_km):
    if self.drive_schedule is None or self.drive_schedule.is_drive_time(t_utc):
        return t_utc, float(z), float(Tb), 0.0, 0.0
    t_start = self.drive_schedule.next_drive_start(t_utc)
    dt_wait = max(0.0, (t_start - t_utc).total_seconds())
    return integrate_stationary_duration(
        t_utc, z, Tb, s_km, dt_wait, control_stop=False
    )
```

L1547 関数 `MPCNode._mpc_solve_solar_distance.apply_control_stop_at`

- 行範囲: L1547–L1563
- 所属: `MPCNode._mpc_solve_solar_distance`
- 定義: `apply_control_stop_at(t_utc, z, Tb, s_km)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `float`, `integrate_stationary_duration`
- 戻り値の要点: `(t_utc, float(z), float(Tb), total_wait, total_cost)`
- 主なローカル代入名: `Tb`, `dt_stop`, `stop`, `stop_cost`, `t_utc`, `total_cost`, `total_wait`, `z`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. total_wait に 0.0 の結果を代入する。
2. total_cost に 0.0 の結果を代入する。
3. control_stops を順に走査し、各要素を stop に入れて処理する。
4. 条件 $\text{abs}(\text{float}(\text{stop}['s_km']) - \text{float}(s_km)) > 1e-07$ を判定し、真なら内部処理を行う。
5. Continue 文を実行する。
6. (t_utc, z, Tb, dt_stop, stop_cost) に `integrate_stationary_duration(t_utc, z, Tb, float(stop['s_k']), float(stop['dwell_sec']), control_stop=True)` の結果を代入する。
7. total_wait を Add で更新する。
8. total_cost を Add で更新する。
9. (t_utc, float(z), float(Tb), total_wait, total_cost) を返す。

代表コード断片

```
def apply_control_stop_at(t_utc, z, Tb, s_km):
    total_wait = 0.0
    total_cost = 0.0
    for stop in control_stops:
        if abs(float(stop['s_k']) - float(s_km)) > 1.0e-7:
            continue
        t_utc, z, Tb, dt_stop, stop_cost = integrate_stationary_duration(
            t_utc,
            z,
            Tb,
            float(stop['s_k']),
            float(stop['dwell_sec']),
            control_stop=True,
        )
        total_wait += dt_stop
        total_cost += stop_cost
    return t_utc, float(z), float(Tb), total_wait, total_cost
```

L1565 関数 MPCNode._mpc_solve_solar_distance.cost

- 行範囲: L1565–L1714
- 所属: MPCNode._mpc_solve_solar_distance
- 定義: `cost(u_vec)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_forecast_at_time`, `_route_average`, `_route_value`, `_speed_limit_at`, `apply_control_stop_at`, `bool`, `current_drive_window`, `electrical_balance`, `expand_ctrl`, `float`, `get`, `is_drive_time`
- 戻り値の要点: J
- 主なローカル代入名: I, J, P_mech_wheel, P_pack, P_pv, Tb, Tb_after, Tb_next, V, day_end_crossing, ds_step_km, dt_travel, dt_wait, elapsed_plan_sec, env, forces, headwind_ms, inertial_power_w, k, kinetic_delta_wh
- 読み取る主なインスタンス属性: `self.Tb`, `self._forecast_at_time`, `self._route_average`, `self._route_value`, `self._speed_limit_at`, `self.drive_schedule`, `self.model`, `self.soc_day_end_target`, `self.soc_finish_tol`, `self.upper_cost_cfg`, `self.z`

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `z` に `float(self.z)` の結果を代入する。
2. `Tb` に `float(self.Tb)` の結果を代入する。
3. `s_km` に `float(s0_km)` の結果を代入する。
4. `t_utc` に `t0_utc` の結果を代入する。
5. `v_prev` に `v0` の結果を代入する。
6. `p_pack_prev` に `None` の結果を代入する。
7. `elapsed_plan_sec` に `0.0` の結果を代入する。
8. `J` に `0.0` の結果を代入する。
9. `v_seq` に `expand_ctrl(u_vec)` の結果を代入する。
10. `range(Np)` を順に走査し、各要素を `k` に入れて処理する。
11. `(t_utc, z, Tb, dt_wait, wait_cost)` に `step_wait(t_utc, z, Tb, s_km)` の結果を代入する。
12. `J` を `Add` で更新する。
13. 条件 `dt_wait > 1e-09` を判定し、真なら内部処理を行う。
14. `v_prev` に `0.0` の結果を代入する。
15. `v_k` に `float(v_seq[k])` の結果を代入する。
16. `ds_step_km` に `float(ds_seq[k])` の結果を代入する。
17. `vmax_local` に `self.speed_limit_at(s_km, v_max_kmh)` の結果を代入する。
18. 条件 `vmax_local >= v_min_solver` を判定し、真なら内部処理を行う。
19. `v_k` に `max(v_min_solver, min(v_k, vmax_local))` の結果を代入する。
20. 上の条件が偽の場合:
21. `v_k` に `max(0.0, min(v_k, vmax_local))` の結果を代入する。
22. `limits` に `None` の結果を代入する。
23. 条件 `self.drive_schedule is not None` を判定し、真なら内部処理を行う。
24. `limits` に `self.drive_schedule.speed_limits(t_utc)` の結果を代入する。
25. `dt_travel` に `ds_step_km / max(v_k, 0.001) * 3600.0` の結果を代入する。
26. `env` に `self._forecast_at_time(t_utc, s_km, drive=True)` の結果を代入する。

27. slope_pct に self._route_average(s_km, s_km + ds_step_km, 'slope_pct', 0.0) の結果を代入する。
28. headwind_ms に self._route_value(s_km, 'headwind_ms', env.get('headwind_ms', 0.0)) の結果を代入する。
29. kinetic_delta_wh に $0.5 * p.m * ((v_k / 3.6) ** 2 - (v_{prev} / 3.6) ** 2) / 3600.0$ の結果を代入する。
30. inertial_power_w に kinetic_delta_wh * 3600.0 / max(dt_travel, 1e-09) の結果を代入する。
31. out に self.model.electrical_balance(v_k/3.6, slope_pct, z, Tb, env['G_poa'], env['Tcell_C'], headwind_ms=headwind_ms, inertial_power_w=inertial_power_w, ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0))) の結果を代入する。
32. I に float(out['I']) の結果を代入する。
33. V に float(out['V']) の結果を代入する。
34. P_pv に float(out.get('P_pv', 0.0)) の結果を代入する。
35. P_pack に float(out['P_pack']) の結果を代入する。
36. loss_int に float(out['losses_int']) の結果を代入する。
37. loss_line に float(out.get('losses_line', 0.0)) の結果を代入する。
38. P_mech_wheel に float(out.get('P_mech_wheel', 0.0)) の結果を代入する。
39. kinetic_step_wh に max(0.0, kinetic_delta_wh) の結果を代入する。
40. z_next に self.model.soc_step(z, P_pack, dt_travel, current_a=I, Tbat_C=Tb) の結果を代入する。
41. Tb_next に $Tb + dt_travel / 1800.0 * (env['Tamb_C'] - Tb) + loss_int * dt_travel / 50000.0$ の結果を代入する。
42. forces に self.model.resistive_forces(v_k/3.6, slope_pct, headwind_ms=headwind_ms, ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', self._route_value(s_km, 'elev_m', 0.0))) の結果を代入する。
43. soc_line に None の結果を代入する。
44. 条件 self.drive_schedule is not None and self.soc_day_end_target > 0.0 を判定し、真なら内部処理を行う。
45. win に self.drive_schedule.current_drive_window(t_utc) の結果を代入する。
46. 条件 win is not None を判定し、真なら内部処理を行う。
47. (t_start, t_end) に win の結果を代入する。
48. 条件 t_end > t_start を判定し、真なら内部処理を行う。
49. t_next に t_utc + timedelta(seconds=dt_travel) の結果を代入する。
50. day_end_crossing に bool(self.drive_schedule is not None and self.drive_schedule.is_drive_time(t_utc) and (not self.drive_schedule.is_drive_time(t_next))) の結果を代入する。
51. elapsed_plan_sec を Add で更新する。
52. J を Add で更新する。
53. s_next_km に s_km + ds_step_km の結果を代入する。
54. (t_after, z_after, Tb_after, stop_wait, stop_cost) に apply_control_stop_at(t_next, z_next, Tb_next, s_next_km) の結果を代入する。
55. J を Add で更新する。
56. elapsed_plan_sec を Add で更新する。

57. t_utc に t_after の結果を代入する。
58. s_km に s_next_km の結果を代入する。
59. (z, Tb) に (z_after, Tb_after) の結果を代入する。
60. 条件 stop_wait > 1e-09 を判定し、真なら内部処理を行う。
61. v_prev に 0.0 の結果を代入する。
62. p_pack_prev に None の結果を代入する。
63. 上の条件が偽の場合:
64. v_prev に v_k の結果を代入する。
65. p_pack_prev に P_pack の結果を代入する。
66. J を Add で更新する。
67. J を返す。

代表コード断片

```
def cost(u_vec):
    z = float(self.z)
    Tb = float(self.Tb)
    s_km = float(s0_km)
    t_utc = t0_utc
    v_prev = v0
    p_pack_prev = None
    elapsed_plan_sec = 0.0
    J = 0.0
    v_seq = expand_ctrl(u_vec)
    for k in range(Np):
        t_utc, z, Tb, dt_wait, wait_cost = step_wait(t_utc, z, Tb, s_km)
        J += wait_cost
        if dt_wait > 1.0e-9:
            v_prev = 0.0
        v_k = float(v_seq[k])
        ds_step_km = float(ds_seq[k])
        vmax_local = self._speed_limit_at(s_km, v_max_kmh)
        if vmax_local >= v_min_solver:
            v_k = max(v_min_solver, min(v_k, vmax_local))
        else:
            v_k = max(0.0, min(v_k, vmax_local))
        limits = None
        if self.drive_schedule is not None:
            limits = self.drive_schedule.speed_limits(t_utc)

        dt_travel = ds_step_km / max(v_k, 1.0e-3) * 3600.0
        env = self._forecast_at_time(t_utc, s_km, drive=True)
        slope_pct = self._route_average(
            s_km, s_km + ds_step_km, 'slope_pct', 0.0
        )
        headwind_ms = self._route_value(s_km, 'headwind_ms', env.get('headwind_ms',
0.0))

        kinetic_delta_wh = 0.5 * p.m * (
            (v_k / 3.6) ** 2 - (v_prev / 3.6) ** 2
        ) / 3600.0
    ...
```

L1816 関数 MPCNode._publish_upper_plan

- 行範囲: L1816–L1821
- 所属: MPCNode
- 定義: `_publish_upper_plan(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32MultiArray`, `float`, `publish`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `msg`, `v`
- 読み取る主なインスタンス属性: `self.plan_dt_sec`, `self.pub_plan`, `self.v_plan_kmh`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `not self.v_plan_kmh` を判定し、真なら内部処理を行う。
2. を返す。
3. `msg` に `Float32MultiArray()` の結果を代入する。
4. `msg.data` に `[float(self.plan_dt_sec)] + [float(v) for v in self.v_plan_kmh]` の結果を代入する。
5. `self.pub_plan.publish(...)` を実行する。

代表コード断片

```
def _publish_upper_plan(self):
    if not self.v_plan_kmh:
        return
    msg = Float32MultiArray()
    msg.data = [float(self.plan_dt_sec)] + [float(v) for v in self.v_plan_kmh]
    self.pub_plan.publish(msg)
```

L1823 関数 MPCNode._publish_lower_plan

- 行範囲: L1823–L1828
- 所属: MPCNode
- 定義: `_publish_lower_plan(self, v_seq_ms)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32MultiArray`, `float`, `publish`

- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `msg, v`
- 読み取る主なインスタンス属性: `self.lower_dt, self.pub_lower_plan`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `not v_seq_ms` を判定し、真なら内部処理を行う。
2. を返す。
3. `msg` に `Float32MultiArray()` の結果を代入する。
4. `msg.data` に `[float(self.lower_dt)] + [float(v * 3.6) for v in v_seq_ms]` の結果を代入する。
5. `self.pub_lower_plan.publish(...)` を実行する。

代表コード断片

```
def _publish_lower_plan(self, v_seq_ms):
    if not v_seq_ms:
        return
    msg = Float32MultiArray()
    msg.data = [float(self.lower_dt)] + [float(v * 3.6) for v in v_seq_ms]
    self.pub_lower_plan.publish(msg)
```

L1830 関数 `MPCNode._publish_plan_path`

- 行範囲: L1830–L1856
- 所属: `MPCNode`
- 定義: `_publish_plan_path(self, data)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path, PoseStamped, append, bool, float, get_clock, get_parameter, isfinite, len, now, publish, to_msg`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `path, pose, s_tmp, seg, v_kmh, v_list`
- 読み取る主なインスタンス属性: `self.get_clock, self.get_parameter, self.model, self.pub_path, self.s_km, self.s_meas, self.upper_plan_mode, self.v_plan_kmh, self.v_plan_segments, self.v_upper_cmd`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 2、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `len(data) == 0` を判定し、真なら内部処理を行う。
2. を返す。
3. `path` に `Path()` の結果を代入する。
4. `path.header.stamp` に `self.get_clock().now().to_msg()` の結果を代入する。
5. `path.header.frame_id` に 'map' の結果を代入する。
6. `s_tmp` に `float(self.s_km)` の結果を代入する。
7. 条件 `bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
8. `s_tmp` に `float(self.s_meas)` の結果を代入する。
9. 条件 `self.upper_plan_mode == 'distance' and self.v_plan_segments` を判定し、真なら内部処理を行う。
10. `self.v_plan_segments` を順に走査し、各要素を `seg` に入れて処理する。
11. `s_tmp` を Add で更新する。
12. `pose` に `PoseStamped()` の結果を代入する。
13. `pose.header` に `path.header` の結果を代入する。
14. `pose.pose.position.x` に `s_tmp` の結果を代入する。
15. `pose.pose.position.y` に `0.0` の結果を代入する。
16. `path.poses.append(...)` を実行する。
17. 上の条件が偽の場合:
18. `v_list` に `self.v_plan_kmh if self.v_plan_kmh else [float(self.v_upper_cmd)] * len(data)` の結果を代入する。
19. `v_list[:len(data)]` を順に走査し、各要素を `v_kmh` に入れて処理する。
20. `s_tmp` を Add で更新する。
21. `pose` に `PoseStamped()` の結果を代入する。
22. `pose.header` に `path.header` の結果を代入する。
23. `pose.pose.position.x` に `s_tmp` の結果を代入する。
24. `pose.pose.position.y` に `0.0` の結果を代入する。
25. `path.poses.append(...)` を実行する。
26. `self.pub_path.publish(...)` を実行する。

代表コード断片

```
def _publish_plan_path(self, data):
    if len(data) == 0:
        return
    path = Path()
    path.header.stamp = self.get_clock().now().to_msg()
    path.header.frame_id = 'map'
    s_tmp = float(self.s_kmh)
    if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
        s_tmp = float(self.s_meas)
    if self.upper_plan_mode == 'distance' and self.v_plan_segments:
        for seg in self.v_plan_segments:
            s_tmp += float(seg['v_kmh']) * (seg['dt_sec'] / 3600.0)
            pose = PoseStamped()
            pose.header = path.header
            pose.pose.position.x = s_tmp
            pose.pose.position.y = 0.0
            path.poses.append(pose)
    else:
        v_list = self.v_plan_kmh if self.v_plan_kmh else [float(self.v_upper_cmd)] * len
(data)
        for v_kmh in v_list[:len(data)]:
            s_tmp += float(v_kmh) * (self.model.p.dt / 3600.0)
            pose = PoseStamped()
            pose.header = path.header
            pose.pose.position.x = s_tmp
            pose.pose.position.y = 0.0
            path.poses.append(pose)
    self.pub_path.publish(path)
```

L1858 関数 MPCNode._publish_metrics

- 行範囲: L1858–L1895
- 所属: MPCNode
- 定義: `_publish_metrics(self, d0:dict, v_exec_kmh:float, s_for_profile:float)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32MultiArray`, `_route_value`, `abs`, `electrical_balance`, `float`, `get`, `publish`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `I`, `I_motor`, `P_mech_wheel`, `P_motor_elec`, `P_pack`, `P_pv`, `V`, `headwind_ms`, `msg`, `out`, `slope_pct`
- 読み取る主なインスタンス属性: `self.Tb`, `self._route_value`, `self.model`, `self.pub_metrics`, `self.route_profile`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. slope_pct に d0.get('slope_pct', 0.0) の結果を代入する。
2. 条件 self.route_profile is not None を判定し、真なら内部処理を行う。
3. slope_pct に self._route_value(s_for_profile, 'slope_pct', slope_pct) の結果を代入する。
4. headwind_ms に d0.get('headwind_ms', 0.0) の結果を代入する。
5. 条件 self.route_profile is not None を判定し、真なら内部処理を行う。
6. headwind_ms に self._route_value(s_for_profile, 'headwind_ms', headwind_ms) の結果を代入する。
7. out に self.model.electrical_balance(v_exec_kmh / 3.6, slope_pct, self.z, self.Tb, d0.get('G_poa', 0.0), d0.get('Tcell_C', 40.0), headwind_ms=headwind_ms, ambient_temp_c=d0.get('Tamb_C'), elevation_m=d0.get('elevation_m', self._route_value('elev_m', 0.0))) の結果を代入する。
8. V に float(out['V']) の結果を代入する。
9. I に float(out['I']) の結果を代入する。
10. P_pv に float(out['P_pv']) の結果を代入する。
11. P_pack に float(out['P_pack']) の結果を代入する。
12. P_mech_wheel に float(out.get('P_mech_wheel', out.get('P_mech', 0.0))) の結果を代入する。
13. P_motor_elec に float(out.get('P_dc_to_drv', 0.0)) - float(out.get('P_reg_to_dc', 0.0)) の結果を代入する。
14. I_motor に P_motor_elec / V if abs(V) > 0.001 else 0.0 の結果を代入する。
15. msg に Float32MultiArray() の結果を代入する。
16. msg.data に [float(V), float(I), float(self.z), float(P_motor_elec), float(I_motor), float(P_pv), float(v_exec_kmh), float(P_mech_wheel), float(P_pack)] の結果を代入する。
17. self.pub_metrics.publish(...) を実行する。

代表コード断片

```
def _publish_metrics(self, d0: dict, v_exec_kmh: float, s_for_profile: float):
    slope_pct = d0.get('slope_pct', 0.0)
    if self.route_profile is not None:
        slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
    headwind_ms = d0.get('headwind_ms', 0.0)
    if self.route_profile is not None:
        headwind_ms = self._route_value(s_for_profile, 'headwind_ms', headwind_ms)
    out = self.model.electrical_balance(
        v_exec_kmh / 3.6,
        slope_pct,
        self.z,
        self.Tb,
        d0.get('G_poa', 0.0),
        d0.get('Tcell_C', 40.0),
        headwind_ms=headwind_ms,
        ambient_temp_c=d0.get('Tamb_C'),
        elevation_m=d0.get('elevation_m', self._route_value(s_for_profile, 'elev_m',
0.0))),
    )
    V = float(out['V'])
    I = float(out['I'])
    P_pv = float(out['P_pv'])
    P_pack = float(out['P_pack'])
    P_mech_wheel = float(out.get('P_mech_wheel', out.get('P_mech', 0.0)))
```

```

        P_motor_elec = float(out.get('P_dc_to_drv', 0.0)) - float(out.get('P_reg_to_dc',
0.0))
        I_motor = P_motor_elec / V if abs(V) > 1e-3 else 0.0
        msg = Float32MultiArray()
        msg.data = [
            float(V),
            float(I),
            float(self.z),
            float(P_motor_elec),
            float(I_motor),
            float(P_pv),
            float(v_exec_kmh),
            float(P_mech_wheel),
        ]
    ...

```

L1897 関数 MPCNode._publish_summary

- 行範囲: L1897–L1927
- 所属: MPCNode
- 定義: `_publish_summary(self, _v_exec_kmh:float=math.nan) ->None`
- docstring: Publish current state even while a long upper solve is in progress.
- このブロックが直接呼ぶ主な関数・メソッド: `Float32MultiArray`, `all`, `bool`, `float`, `isna`, `len`, `lower`, `max`, `monotonic`, `notna`, `now`, `publish`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `elapsed`, `has_time`, `mode`, `now`, `sec_to_next`, `st`, `stop_remaining_sec`, `t_next`
- 読み取る主なインスタンス属性: `self.Tb`, `self.completed_control_stops`, `self.control_stop_end_monotonic`, `self.control_stop_hold`, `self.df`, `self.forecast_start_time`, `self.forecast_time_mode`, `self.forecast_time_offset`, `self.k`, `self.model`, `self.pub_status`, `self.s_km`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `mode` に `str(self.forecast_time_mode).lower()` の結果を代入する。
2. `has_time` に `'time' in self.df.columns and (not self.df['time'].isna().all())` の結果を代入する。
3. 条件 `mode == 'auto'` を判定し、真なら内部処理を行う。
4. `mode` に `'absolute' if has_time else 'relative'` の結果を代入する。
5. 条件 `has_time and mode == 'absolute'` を判定し、真なら内部処理を行う。

6. 条件 `self.k + 1 < len(self.df)` and `pd.notna(self.df['time'].iloc[self.k + 1])` を判定し、真なら内部処理を行う。
7. `t_next` に `self.df['time'].iloc[self.k + 1].to_pydatetime()` の結果を代入する。
8. `sec_to_next` に `max(0.0, (t_next - datetime.now(timezone.utc)).total_seconds())` の結果を代入する。
9. 上の条件が偽の場合:
10. `sec_to_next` に `0.0` の結果を代入する。
11. 上の条件が偽の場合:
12. `now` に `datetime.now(timezone.utc) + timedelta(seconds=self.forecast_time_offset)` の結果を代入する。
13. `elapsed` に `max(0.0, (now - self.forecast_start_time).total_seconds())` の結果を代入する。
14. `sec_to_next` に `max(0.0, self.model.p.dt - elapsed % self.model.p.dt)` の結果を代入する。
15. `stop_remaining_sec` に `0.0` の結果を代入する。
16. 条件 `self.control_stop_end_monotonic is not None` を判定し、真なら内部処理を行う。
17. `stop_remaining_sec` に `max(0.0, self.control_stop_end_monotonic - time.monotonic())` の結果を代入する。
18. `st` に `Float32MultiArray()` の結果を代入する。
19. `st.data` に `[float(self.z), float(self.Tb), float(self.s_km), float(self.k), float(sec_to_next), float(bool(self.control_stop_hold)), float(stop_remaining_sec), float(len(self.completed_control_stops))]` の結果を代入する。
20. `self.pub_status.publish(...)` を実行する。

代表コード断片

```
def _publish_summary(self, _v_exec_kmh: float = math.nan) -> None:
    """Publish current state even while a long upper solve is in progress."""
    mode = str(self.forecast_time_mode).lower()
    has_time = ('time' in self.df.columns) and (not self.df['time'].isna().all())
    if mode == 'auto':
        mode = 'absolute' if has_time else 'relative'
    if has_time and mode == 'absolute':
        if self.k + 1 < len(self.df) and pd.notna(self.df['time'].iloc[self.k + 1]):
            t_next = self.df['time'].iloc[self.k + 1].to_pydatetime()
            sec_to_next = max(0.0, (t_next - datetime.now(timezone.utc)).total_seconds())
        )
        else:
            sec_to_next = 0.0
    else:
        now = datetime.now(timezone.utc) + timedelta(seconds=self.forecast_time_offset)
        elapsed = max(0.0, (now - self.forecast_start_time).total_seconds())
        sec_to_next = max(0.0, self.model.p.dt - (elapsed % self.model.p.dt))
    stop_remaining_sec = 0.0
    if self.control_stop_end_monotonic is not None:
        stop_remaining_sec = max(0.0, self.control_stop_end_monotonic - time.monotonic())
    )
    st = Float32MultiArray()
    st.data = [
        float(self.z),
        float(self.Tb),
        float(self.s_km),
        float(self.k),
        float(sec_to_next),
        float(bool(self.control_stop_hold)),
        float(stop_remaining_sec),
        float(len(self.completed_control_stops)),
    ]
    self.pub_status.publish(st)
```

L1929 関数 MPCNode._interp_upper_speed

- 行範囲: L1929–L1950
- 所属: MPCNode
- 定義: `_interp_upper_speed(self, t_sec: float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `floor`, `int`, `len`
- 戻り値の要点: `float((1.0 - alpha) * self.v_plan_kmh[i] + alpha * self.v_plan_kmh[i + 1]) / float(self.v_plan_segments[-1]['v_kmh']) / float(self.v_upper_cmd) / float(self.v_plan_kmh[0])`
- 主なローカル代入名: `acc`, `acc_next`, `alpha`, `dt`, `i`, `idx`, `seg`
- 読み取る主なインスタンス属性: `self.plan_dt_sec`, `self.upper_plan_mode`, `self.v_plan_kmh`, `self.v_plan_segments`, `self.v_upper_cmd`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `self.upper_plan_mode == 'distance' and self.v_plan_segments` を判定し、真なら内部処理を行う。
2. `acc` に 0.0 の結果を代入する。
3. `self.v_plan_segments` を順に走査し、各要素を `seg` に入れて処理する。
4. `acc_next` に `acc + seg['dt_sec']` の結果を代入する。
5. 条件 `t_sec <= acc_next` を判定し、真なら内部処理を行う。
6. `float(seg['v_kmh'])` を返す。
7. `acc` に `acc_next` の結果を代入する。
8. `float(self.v_plan_segments[-1]['v_kmh'])` を返す。
9. 条件 `not self.v_plan_kmh` を判定し、真なら内部処理を行う。
10. `float(self.v_upper_cmd)` を返す。
11. `dt` に `float(self.plan_dt_sec)` の結果を代入する。
12. 条件 `dt <= 0.0` を判定し、真なら内部処理を行う。
13. `float(self.v_plan_kmh[0])` を返す。
14. `idx` に `t_sec / dt` の結果を代入する。
15. `i` に `int(math.floor(idx))` の結果を代入する。

16. 条件 $i \leq 0$ を判定し、真なら内部処理を行う。
17. `float(self.v_plan_kmh[0])` を返す。
18. 条件 $i \geq \text{len}(\text{self.v_plan_kmh}) - 1$ を判定し、真なら内部処理を行う。
19. `float(self.v_plan_kmh[-1])` を返す。
20. `alpha` に `idx - i` の結果を代入する。
21. `float((1.0 - alpha) * self.v_plan_kmh[i] + alpha * self.v_plan_kmh[i + 1])` を返す。

代表コード断片

```
def _interp_upper_speed(self, t_sec: float) -> float:
    if self.upper_plan_mode == 'distance' and self.v_plan_segments:
        acc = 0.0
        for seg in self.v_plan_segments:
            acc_next = acc + seg['dt_sec']
            if t_sec <= acc_next:
                return float(seg['v_kmh'])
            acc = acc_next
        return float(self.v_plan_segments[-1]['v_kmh'])
    if not self.v_plan_kmh:
        return float(self.v_upper_cmd)
    dt = float(self.plan_dt_sec)
    if dt <= 0.0:
        return float(self.v_plan_kmh[0])
    idx = t_sec / dt
    i = int(math.floor(idx))
    if i <= 0:
        return float(self.v_plan_kmh[0])
    if i >= len(self.v_plan_kmh) - 1:
        return float(self.v_plan_kmh[-1])
    alpha = idx - i
    return float((1.0 - alpha) * self.v_plan_kmh[i] + alpha * self.v_plan_kmh[i + 1])
```

L1952 関数 `MPCNode._distance_plan_speed`

- 行範囲: L1952–L1956
- 所属: `MPCNode`
- 定義: `_distance_plan_speed(self, s_km: float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `plan_segment_index`
- 戻り値の要点: `float(self.v_plan_segments[idx]['v_kmh']) / float(self.v_upper_cmd)`
- 主なローカル代入名: `idx`
- 読み取る主なインスタンス属性: `self.v_plan_segments`, `self.v_upper_cmd`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `idx` に `plan_segment_index(self.v_plan_segments or [], s_km)` の結果を代入する。
2. 条件 `idx < 0` を判定し、真なら内部処理を行う。
3. `float(self.v_upper_cmd)` を返す。
4. `float(self.v_plan_segments[idx]['v_kmh'])` を返す。

代表コード断片

```
def _distance_plan_speed(self, s_km: float) -> float:
    idx = plan_segment_index(self.v_plan_segments or [], s_km)
    if idx < 0:
        return float(self.v_upper_cmd)
    return float(self.v_plan_segments[idx]['v_kmh'])
```

L1958 関数 `MPCNode._tau_max_for_mode`

- 行範囲: L1958–L1963
- 所属: `MPCNode`
- 定義: `_tau_max_for_mode(self, maps, mode: str) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`
- 戻り値の要点: `float(max(maps[key][1])) / 0.0`
- 主なローカル代入名: `key`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. key に mode if mode in maps else 'default' の結果を代入する。
2. 例外処理を伴う try ブロックを実行する。
3. float(max(maps[key][1])) を返す。
4. Exception を捕捉した場合:
5. 0.0 を返す。

代表コード断片

```
def _tau_max_for_mode(self, maps, mode: str) -> float:
    key = mode if mode in maps else 'default'
    try:
        return float(max(maps[key][1]))
    except Exception:
        return 0.0
```

L1965 関数 MPCNode._tau_limits

- 行範囲: L1965–L1979
- 所属: MPCNode
- 定義: _tau_limits(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _tau_max_for_mode, lower, str
- 戻り値の要点: (tau_drive, tau_regen)
- 主なローカル代入名: mode, tau_drive, tau_regen
- 読み取る主なインスタンス属性: self._tau_max_for_mode, self.model
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. mode に str(self.model.drive_mode or 'default').lower() の結果を代入する。
2. 条件 mode in ('eco', 'power') を判定し、真なら内部処理を行う。
3. tau_drive に self._tau_max_for_mode(self.model.maps_drive, mode) の結果を代入する。
4. tau_regen に self._tau_max_for_mode(self.model.maps_regen, mode) の結果を代入する。
5. 上の条件が偽の場合:

6. tau_drive に self._tau_max_for_mode(self.model.maps_drive, 'power') if 'power' in self.model.maps_drive else self._tau_max_for_mode(self.model.maps_drive, 'eco') if 'eco' in self.model.maps_drive else self._tau_max_for_mode(self.model.maps_drive, 'default') の結果を代入する。
7. tau_regen に self._tau_max_for_mode(self.model.maps_regen, 'power') if 'power' in self.model.maps_regen else self._tau_max_for_mode(self.model.maps_regen, 'eco') if 'eco' in self.model.maps_regen else self._tau_max_for_mode(self.model.maps_regen, 'default') の結果を代入する。
8. 条件 tau_regen <= 0.0 を判定し、真なら内部処理を行う。
9. tau_regen に tau_drive の結果を代入する。
10. (tau_drive, tau_regen) を返す。

代表コード断片

```
def _tau_limits(self):
    mode = str(self.model.drive_mode or 'default').lower()
    if mode in ('eco', 'power'):
        tau_drive = self._tau_max_for_mode(self.model.maps_drive, mode)
        tau_regen = self._tau_max_for_mode(self.model.maps_regen, mode)
    else:
        tau_drive = self._tau_max_for_mode(self.model.maps_drive, 'power') if 'power' in
self.model.maps_drive else \
self._tau_max_for_mode(self.model.maps_drive, 'eco') if 'eco' in self.model.
maps_drive else \
self._tau_max_for_mode(self.model.maps_drive, 'default')
        tau_regen = self._tau_max_for_mode(self.model.maps_regen, 'power') if 'power' in
self.model.maps_regen else \
self._tau_max_for_mode(self.model.maps_regen, 'eco') if 'eco' in self.model.
maps_regen else \
self._tau_max_for_mode(self.model.maps_regen, 'default')
    if tau_regen <= 0.0:
        tau_regen = tau_drive
    return tau_drive, tau_regen
```

L1981 関数 MPCNode._traction_force

- 行範囲: L1981–L1986
- 所属: MPCNode
- 定義: _traction_force(self, tau_nm:float)->float
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, int, max
- 戻り値の要点: float(tau_nm) * float(p.gear_ratio) * float(p.gear_eta) * motor_count / wheel_r
- 主なローカル代入名: motor_count, p, wheel_r
- 読み取る主なインスタンス属性: self.model
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. p に self.model.p の結果を代入する。
2. wheel_r に max(0.001, float(p.wheel_radius)) の結果を代入する。
3. motor_count に int(p.motor_count) if int(p.motor_count) > 0 else int(p.driven_wheel_count or 1) の結果を代入する。
4. motor_count に max(1, motor_count) の結果を代入する。
5. float(tau_nm) * float(p.gear_ratio) * float(p.gear_eta) * motor_count / wheel_r を返す。

代表コード断片

```
def _traction_force(self, tau_nm: float) -> float:
    p = self.model.p
    wheel_r = max(1e-3, float(p.wheel_radius))
    motor_count = int(p.motor_count) if int(p.motor_count) > 0 else int(p.
driven_wheel_count or 1)
    motor_count = max(1, motor_count)
    return float(tau_nm) * float(p.gear_ratio) * float(p.gear_eta) * motor_count /
wheel_r
```

L1988 関数 MPCNode._pack_from_tau

- 行範囲: L1988–L2010
- 所属: MPCNode
- 定義: _pack_from_tau(self, v_ms: float, tau_nm: float, z: float, Tb: float, env: dict)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: battery_iv, dict, eff_drive, eff_regen, float, int, max, pv_power_mppt
- 戻り値の要点: dict(P_pack=P_pack, I=I, V=V, loss_int=loss_int, eff=eff, P_pv=P_pv, P_elec=P_elec)
- 主なローカル代入名: I, P_elec, P_mech_motor, P_pack, P_pv, Rint, V, eff, iv, loss_int, motor_count, omega_m, p, wheel_r
- 読み取る主なインスタンス属性: self.model
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. p に self.model.p の結果を代入する。
2. wheel_r に max(0.001, float(p.wheel_radius)) の結果を代入する。
3. motor_count に int(p.motor_count) if int(p.motor_count) > 0 else int(p.driven_wheel_count or 1) の結果を代入する。
4. motor_count に max(1, motor_count) の結果を代入する。
5. omega_m に float(v_ms) / wheel_r * float(p.gear_ratio) の結果を代入する。
6. P_mech_motor に float(tau_nm) * omega_m * motor_count の結果を代入する。
7. 条件 tau_nm >= 0.0 を判定し、真なら内部処理を行う。
8. eff に float(self.model.eff_drive(v_ms, tau_nm)) の結果を代入する。
9. eff に max(0.001, eff) の結果を代入する。
10. P_elec に P_mech_motor / (eff * float(p.inverter_eta)) の結果を代入する。
11. 上の条件が偽の場合:
12. eff に float(self.model.eff_regen(v_ms, tau_nm)) の結果を代入する。
13. eff に max(0.001, eff) の結果を代入する。
14. P_elec に P_mech_motor * eff * float(p.inverter_eta) の結果を代入する。
15. P_pv に float(self.model.pv_power_mppt(env['G_poa'], env['Tcell_C'])) の結果を代入する。
16. P_pack に P_elec + float(p.P_aux) - P_pv の結果を代入する。
17. iv に self.model.battery_iv(P_pack, z, Tb) の結果を代入する。
18. I に float(iv['I']) の結果を代入する。
19. V に float(iv['V']) の結果を代入する。
20. Rint に float(iv['Rint']) の結果を代入する。
21. loss_int に I * I * Rint の結果を代入する。
22. dict(P_pack=P_pack, I=I, V=V, loss_int=loss_int, eff=eff, P_pv=P_pv, P_elec=P_elec) を返す。

代表コード断片

```
def _pack_from_tau(self, v_ms: float, tau_nm: float, z: float, Tb: float, env: dict):
    p = self.model.p
    wheel_r = max(1e-3, float(p.wheel_radius))
    motor_count = int(p.motor_count) if int(p.motor_count) > 0 else int(p.
driven_wheel_count or 1)
    motor_count = max(1, motor_count)
    omega_m = float(v_ms) / wheel_r * float(p.gear_ratio)
    P_mech_motor = float(tau_nm) * omega_m * motor_count
    if tau_nm >= 0.0:
        eff = float(self.model.eff_drive(v_ms, tau_nm))
        eff = max(1.0e-3, eff)
        P_elec = P_mech_motor / (eff * float(p.inverter_eta))
    else:
        eff = float(self.model.eff_regen(v_ms, tau_nm))
        eff = max(1.0e-3, eff)
        P_elec = P_mech_motor * eff * float(p.inverter_eta)
    P_pv = float(self.model.pv_power_mppt(env['G_poa'], env['Tcell_C']))
```



```

P_pack = P_elec + float(p.P_aux) - P_pv
iv = self.model.battery_iv(P_pack, z, Tb)
I = float(iv['I'])
V = float(iv['V'])
Rint = float(iv['Rint'])
loss_int = I * I * Rint
return dict(P_pack=P_pack, I=I, V=V, loss_int=loss_int, eff=eff, P_pv=P_pv, P_elec=
P_elec)

```

L2012 関数 MPCNode._build_lower_ref

- 行範囲: L2012–L2037
- 所属: MPCNode
- 定義: `_build_lower_ref(self, base_time_utc, s_km:float, d0:dict)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_interp_upper_speed`, `_soc_guard_speed`, `_speed_limit_at`, `append`, `clip`, `float`, `get_parameter`, `max`, `min`, `monotonic`, `range`, `speed_limits`
- 戻り値の要点: `ref`
- 主なローカル代入名: `guard_speed`, `i`, `limits`, `offset`, `ref`, `s_tmp`, `soc_guard`, `t_sec`, `t_utc`, `v_ref`, `vmax_kmh`, `vmax_local`, `vmin_kmh`
- 読み取る主なインスタンス属性: `self._interp_upper_speed`, `self._soc_guard_speed`, `self._speed_limit_at`, `self.drive_schedule`, `self.get_parameter`, `self.lower_N`, `self.lower_dt`, `self.model`, `self.plan_start_monotonic`, `self.v_upper_cmd`, `self.z`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `ref` に `[]` の結果を代入する。
2. `s_tmp` に `float(s_km)` の結果を代入する。
3. `offset` に `0.0` の結果を代入する。
4. 条件 `self.plan_start_monotonic is not None` を判定し、真なら内部処理を行う。
5. `offset` に `max(0.0, time.monotonic() - self.plan_start_monotonic)` の結果を代入する。
6. `soc_guard` に `float(self.get_parameter('soc_guard_margin').value)` の結果を代入する。
7. `guard_speed` に `None` の結果を代入する。
8. 条件 `self.z <= self.model.p.soc_min + soc_guard` を判定し、真なら内部処理を行う。
9. `guard_speed` に `self._soc_guard_speed(self.v_upper_cmd, s_tmp, d0)` の結果を代入する。

10. `range(max(1, self.lower_N))` を順に走査し、各要素を `i` に入れて処理する。
11. `t_sec` に `offset + i * self.lower_dt` の結果を代入する。
12. `v_ref` に `self._interp_upper_speed(t_sec)` の結果を代入する。
13. 条件 `guard_speed is not None` を判定し、真なら内部処理を行う。
14. `v_ref` に `min(v_ref, guard_speed)` の結果を代入する。
15. `vmax_local` に `self._speed_limit_at(s_tmp, self.get_parameter('v_max_kmh').value)` の結果を代入する。
16. `v_ref` に `min(v_ref, vmax_local)` の結果を代入する。
17. 条件 `self.drive_schedule is not None and base_time_utc is not None` を判定し、真なら内部処理を行う。
18. `t_utc` に `base_time_utc + timedelta(seconds=t_sec)` の結果を代入する。
19. `limits` に `self.drive_schedule.speed_limits(t_utc)` の結果を代入する。
20. 条件 `limits is not None` を判定し、真なら内部処理を行う。
21. `(vmin_kmh, vmax_kmh)` に `limits` の結果を代入する。
22. `v_ref` に `float(np.clip(v_ref, vmin_kmh, vmax_kmh))` の結果を代入する。
23. `ref.append(...)` を実行する。
24. `s_tmp` を `Add` で更新する。
25. `ref` を返す。

代表コード断片

```
def _build_lower_ref(self, base_time_utc, s_kmh: float, d0: dict):
    ref = []
    s_tmp = float(s_kmh)
    offset = 0.0
    if self.plan_start_monotonic is not None:
        offset = max(0.0, time.monotonic() - self.plan_start_monotonic)
    soc_guard = float(self.get_parameter('soc_guard_margin').value)
    guard_speed = None
    if self.z <= (self.model.p.soc_min + soc_guard):
        guard_speed = self._soc_guard_speed(self.v_upper_cmd, s_tmp, d0)
    for i in range(max(1, self.lower_N)):
        t_sec = offset + i * self.lower_dt
        v_ref = self._interp_upper_speed(t_sec)
        if guard_speed is not None:
            v_ref = min(v_ref, guard_speed)
        vmax_local = self._speed_limit_at(s_tmp, self.get_parameter('v_max_kmh').value)
        v_ref = min(v_ref, vmax_local)
        if self.drive_schedule is not None and base_time_utc is not None:
            t_utc = base_time_utc + timedelta(seconds=t_sec)
            limits = self.drive_schedule.speed_limits(t_utc)
            if limits is not None:
                vmin_kmh, vmax_kmh = limits
                v_ref = float(np.clip(v_ref, vmin_kmh, vmax_kmh))
        ref.append(float(v_ref) / 3.6)
        s_tmp += float(v_ref) * (self.lower_dt / 3600.0)
    return ref
```

L2039 関数 MPCNode._lower_rollout

- 行範囲: L2039–L2070
- 所属: MPCNode
- 定義: `_lower_rollout(self, v0_ms:float, u_seq:np.ndarray, env:dict, z0:float, Tb0:float, tau_drive:float, tau_regen:float)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_pack_from_tau`, `_traction_force`, `append`, `float`, `get`, `max`, `resistive_forces`, `soc_step`
- 戻り値の要点: `v_seq`
- 主なローカル代入名: `F_res`, `F_trac`, `Tb`, `a`, `forces`, `p`, `pack`, `tau`, `u`, `v`, `v_seq`, `z`
- 読み取る主なインスタンス属性: `self._pack_from_tau`, `self._traction_force`, `self.lower_dt`, `self.model`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `p` に `self.model.p` の結果を代入する。
2. `v` に `float(v0_ms)` の結果を代入する。
3. `z` に `float(z0)` の結果を代入する。
4. `Tb` に `float(Tb0)` の結果を代入する。
5. `v_seq` に `[]` の結果を代入する。
6. `u_seq` を順に走査し、各要素を `u` に入れて処理する。
7. `u` に `float(u)` の結果を代入する。
8. 条件 `u >= 0.0` を判定し、真なら内部処理を行う。
9. `tau` に `u * tau_drive` の結果を代入する。
10. 上の条件が偽の場合:
11. `tau` に `u * tau_regen` の結果を代入する。
12. `forces` に `self.model.resistive_forces(v, env['slope_pct'], env['headwind_ms'], ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0))` の結果を代入する。
13. `F_res` に `float(forces['F_total'])` の結果を代入する。
14. `F_trac` に `self._traction_force(tau)` の結果を代入する。
15. `a` に `(F_trac - F_res) / float(p.m)` の結果を代入する。

16. `v` に `max(0.0, v + a * self.lower_dt)` の結果を代入する。
17. `pack` に `self._pack_from_tau(v, tau, z, Tb, env)` の結果を代入する。
18. `z` に `self.model.soc_step(z, float(pack['P_pack']), self.lower_dt, current_a=float(pack['I']), Tbat_C=Tb)` の結果を代入する。
19. `Tb` に `Tb + self.lower_dt / 1800.0 * (env['Tamb_C'] - Tb) + float(pack['loss_int']) * self.lower_dt / 50000.0` の結果を代入する。
20. `v_seq.append(...)` を実行する。
21. `v_seq` を返す。

代表コード断片

```
def _lower_rollout(self, v0_ms: float, u_seq: np.ndarray, env: dict, z0: float, Tb0: float,
                  tau_drive: float, tau_regen: float):
    p = self.model.p
    v = float(v0_ms)
    z = float(z0)
    Tb = float(Tb0)
    v_seq = []
    for u in u_seq:
        u = float(u)
        if u >= 0.0:
            tau = u * tau_drive
        else:
            tau = u * tau_regen
        forces = self.model.resistive_forces(
            v, env['slope_pct'], env['headwind_ms'],
            ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0),
        )
        F_res = float(forces['F_total'])
        F_trac = self._traction_force(tau)
        a = (F_trac - F_res) / float(p.m)
        v = max(0.0, v + a * self.lower_dt)
        pack = self._pack_from_tau(v, tau, z, Tb, env)
        z = self.model.soc_step(
            z,
            float(pack['P_pack']),
            self.lower_dt,
            current_a=float(pack['I']),
            Tbat_C=Tb,
        )
        Tb = Tb + (self.lower_dt / 1800.0) * (env['Tamb_C'] - Tb) + (float(pack['loss_int']) * self.lower_dt) / 50000.0
        v_seq.append(float(v))
    return v_seq
```

L2072 関数 MPCNode._lower_mpc_solve

- 行範囲: L2072–L2271
- 所属: MPCNode
- 定義: `_lower_mpc_solve(self, v0_ms: float, s0_km: float, z0: float, Tb0: float, env: dict, v_ref_seq)`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `_lower_rollout`, `_pack_from_tau`, `_tau_limits`, `_traction_force`, `abs`, `all`, `append`, `array`, `asarray`, `bool`, `clip`, `concatenate`
- 戻り値の要点: `(u_seq, v_pred, mode) / (np.zeros(1), [v0_ms], 'eco') / J`
- 主なローカル代入名: `F_res`, `F_trac`, `I`, `J`, `N`, `P_pack`, `Tb`, `V`, `a`, `best_fallback`, `best_feasible`, `best_feasible_score`, `best_violation_score`, `bounds`, `candidate_controls`, `candidate_u`, `current`, `desired_accel`, `du`, `feasible`
- 読み取る主なインスタンス属性: `self._lower_rollout`, `self._lower_upper_busy_logged`, `self._pack_from_tau`, `self._tau_limits`, `self._traction_force`, `self._upper_solve_in_progress`, `self.get_logger`, `self.get_parameter`, `self.lower_N`, `self.lower_dt`, `self.lower_last_u`, `self.lower_max_iter`, `self.lower_maxfun`, `self.lower_skip_optimization_during_upper_solve`, `self.model`, `self.throttle_rate_limit`, `self.w_throttle`, `self.w_track`
- 更新する主なインスタンス属性: `self._lower_upper_busy_logged`, `self.last_lower_solve_ok`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 9、ループ 3、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `N` に `max(1, min(self.lower_N, len(v_ref_seq)))` の結果を代入する。
2. 条件 `N <= 0` を判定し、真なら内部処理を行う。
3. `(np.zeros(1), [v0_ms], 'eco')` を返す。
4. `v_ref_seq` に `v_ref_seq[:N]` の結果を代入する。
5. `(tau_drive, tau_regen)` に `self._tau_limits()` の結果を代入する。
6. `p` に `self.model.p` の結果を代入する。
7. `w_track` に `float(self.w_track)` の結果を代入する。
8. `w_throttle` に `float(self.w_throttle)` の結果を代入する。
9. `w_current` に `float(self.get_parameter('w_current').value)` の結果を代入する。
10. `w_T` に `float(self.get_parameter('w_T').value)` の結果を代入する。
11. `rate_lim` に `float(self.throttle_rate_limit) / 100.0 if self.throttle_rate_limit > 0.0 else 0.0` の結果を代入する。
12. `u0` に `float(np.clip(self.lower_last_u, -1.0, 1.0))` の結果を代入する。
13. `bounds` に `[(-1.0, 1.0)] * N` の結果を代入する。
14. `seed` に `[]` の結果を代入する。
15. `v_seed` に `float(v0_ms)` の結果を代入する。
16. `z_seed` に `float(z0)` の結果を代入する。
17. `tb_seed` に `float(Tb0)` の結果を代入する。
18. `u_prev_seed` に `u0` の結果を代入する。
19. `motor_count` に `int(p.motor_count) if int(p.motor_count) > 0 else int(p.driven_wheel_count or 1)` の結果を代入する。

20. motor_count に $\max(1, \text{motor_count})$ の結果を代入する。
21. torque_factor に $\max(0.001, \text{float}(\text{p.wheel_radius})) / \max(1\text{e-}06, \text{float}(\text{p.gear_ratio}) * \text{float}(\text{p.gear_eta}) * \text{motor_count})$ の結果を代入する。
22. max_du に $\text{rate_lim} * \max(\text{self.lower_dt}, 0.001)$ if $\text{rate_lim} > 0.0$ else 2.0 の結果を代入する。
23. v_ref_seq を順に走査し、各要素を v_ref に入れて処理する。
24. forces に $\text{self.model.resistive_forces}(\text{v_seed}, \text{env['slope_pct']}, \text{env['headwind_ms']}, \text{ambient_temp_c}=\text{env.get('Tamb_C')}, \text{elevation_m}=\text{env.get('elevation_m')}, 0.0))$ の結果を代入する。
25. desired_accel に $(\text{float}(\text{v_ref}) - \text{v_seed}) / \max(\text{self.lower_dt}, 0.001)$ の結果を代入する。
26. tau_request に $(\text{float}(\text{p.m}) * \text{desired_accel} + \text{float}(\text{forces['F_total']})) * \text{torque_factor}$ の結果を代入する。
27. tau_limit に tau_drive if $\text{tau_request} \geq 0.0$ else tau_regen の結果を代入する。
28. u_request に $\text{tau_request} / \max(\text{abs}(\text{float}(\text{tau_limit})), 1\text{e-}06)$ の結果を代入する。
29. u_request に $\text{float}(\text{np.clip}(\text{u_request}, \text{u_prev_seed} - \text{max_du}, \text{u_prev_seed} + \text{max_du}))$ の結果を代入する。
30. u_request に $\text{float}(\text{np.clip}(\text{u_request}, -1.0, 1.0))$ の結果を代入する。
31. u_min に $\max(-1.0, \text{u_prev_seed} - \text{max_du})$ の結果を代入する。
32. u_max に $\min(1.0, \text{u_prev_seed} + \text{max_du})$ の結果を代入する。
33. candidate_controls に $\text{np.unique}(\text{np.concatenate}((\text{np.linspace}(\text{u_min}, \text{u_max}, 25, \text{dtype}=\text{float}), \text{np.array}([\text{u_request}, \text{np.clip}(0.0, \text{u_min}, \text{u_max})], \text{dtype}=\text{float})]))$ の結果を代入する。
34. best_feasible に None の結果を代入する。
35. best_feasible_score に $\text{float}('inf')$ の結果を代入する。
36. best_fallback に $\text{float}(\text{np.clip}(\text{u_request}, \text{u_min}, \text{u_max}))$ の結果を代入する。
37. best_violation_score に $\text{float}('inf')$ の結果を代入する。
38. candidate_controls を順に走査し、各要素を candidate_u に入れて処理する。
39. candidate_u に $\text{float}(\text{np.clip}(\text{candidate_u}, \text{u_min}, \text{u_max}))$ の結果を代入する。
40. tau に $\text{candidate_u} * (\text{tau_drive} \text{ if } \text{candidate_u} \geq 0.0 \text{ else } \text{tau_regen})$ の結果を代入する。
41. F_trac に $\text{self.traction_force}(\text{tau})$ の結果を代入する。
42. v_trial に $\max(0.0, \text{v_seed} + (\text{F_trac} - \text{float}(\text{forces['F_total']})) / \text{float}(\text{p.m}) * \text{self.lower_dt})$ の結果を代入する。
43. pack に $\text{self.pack_from_tau}(\text{v_trial}, \text{tau}, \text{z_seed}, \text{tb_seed}, \text{env})$ の結果を代入する。
44. current に $\text{float}(\text{pack['I']})$ の結果を代入する。
45. voltage に $\text{float}(\text{pack['V']})$ の結果を代入する。
46. tracking_score に $(\text{v_trial} - \text{float}(\text{v_ref})) ** 2 + 0.01 * (\text{candidate_u} - \text{u_request}) ** 2$ の結果を代入する。
47. feasible に $\text{float}(\text{p.I_chg_min}) \leq \text{current} \leq \text{float}(\text{p.I_max})$ and $\text{float}(\text{p.V_min}) \leq \text{voltage} \leq \text{float}(\text{p.V_max})$ の結果を代入する。
48. 条件 $\text{feasible and tracking_score} < \text{best_feasible_score}$ を判定し、真なら内部処理を行う。
49. best_feasible に candidate_u の結果を代入する。
50. best_feasible_score に tracking_score の結果を代入する。

51. violation_score に $(\max(0.0, \text{current} - \text{float}(\text{p.I_max})) / \max(\text{abs}(\text{float}(\text{p.I_max})), 1.0)) ** 2 + (\max(0.0, \text{float}(\text{p.I_chg_min}) - \text{current}) / \max(\text{abs}(\text{float}(\text{p.I_chg_min})), 1.0)) ** 2 + (\max(0.0, \text{float}(\text{p.V_min}) - \text{voltage}) / \max(\text{abs}(\text{float}(\text{p.V_min})), 1.0)) ** 2 + (\max(0.0, \text{voltage} - \text{float}(\text{p.V_max})) / \max(\text{abs}(\text{float}(\text{p.V_max})), 1.0)) ** 2 + 0.001 * \text{tracking_score}$ の結果を代入する。
52. 条件 $\text{violation_score} < \text{best_violation_score}$ を判定し、真なら内部処理を行う。
53. best_violation_score に violation_score の結果を代入する。
54. best_fallback に candidate_u の結果を代入する。
55. u_request に $\text{float}(\text{best_feasible if best_feasible is not None else best_fallback})$ の結果を代入する。
56. seed.append(...) を実行する。
57. tau に $\text{u_request} * (\text{tau_drive if u_request} \geq 0.0 \text{ else tau_regen})$ の結果を代入する。
58. F_trac に self.traction_force(tau) の結果を代入する。
59. v_seed に $\max(0.0, v_seed + (\text{F_trac} - \text{float}(\text{forces['F_total']})) / \text{float}(\text{p.m}) * \text{self.lower_dt})$ の結果を代入する。
60. pack に self.pack_from_tau(v_seed, tau, z_seed, tb_seed, env) の結果を代入する。
61. z_seed に self.model.soc_step(z_seed, float(pack['P_pack']), self.lower_dt, current_a=float(pack['I']), Tbat_C=tb_seed) の結果を代入する。
62. tb_seed を Add で更新する。
63. u_prev_seed に u_request の結果を代入する。
64. x0 に np.asarray(seed, dtype=float) の結果を代入する。
65. 関数 cost を定義する。
66. skip_refine に $\text{bool}(\text{self.upper_solve_in_progress and self.lower_skip_optimization_during_upper_solve})$ の結果を代入する。
67. 条件 $\text{skip_refine or self.lower_max_iter} \leq 0$ を判定し、真なら内部処理を行う。
68. self.last_lower_solve_ok に True の結果を代入する。
69. u_seq に x0 の結果を代入する。
70. 条件 $\text{skip_refine and (not self.lower_upper_busy_logged)}$ を判定し、真なら内部処理を行う。
71. self.get_logger().info(...) を実行する。
72. self.lower_upper_busy_logged に True の結果を代入する。
73. 上の条件が偽の場合:
74. res に $\text{minimize}(\text{cost}, \text{x0}, \text{method}='L-BFGS-B', \text{bounds}=\text{bounds}, \text{options}=\text{dict}(\text{maxiter}=\text{self.lower_max_iter}, \text{maxfun}=\text{self.lower_maxfun}, \text{ftol}=1\text{e-}06))$ の結果を代入する。
75. 条件 $\text{np.all}(\text{np.isfinite}(\text{res.x}))$ を判定し、真なら内部処理を行う。
76. self.last_lower_solve_ok に $\text{bool}(\text{res.success})$ の結果を代入する。
77. u_seq に res.x の結果を代入する。
78. 条件 not res.success を判定し、真なら内部処理を行う。
79. self.get_logger().warn(...) を実行する。
80. 上の条件が偽の場合:

代表コード断片

```
def _lower_mpc_solve(self, v0_ms: float, s0_km: float, z0: float, Tb0: float, env: dict,
v_ref_seq):
    N = max(1, min(self.lower_N, len(v_ref_seq)))
    if N <= 0:
        return np.zeros(1), [v0_ms], 'eco'
    v_ref_seq = v_ref_seq[:N]
    tau_drive, tau_regen = self._tau_limits()
    p = self.model.p
    w_track = float(self.w_track)
    w_throttle = float(self.w_throttle)
    w_current = float(self.get_parameter('w_current').value)
    w_T = float(self.get_parameter('w_T').value)
    rate_lim = float(self.throttle_rate_limit) / 100.0 if self.throttle_rate_limit > 0.0
else 0.0
    u0 = float(np.clip(self.lower_last_u, -1.0, 1.0))
    bounds = [(-1.0, 1.0)] * N

    # Inverse dynamics gives a physically meaningful, bounded horizon seed in
    # deterministic time. It also remains the safe lower policy while the
    # hourly full-race optimizer is using the other executor thread.
    seed = []
    v_seed = float(v0_ms)
    z_seed = float(z0)
    tb_seed = float(Tb0)
    u_prev_seed = u0
    motor_count = int(p.motor_count) if int(p.motor_count) > 0 else int(p.
driven_wheel_count or 1)
    motor_count = max(1, motor_count)
    torque_factor = (
        max(1.0e-3, float(p.wheel_radius))
        / max(1.0e-6, float(p.gear_ratio) * float(p.gear_eta) * motor_count)
    )
    max_du = rate_lim * max(self.lower_dt, 1.0e-3) if rate_lim > 0.0 else 2.0
    for v_ref in v_ref_seq:
        forces = self.model.resistive_forces(
            v_seed, env['slope_pct'], env['headwind_ms'],
            ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0),
        )
    ...
```

L2185 関数 MPCNode._lower_mpc_solve.cost

- 行範囲: L2185–L2237
- 所属: MPCNode._lower_mpc_solve
- 定義: cost(u_vec)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _pack_from_tau, _traction_force, abs, float, get, max, range, resistive_forces, soc_step
- 戻り値の要点: J
- 主なローカル代入名: F_res, F_trac, I, J, P_pack, Tb, V, a, du, forces, i, loss_int, pack, tau, u, u_prev, v, v_ref, z
- 読み取る主なインスタンス属性: self._pack_from_tau, self._traction_force, self.lower_dt, self.model
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `v` に `float(v0_ms)` の結果を代入する。
2. `z` に `float(z0)` の結果を代入する。
3. `Tb` に `float(Tb0)` の結果を代入する。
4. `u_prev` に `u0` の結果を代入する。
5. `J` に `0.0` の結果を代入する。
6. `range(N)` を順に走査し、各要素を `i` に入れて処理する。
7. `u` に `float(u_vec[i])` の結果を代入する。
8. 条件 `u >= 0.0` を判定し、真なら内部処理を行う。
9. `tau` に `u * tau_drive` の結果を代入する。
10. 上の条件が偽の場合:
11. `tau` に `u * tau_regen` の結果を代入する。
12. `forces` に `self.model.resistive_forces(v, env['slope_pct'], env['headwind_ms'], ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m', 0.0))` の結果を代入する。
13. `F_res` に `float(forces['F_total'])` の結果を代入する。
14. `F_trac` に `self._traction_force(tau)` の結果を代入する。
15. `a` に `(F_trac - F_res) / float(p.m)` の結果を代入する。
16. `v` に `max(0.0, v + a * self.lower_dt)` の結果を代入する。
17. `pack` に `self._pack_from_tau(v, tau, z, Tb, env)` の結果を代入する。
18. `P_pack` に `float(pack['P_pack'])` の結果を代入する。
19. `I` に `float(pack['I'])` の結果を代入する。
20. `V` に `float(pack['V'])` の結果を代入する。
21. `loss_int` に `float(pack['loss_int'])` の結果を代入する。
22. `z` に `self.model.soc_step(z, P_pack, self.lower_dt, current_a=I, Tbat_C=Tb)` の結果を代入する。
23. `Tb` に `Tb + self.lower_dt / 1800.0 * (env['Tamb_C'] - Tb) + loss_int * self.lower_dt / 50000.0` の結果を代入する。
24. `v_ref` に `float(v_ref_seq[i])` の結果を代入する。
25. `J` を Add で更新する。
26. `J` を Add で更新する。

27. du に $(u - u_{\text{prev}}) / \max(\text{self.lower_dt}, 0.001)$ の結果を代入する。
28. 条件 $\text{rate_lim} > 0.0$ を判定し、真なら内部処理を行う。
29. J を Add で更新する。
30. J を Add で更新する。
31. J を Add で更新する。
32. J を Add で更新する。
33. J を Add で更新する。
34. J を Add で更新する。
35. J を Add で更新する。
36. J を Add で更新する。
37. J を Add で更新する。
38. J を Add で更新する。
39. u_{prev} に u の結果を代入する。
40. J を返す。

代表コード断片

```
def cost(u_vec):
    v = float(v0_ms)
    z = float(z0)
    Tb = float(Tb0)
    u_prev = u0
    J = 0.0
    for i in range(N):
        u = float(u_vec[i])
        if u >= 0.0:
            tau = u * tau_drive
        else:
            tau = u * tau_regen
        forces = self.model.resistive_forces(
            v, env['slope_pct'], env['headwind_ms'],
            ambient_temp_c=env.get('Tamb_C'), elevation_m=env.get('elevation_m',
0.0),
        )
        F_res = float(forces['F_total'])
        F_trac = self._traction_force(tau)
        a = (F_trac - F_res) / float(p.m)
        v = max(0.0, v + a * self.lower_dt)
        pack = self._pack_from_tau(v, tau, z, Tb, env)
        P_pack = float(pack['P_pack'])
        I = float(pack['I'])
        V = float(pack['V'])
        loss_int = float(pack['loss_int'])
        z = self.model.soc_step(
            z,
            P_pack,
            self.lower_dt,
            current_a=I,
            Tbat_C=Tb,
        )
        Tb = Tb + (self.lower_dt / 1800.0) * (env['Tamb_C'] - Tb) + (loss_int * self
.lower_dt) / 50000.0
```

```
v_ref = float(v_ref_seq[i])
```

```
...
```

L2273 関数 MPCNode._step_solar

- 行範囲: L2273–L2547
- 所属: MPCNode
- 定義: `_step_solar(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `Float32MultiArray`, `MheInput`, `MheMeas`, `_current_bin_index`, `_distance_plan_speed`, `_forecast_at_time`, `_horizon_data`, `_interp_upper_speed`, `_maybe_reload_forecast`, `_measured_distance_km`, `_mpc_solve_solar`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `P_pack`, `current_meas_fresh`, `d0`, `data`, `distance_meas_km`, `env_msg`, `env_stop`, `hard_stop`, `headwind_ms`, `inertial_power_w`, `k_now`, `limits`, `loss_int`, `meas`, `meas_now_sec`, `moved_to_new_bin`, `need_plan`, `out`, `plan_end_km`, `s_for_plan`
- 読み取る主なインスタンス属性: `self.Tb`, `self._current_bin_index`, `self._distance_plan_speed`, `self._forecast_at_time`, `self._horizon_data`, `self._interp_upper_speed`, `self._maybe_reload_forecast`, `self._measured_distance_km`, `self._mpc_solve`, `self._mpc_solve_solar_distance`, `self._publish_metrics`, `self._publish_plan_path`, `self._publish_summary`, `self._publish_upper_solve`, `self._route_value`, `self._sample_plan_segments`, `self._soc_guard_speed`, `self._sync_measured_state`, `self._update_live_control`, `self._upper_solve_logged`, `self.battery_meas_timeout_sec`, `self.control_stop_hold`, `self.current_meas_time`, `self.drive_schedule`
- 更新する主なインスタンス属性: `self.Tb`, `self._upper_solve_in_progress`, `self._upper_solve_logged`, `self.control_stop_hold`, `self.forecast_reloaded`, `self.k`, `self.last_bin`, `self.last_data`, `self.last_plan_time`, `self.model_prev_speed_ms`, `self.plan_dt_sec`, `self.plan_start_monotonic`, `self.s_km`, `self.upper_plan_id`, `self.upper_plan_mode`, `self.upper_plan_s_km`, `self.upper_plan_soc`, `self.upper_plan_stop_hold`, `self.upper_plan_tb_c`, `self.upper_plan_time`, `self.v_cmd`, `self.v_plan_kmh`, `self.v_plan_segments`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 52、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `self._maybe_reload_forecast(...)` を実行する。
2. `self._sync_measured_state(...)` を実行する。
3. `s_for_stop` に `float(self.s_km)` の結果を代入する。
4. 条件 `bool(self.get_parameter('use_measured_s').value)` and `np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
5. `s_for_stop` に `float(self.s_meas)` の結果を代入する。

6. `v_for_stop` に `float(self.v_upper_cmd)` の結果を代入する。
7. 条件 `bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now)` を判定し、真なら内部処理を行う。
8. `v_for_stop` に `float(self.v_now)` の結果を代入する。
9. `self.control_stop_hold` に `self.update_live_control_stop_hold(s_for_stop, v_for_stop)` の結果を代入する。
10. `k_now` に `self.current_bin_index()` の結果を代入する。
11. `moved_to_new_bin` に `self.last_bin is None or k_now != self.last_bin` の結果を代入する。
12. `self.k` に `k_now` の結果を代入する。
13. `data` に `self.horizon_data(self.k)` の結果を代入する。
14. 条件 `self.control_stop_hold and data` を判定し、真なら内部処理を行う。
15. `env_stop` に `self.forecast_at_time(data[0].get('t_utc', datetime.now(timezone.utc)), s_for_stop, drive=False, control_stop=True)` の結果を代入する。
16. `env_stop['slope_pct']` に `self.route_value(s_for_stop, 'slope_pct', 0.0)` の結果を代入する。
17. `env_stop['t_utc']` に `data[0].get('t_utc', datetime.now(timezone.utc))` の結果を代入する。
18. `data[0]` に `env_stop` の結果を代入する。
19. `self.last_data` に `data` の結果を代入する。
20. 条件 `len(data) > 0` を判定し、真なら内部処理を行う。
21. `d0` に `data[0]` の結果を代入する。
22. `s_for_profile` に `self.s_km` の結果を代入する。
23. 条件 `bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
24. `s_for_profile` に `float(self.s_meas)` の結果を代入する。
25. `slope_pct` に `d0.get('slope_pct', 0.0)` の結果を代入する。
26. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
27. `slope_pct` に `self.route_value(s_for_profile, 'slope_pct', slope_pct)` の結果を代入する。
28. `headwind_ms` に `d0.get('headwind_ms', 0.0)` の結果を代入する。
29. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
30. `headwind_ms` に `self.route_value(s_for_profile, 'headwind_ms', headwind_ms)` の結果を代入する。
31. `env_msg` に `Float32MultiArray()` の結果を代入する。
32. `env_msg.data` に `[float(d0.get('G_poa', 0.0)), float(d0.get('Tcell_C', 40.0)), float(d0.get('Tamb_C', 30.0)), float(slope_pct), float(headwind_ms)]` の結果を代入する。
33. `self.pub_env.publish(...)` を実行する。
34. `v_exec_kmh` に `self.v_upper_cmd` の結果を代入する。
35. 条件 `self.hierarchical and self.timer_lower is not None` を判定し、真なら内部処理を行う。
36. `v_exec_kmh` に `self.v_lower_cmd` の結果を代入する。

37. 条件 `bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now)` を判定し、真なら内部処理を行う。
38. `v_exec_kmh` に `float(self.v_now)` の結果を代入する。
39. `self.publish_metrics(...)` を実行する。
40. `self.publish_summary(...)` を実行する。
41. `need_plan` に `self.v_plan_kmh` is None or (`self.forecast_reloaded` and `self.replan_on_forecast_reload`) の結果を代入する。
42. 条件 `self.upper_mode == 'distance' and self.v_plan_segments` is None を判定し、真なら内部処理を行う。
43. `need_plan` に True の結果を代入する。
44. 条件 `self.upper_mode == 'distance' and self.v_plan_segments` を判定し、真なら内部処理を行う。
45. `plan_end_km` に `float(self.v_plan_segments[-1].get('s_end_km', self.s_km))` の結果を代入する。
46. 条件 `self.s_km >= plan_end_km - 1e-06` を判定し、真なら内部処理を行う。
47. `need_plan` に True の結果を代入する。
48. 条件 `self.upper_replan_km > 0.0 and self.upper_plan_s_km` is not None を判定し、真なら内部処理を行う。
49. 条件 `self.s_km - self.upper_plan_s_km >= self.upper_replan_km` を判定し、真なら内部処理を行う。
50. `need_plan` に True の結果を代入する。
51. 条件 `self.upper_replan_sec > 0.0 and self.upper_plan_time` is not None を判定し、真なら内部処理を行う。
52. 条件 `data` and `data[0].get('t_utc')` and `((data[0]['t_utc'] - self.upper_plan_time).total_seconds() >= self.upper_replan_sec)` を判定し、真なら内部処理を行う。
53. `need_plan` に True の結果を代入する。
54. 条件 `self.upper_replan_soc_delta > 0.0 and self.upper_plan_soc` is not None を判定し、真なら内部処理を行う。
55. 条件 `abs(float(self.z) - float(self.upper_plan_soc)) >= self.upper_replan_soc_delta` を判定し、真なら内部処理を行う。
56. `need_plan` に True の結果を代入する。
57. 条件 `self.upper_replan_tb_delta_c > 0.0 and self.upper_plan_tb_c` is not None を判定し、真なら内部処理を行う。
58. 条件 `abs(float(self.Tb) - float(self.upper_plan_tb_c)) >= self.upper_replan_tb_delta_c` を判定し、真なら内部処理を行う。
59. `need_plan` に True の結果を代入する。
60. 条件 `self.upper_replan_on_stop_transition` and `self.upper_plan_stop_hold` is not None and `bool(self.upper_plan_stop_hold and (not bool(self.control_stop_hold)))` を判定し、真なら内部処理を行う。
61. `need_plan` に True の結果を代入する。
62. `self.upper_plan_stop_hold` に `bool(self.control_stop_hold)` の結果を代入する。
63. 条件 `self.upper_replan_km <= 0.0 and self.upper_replan_sec <= 0.0` を判定し、真なら内部処理を行う。
64. `need_plan` に `need_plan` or `moved_to_new_bin` の結果を代入する。

65. 条件 `need_plan and self.upper_mode == 'distance' and self.v_plan_segments and (self.drive_schedule is not None) and (len(data) > 0) and data[0].get('t_utc') and (not self.drive_schedule.is_drive_time(data[0]['t_utc']))` を判定し、真なら内部処理を行う。
66. `plan_end_km` に `float(self.v_plan_segments[-1].get('s_end_km', self.s_km))` の結果を代入する。
67. 条件 `self.s_km < plan_end_km - 1e-06` を判定し、真なら内部処理を行う。
68. `need_plan` に `False` の結果を代入する。
69. 条件 `need_plan and len(data) > 0` を判定し、真なら内部処理を行う。
70. `s_for_profile` に `self.s_km` の結果を代入する。
71. 条件 `bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
72. `s_for_profile` に `float(self.s_meas)` の結果を代入する。
73. `t0` に `data[0].get('t_utc', datetime.now(timezone.utc))` の結果を代入する。
74. `upper_started` に `time.monotonic()` の結果を代入する。
75. `self._upper_solve_in_progress` に `True` の結果を代入する。
76. 例外処理を伴う `try` ブロックを実行する。
77. 条件 `self.upper_mode == 'distance'` を判定し、真なら内部処理を行う。
78. `(self.v_upper_cmd, self.v_plan_segments, self.upper_plan_seq)` に `self._mpc_solve_solar_distance(t0, s_for_profile, self.upper_plan_seq)` の結果を代入する。
79. `self.upper_plan_mode` に `'distance'` の結果を代入する。
80. `self.plan_dt_sec` に `float(self.model.p.dt)` の結果を代入する。

代表コード断片

```
def _step_solar(self):
    self._maybe_reload_forecast()
    self._sync_measured_state()
    s_for_stop = float(self.s_km)
    if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
        s_for_stop = float(self.s_meas)
    v_for_stop = float(self.v_upper_cmd)
    if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
        v_for_stop = float(self.v_now)
    self.control_stop_hold = self._update_live_control_stop_hold(s_for_stop, v_for_stop)
    k_now = self._current_bin_index()
    moved_to_new_bin = (self.last_bin is None) or (k_now != self.last_bin)
    self.k = k_now

    data = self._horizon_data(self.k)
    if self.control_stop_hold and data:
        env_stop = self._forecast_at_time(
            data[0].get('t_utc', datetime.now(timezone.utc)),
            s_for_stop,
            drive=False,
            control_stop=True,
        )
        env_stop['slope_pct'] = self._route_value(s_for_stop, 'slope_pct', 0.0)
        env_stop['t_utc'] = data[0].get('t_utc', datetime.now(timezone.utc))
        data[0] = env_stop
    self.last_data = data
    if len(data) > 0:
```

```

        d0 = data[0]
        s_for_profile = self.s_km
        if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas)
:
            s_for_profile = float(self.s_meas)
            slope_pct = d0.get('slope_pct', 0.0)
            if self.route_profile is not None:
                slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
            headwind_ms = d0.get('headwind_ms', 0.0)
...

```

L2549 関数 MPCNode._step_lower

- 行範囲: L2549–L2598
- 所属: MPCNode
- 定義: `_step_lower(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_build_lower_ref`, `_lower_mpc_solve`, `_publish_lower_command_cycle`, `_route_value`, `bool`, `dict`, `float`, `get`, `get_logger`, `get_parameter`, `info`, `isfinite`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `base_time`, `d0`, `env`, `headwind_ms`, `mode`, `s_for_profile`, `slope_pct`, `solve_started`, `u_seq`, `v0_ms`, `v_cmd_ms`, `v_pred`, `v_ref_seq`
- 読み取る主なインスタンス属性: `self.Tb`, `self._build_lower_ref`, `self._lower_mpc_solve`, `self._lower_solve_logged`, `self._publish_lower_command_cycle`, `self._route_value`, `self.get_logger`, `self.get_parameter`, `self.hierarchical`, `self.last_data`, `self.route_profile`, `self.s_km`, `self.s_meas`, `self.timer_lower`, `self.v_lower_cmd`, `self.v_now`, `self.v_upper_cmd`, `self.z`
- 更新する主なインスタンス属性: `self._lower_solve_logged`, `self.lower_last_mode`, `self.lower_last_u`, `self.lower_last_v`, `self.v_cmd`, `self.v_lower_cmd`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 10、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `not self.hierarchical or self.timer_lower is None` を判定し、真なら内部処理を行う。
2. を返す。
3. `self._publish_lower_command_cycle(...)` を実行する。
4. 条件 `not self.last_data` を判定し、真なら内部処理を行う。
5. を返す。
6. `d0` に `self.last_data[0]` の結果を代入する。
7. `s_for_profile` に `self.s_km` の結果を代入する。

8. 条件 `bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas)` を判定し、真なら内部処理を行う。
9. `s_for_profile` に `float(self.s_meas)` の結果を代入する。
10. `slope_pct` に `d0.get('slope_pct', 0.0)` の結果を代入する。
11. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
12. `slope_pct` に `self._route_value(s_for_profile, 'slope_pct', slope_pct)` の結果を代入する。
13. `headwind_ms` に `d0.get('headwind_ms', 0.0)` の結果を代入する。
14. 条件 `self.route_profile is not None` を判定し、真なら内部処理を行う。
15. `headwind_ms` に `self._route_value(s_for_profile, 'headwind_ms', headwind_ms)` の結果を代入する。
16. `env` に `dict(slope_pct=float(slope_pct), headwind_ms=float(headwind_ms), G_poa=float(d0.get('G_poa', 0.0)), Tcell_C=float(d0.get('Tcell_C', 40.0)), Tamb_C=float(d0.get('Tamb_C', 30.0)))` の結果を代入する。
17. `base_time` に `d0.get('t_utc', None)` の結果を代入する。
18. 条件 `base_time is None` を判定し、真なら内部処理を行う。
19. `base_time` に `datetime.now(timezone.utc)` の結果を代入する。
20. `v_ref_seq` に `self._build_lower_ref(base_time, s_for_profile, d0)` の結果を代入する。
21. 条件 `bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now)` を判定し、真なら内部処理を行う。
22. `v0_ms` に `float(self.v_now) / 3.6` の結果を代入する。
23. 上の条件が偽の場合:
24. `v0_ms` に `float(self.v_lower_cmd) / 3.6` の結果を代入する。
25. 条件 `not np.isfinite(v0_ms) or v0_ms <= 0.0` を判定し、真なら内部処理を行う。
26. `v0_ms` に `float(self.v_upper_cmd) / 3.6` の結果を代入する。
27. `solve_started` に `time.monotonic()` の結果を代入する。
28. `(u_seq, v_pred, mode)` に `self._lower_mpc_solve(v0_ms, s_for_profile, self.z, self.Tb, env, v_ref_seq)` の結果を代入する。
29. 条件 `not self._lower_solve_logged` を判定し、真なら内部処理を行う。
30. `self.get_logger().info(...)` を実行する。
31. `self._lower_solve_logged` に `True` の結果を代入する。
32. `v_cmd_ms` に `v_pred[0] if v_pred else v0_ms` の結果を代入する。
33. `self.v_lower_cmd` に `float(v_cmd_ms * 3.6)` の結果を代入する。
34. `self.v_cmd` に `float(self.v_lower_cmd)` の結果を代入する。
35. 条件 `len(u_seq) > 0` を判定し、真なら内部処理を行う。
36. `self.lower_last_u` に `float(u_seq[0])` の結果を代入する。
37. `self.lower_last_mode` に `str(mode)` の結果を代入する。
38. `self.lower_last_v_pred` に `list(v_pred) if v_pred else [self.v_lower_cmd / 3.6]` の結果を代入する。
39. `self._publish_lower_command_cycle(...)` を実行する。

代表コード断片

```
def _step_lower(self):
    if not self.hierarchical or self.timer_lower is None:
        return
    self._publish_lower_command_cycle()
    if not self.last_data:
        return
    d0 = self.last_data[0]
    s_for_profile = self.s_km
    if bool(self.get_parameter('use_measured_s').value) and np.isfinite(self.s_meas):
        s_for_profile = float(self.s_meas)
    slope_pct = d0.get('slope_pct', 0.0)
    if self.route_profile is not None:
        slope_pct = self._route_value(s_for_profile, 'slope_pct', slope_pct)
    headwind_ms = d0.get('headwind_ms', 0.0)
    if self.route_profile is not None:
        headwind_ms = self._route_value(s_for_profile, 'headwind_ms', headwind_ms)
    env = dict(
        slope_pct=float(slope_pct),
        headwind_ms=float(headwind_ms),
        G_poa=float(d0.get('G_poa', 0.0)),
        Tcell_C=float(d0.get('Tcell_C', 40.0)),
        Tamb_C=float(d0.get('Tamb_C', 30.0)),
    )
    base_time = d0.get('t_utc', None)
    if base_time is None:
        base_time = datetime.now(timezone.utc)
    v_ref_seq = self._build_lower_ref(base_time, s_for_profile, d0)

    if bool(self.get_parameter('use_measured_speed').value) and np.isfinite(self.v_now):
        v0_ms = float(self.v_now) / 3.6
    else:
        v0_ms = float(self.v_lower_cmd) / 3.6
        if not np.isfinite(v0_ms) or v0_ms <= 0.0:
            v0_ms = float(self.v_upper_cmd) / 3.6

...

```

L2600 関数 MPCNode._publish_lower_command_cycle

- 行範囲: L2600–L2615
- 所属: MPCNode
- 定義: `_publish_lower_command_cycle(self)`->None
- docstring: Non-blocking 1 Hz output path, independent of both optimizers.
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `String`, `_publish_lower_plan`, `_publish_summary`, `_sync_measured_state`, `clip`, `float`, `get_parameter`, `min`, `publish`, `str`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `speed_kmh`, `throttle_pct`
- 読み取る主なインスタンス属性: `self._publish_lower_plan`, `self._publish_summary`, `self._sync_measured_state`, `self.control_stop_hold`, `self.get_parameter`, `self.hierarchical`, `self.lower_last_mode`, `self.lower_last_u`, `self.lower_last_v`, `self.pub_drive_mode`, `self.pub_speed`, `self.pub_throttle`, `self.timer_lower`, `self.v_lower_cmd`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。

- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 not self.hierarchical or self.timer_lower is None を判定し、真なら内部処理を行う。
2. を返す。
3. self._sync_measured_state(...) を実行する。
4. speed_kmh に float(np.clip(self.v_lower_cmd, 0.0, self.get_parameter('v_max_kmh').value)) の結果を代入する。
5. 条件 self.control_stop_hold を判定し、真なら内部処理を行う。
6. speed_kmh に 0.0 の結果を代入する。
7. self.pub_speed.publish(...) を実行する。
8. throttle_pct に float(np.clip(self.lower_last_u * 100.0, -100.0, 100.0)) の結果を代入する。
9. 条件 self.control_stop_hold を判定し、真なら内部処理を行う。
10. throttle_pct に min(0.0, throttle_pct) の結果を代入する。
11. self.pub_throttle.publish(...) を実行する。
12. self.pub_drive_mode.publish(...) を実行する。
13. self._publish_lower_plan(...) を実行する。
14. self._publish_summary(...) を実行する。

代表コード断片

```
def _publish_lower_command_cycle(self) -> None:
    """Non-blocking 1 Hz output path, independent of both optimizers."""
    if not self.hierarchical or self.timer_lower is None:
        return
    self._sync_measured_state()
    speed_kmh = float(np.clip(self.v_lower_cmd, 0.0, self.get_parameter('v_max_kmh').
value))
    if self.control_stop_hold:
        speed_kmh = 0.0
    self.pub_speed.publish(Float32(data=speed_kmh))
    throttle_pct = float(np.clip(self.lower_last_u * 100.0, -100.0, 100.0))
    if self.control_stop_hold:
        throttle_pct = min(0.0, throttle_pct)
    self.pub_throttle.publish(Float32(data=throttle_pct))
    self.pub_drive_mode.publish(String(data=str(self.lower_last_mode)))
    self._publish_lower_plan(self.lower_last_v_pred)
    self._publish_summary()
```

L2618 関数 MPCNode._init_passo

- 行範囲: L2618–L2707
- 所属: MPCNode
- 定義: `_init_passo(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_load_stops`, `array`, `bool`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `deque`, `float`, `get_logger`, `get_parameter`, `info`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `stop_yaml`
- 読み取る主なインスタンス属性: `self._load_stops`, `self._on_config`, `self._on_config_ready`, `self._on_fuel`, `self._on_grade`, `self._on_idle_fuel`, `self._on_obd_ok`, `self._on_s_km`, `self._on_speed`, `self._on_system_state`, `self._on_thrott`, `self._step_passo`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.get_logger`, `self.get_parameter`
- 更新する主なインスタンス属性: `self.Np`, `self.config_ready`, `self.dt`, `self.fuel_rate_lph`, `self.grade`, `self.id_ema_alpha`, `self.id_min_samples`, `self.id_r2`, `self.id_rmse`, `self.id_samples`, `self.id_window_sec`, `self.idle_fuel_lph`, `self.last_step_time`, `self.max_acc_dt_sec`, `self.model_coeffs`, `self.mpc_state`, `self.obd_ok`, `self.online_id_enabled`, `self.prev_speed_kmh`, `self.prev_speed_time`, `self.pub_mpc_state`, `self.pub_path`, `self.pub_speed`, `self.pub_status`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.declare_parameter(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。
13. `self.declare_parameter(...)` を実行する。

14. self.declare_parameter(...) を実行する。
15. self.declare_parameter(...) を実行する。
16. self.declare_parameter(...) を実行する。
17. self.declare_parameter(...) を実行する。
18. self.declare_parameter(...) を実行する。
19. self.declare_parameter(...) を実行する。
20. self.declare_parameter(...) を実行する。
21. self.declare_parameter(...) を実行する。
22. self.declare_parameter(...) を実行する。
23. self.declare_parameter(...) を実行する。
24. self.dt に float(self.get_parameter('passo_dt').value) の結果を代入する。
25. self.Np に int(self.get_parameter('passo_horizon_steps').value) の結果を代入する。
26. self.v_cmd に float(self.get_parameter('v_ref_kmh').value) の結果を代入する。
27. stop_yaml に self.get_parameter('stop_yaml').value の結果を代入する。
28. self._load_stops(...) を実行する。
29. self.v_now に math.nan の結果を代入する。
30. self.s_km に 0.0 の結果を代入する。
31. self.fuel_rate_lph に math.nan の結果を代入する。
32. self.throttle_pct に math.nan の結果を代入する。
33. self.obd_ok に 0.0 の結果を代入する。
34. self.config_ready に False の結果を代入する。
35. self.mpc_state に 'IDLE' の結果を代入する。
36. self.system_state に " の結果を代入する。
37. self.grade に math.nan の結果を代入する。
38. self.idle_fuel_lph に math.nan の結果を代入する。
39. self.model_coeffs に np.array([float(self.get_parameter('model_a0').value), float(self.get_parameter('model_a1').value), float(self.get_parameter('model_a2').value), float(self.get_parameter('model_a3').value), float(self.get_parameter('model_a4').value)], dtype=float) の結果を代入する。
40. self.online_id_enabled に bool(self.get_parameter('online_id_enabled').value) の結果を代入する。
41. self.id_window_sec に float(self.get_parameter('id_window_sec').value) の結果を代入する。
42. self.id_min_samples に int(self.get_parameter('id_min_samples').value) の結果を代入する。
43. self.id_ema_alpha に float(self.get_parameter('id_ema_alpha').value) の結果を代入する。
44. self.max_acc_dt_sec に float(self.get_parameter('max_acc_dt_sec').value) の結果を代入する。
45. self.id_samples に deque() の結果を代入する。
46. self.id_rmse に math.nan の結果を代入する。

47. self.id_r2 に math.nan の結果を代入する。
48. self.prev_speed_kmh に math.nan の結果を代入する。
49. self.prev_speed_time に None の結果を代入する。
50. self.valid_obd_sec に 0.0 の結果を代入する。
51. self.valid_speed_sec に 0.0 の結果を代入する。
52. self.valid_fuel_sec に 0.0 の結果を代入する。
53. self.last_step_time に None の結果を代入する。
54. self.create_subscription(...) を実行する。
55. self.create_subscription(...) を実行する。
56. self.create_subscription(...) を実行する。
57. self.create_subscription(...) を実行する。
58. self.create_subscription(...) を実行する。
59. self.create_subscription(...) を実行する。
60. self.create_subscription(...) を実行する。
61. self.create_subscription(...) を実行する。
62. self.create_subscription(...) を実行する。
63. self.create_subscription(...) を実行する。
64. self.pub_speed に self.create_publisher(Float32, '/planner/speed_cmd', 10) の結果を代入する。
65. self.pub_path に self.create_publisher(Path, '/planner/trajectory', 10) の結果を代入する。
66. self.pub_status に self.create_publisher(Float32MultiArray, '/planner/status', 10) の結果を代入する。
67. self.pub_mpc_state に self.create_publisher(String, '/system/mpc_state', 10) の結果を代入する。
68. self.timer に self.create_timer(1.0, self._step_passo) の結果を代入する。
69. self.get_logger().info(...) を実行する。

代表コード断片

```
def _init_passo(self):
    self.declare_parameter('stop_yaml', 'inputs/stop_points.yaml')
    self.declare_parameter('passo_dt', 1.0)
    self.declare_parameter('passo_horizon_steps', 10)
    self.declare_parameter('v_min_kmh', 0.0)
    self.declare_parameter('v_max_kmh', 110.0)
    self.declare_parameter('v_ref_kmh', 40.0)
    self.declare_parameter('w_fuel', 1.0)
    self.declare_parameter('w_speed', 0.3)
    self.declare_parameter('w_dv', 0.2)
    self.declare_parameter('w_dv_limit', 2.0)
    self.declare_parameter('dv_max_kmhps', 4.0)
    self.declare_parameter('w_stop', 1.0e4)
    self.declare_parameter('model_a0', 0.4)
    self.declare_parameter('model_a1', 0.02)
    self.declare_parameter('model_a2', 0.001)
    self.declare_parameter('model_a3', 0.08)
    self.declare_parameter('model_a4', 0.02)
```

```

self.declare_parameter('online_id_enabled', True)
self.declare_parameter('id_window_sec', 60.0)
self.declare_parameter('id_min_samples', 30)
self.declare_parameter('id_ema_alpha', 0.2)
self.declare_parameter('max_acc_dt_sec', 2.0)
self.declare_parameter('run_ready_sec', 3.0)

self.dt = float(self.get_parameter('passo_dt').value)
self.Np = int(self.get_parameter('passo_horizon_steps').value)
self.v_cmd = float(self.get_parameter('v_ref_kmh').value)

stop_yaml = self.get_parameter('stop_yaml').value
self._load_stops(stop_yaml)

# Inputs
self.v_now = math.nan
self.s_km = 0.0
...

```

L2709 関数 MPCNode._on_s_km

- 行範囲: L2709–L2710
- 所属: MPCNode
- 定義: `_on_s_km(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.s_km`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.s_km` に `float(msg.data)` の結果を代入する。

代表コード断片

```

def _on_s_km(self, msg: Float32):
    self.s_km = float(msg.data)

```

L2712 関数 MPCNode._on_speed

- 行範囲: L2712–L2713
- 所属: MPCNode
- 定義: `_on_speed(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.v_now`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.v_now` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_speed(self, msg: Float32):  
    self.v_now = float(msg.data)
```

L2715 関数 MPCNode._on_fuel

- 行範囲: L2715–L2716
- 所属: MPCNode
- 定義: `_on_fuel(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.fuel_rate_lph`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.fuel_rate_lph` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_fuel(self, msg: Float32):  
    self.fuel_rate_lph = float(msg.data)
```

L2718 関数 `MPCNode._on_throttle`

- 行範囲: L2718–L2719
- 所属: `MPCNode`
- 定義: `_on_throttle(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.throttle_pct`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.throttle_pct` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_throttle(self, msg: Float32):  
    self.throttle_pct = float(msg.data)
```

L2721 関数 `MPCNode._on_obd_ok`

- 行範囲: L2721–L2722
- 所属: `MPCNode`
- 定義: `_on_obd_ok(self, msg: Float32)`
- `docstring`: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.obd_ok
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self.obd_ok に float(msg.data) の結果を代入する。

代表コード断片

```
def _on_obd_ok(self, msg: Float32):
    self.obd_ok = float(msg.data)
```

L2724 関数 MPCNode._on_grade

- 行範囲: L2724–L2725
- 所属: MPCNode
- 定義: _on_grade(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.grade
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self.grade に float(msg.data) の結果を代入する。

代表コード断片

```
def _on_grade(self, msg: Float32):  
    self.grade = float(msg.data)
```

L2727 関数 MPCNode._on_idle_fuel

- 行範囲: L2727–L2730
- 所属: MPCNode
- 定義: `_on_idle_fuel(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.idle_fuel_lph`, `self.model_coeffs`
- 更新する主なインスタンス属性: `self.idle_fuel_lph`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.idle_fuel_lph` に `float(msg.data)` の結果を代入する。
2. 条件 `np.isfinite(self.idle_fuel_lph)` を判定し、真なら内部処理を行う。
3. `self.model_coeffs[0]` に `float(self.idle_fuel_lph)` の結果を代入する。

代表コード断片

```
def _on_idle_fuel(self, msg: Float32):  
    self.idle_fuel_lph = float(msg.data)  
    if np.isfinite(self.idle_fuel_lph):  
        self.model_coeffs[0] = float(self.idle_fuel_lph)
```

L2732 関数 MPCNode._on_config_ready

- 行範囲: L2732–L2733
- 所属: MPCNode
- 定義: `_on_config_ready(self, msg: Bool)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `bool`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。

- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.config_ready`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.config_ready` に `bool(msg.data)` の結果を代入する。

代表コード断片

```
def _on_config_ready(self, msg: Bool):
    self.config_ready = bool(msg.data)
```

L2735 関数 `MPCNode._on_system_state`

- 行範囲: L2735–L2736
- 所属: `MPCNode`
- 定義: `_on_system_state(self, msg: String)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `str`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.system_state`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.system_state` に `str(msg.data)` の結果を代入する。

代表コード断片

```
def _on_system_state(self, msg: String):
    self.system_state = str(msg.data)
```

L2738 関数 MPCNode._on_config

- 行範囲: L2738–L2743
- 所属: MPCNode
- 定義: `_on_config(self, msg:String)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_apply_config`, `safe_load`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `cfg`
- 読み取る主なインスタンス属性: `self._apply_config`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `cfg` に `yaml.safe_load(msg.data)` or `{}` の結果を代入する。
3. `Exception` を捕捉した場合:
4. を返す。
5. `self._apply_config(...)` を実行する。

代表コード断片

```
def _on_config(self, msg: String):
    try:
        cfg = yaml.safe_load(msg.data) or {}
    except Exception:
        return
    self._apply_config(cfg)
```

L2745 関数 MPCNode._apply_config

- 行範囲: L2745–L2776
- 所属: MPCNode
- 定義: `_apply_config(self, cfg:dict)`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `Parameter`, `_load_stops`, `append`, `bool`, `float`, `int`, `set_parameters`, `str`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `key`, `params`
- 読み取る主なインスタンス属性: `self._load_stops`, `self.model_coeffs`, `self.set_parameters`
- 更新する主なインスタンス属性: `self.Np`, `self.dt`, `self.online_id_enabled`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 14、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `params` に `[]` の結果を代入する。
- `_max_kmhps` を順に走査し、各要素を `key` に入れて処理する。
2. 条件 `key in cfg` を判定し、真なら内部処理を行う。
3. `params.append(...)` を実行する。
4. 条件 `'dv_max_kmh_per_s' in cfg and 'dv_max_kmhps' not in cfg` を判定し、真なら内部処理を行う。
5. `params.append(...)` を実行する。
6. 条件 `'horizon_steps' in cfg` を判定し、真なら内部処理を行う。
7. `params.append(...)` を実行する。
8. 条件 `'dt_control' in cfg` を判定し、真なら内部処理を行う。
9. `params.append(...)` を実行する。
10. 条件 `params` を判定し、真なら内部処理を行う。
11. `self.set_parameters(...)` を実行する。
12. 条件 `'horizon_steps' in cfg` を判定し、真なら内部処理を行う。
13. `self.Np` に `int(cfg['horizon_steps'])` の結果を代入する。
14. 条件 `'dt_control' in cfg` を判定し、真なら内部処理を行う。
15. `self.dt` に `float(cfg['dt_control'])` の結果を代入する。
16. 条件 `'model_a0' in cfg` を判定し、真なら内部処理を行う。
17. `self.model_coeffs[0]` に `float(cfg['model_a0'])` の結果を代入する。
18. 条件 `'model_a1' in cfg` を判定し、真なら内部処理を行う。
19. `self.model_coeffs[1]` に `float(cfg['model_a1'])` の結果を代入する。
20. 条件 `'model_a2' in cfg` を判定し、真なら内部処理を行う。
21. `self.model_coeffs[2]` に `float(cfg['model_a2'])` の結果を代入する。

22. 条件'model_a3' in cfg を判定し、真なら内部処理を行う。
23. self.model_coeffs[3] に float(cfg['model_a3']) の結果を代入する。
24. 条件'model_a4' in cfg を判定し、真なら内部処理を行う。
25. self.model_coeffs[4] に float(cfg['model_a4']) の結果を代入する。
26. 条件'online_id_enabled' in cfg を判定し、真なら内部処理を行う。
27. self.online_id_enabled に bool(cfg['online_id_enabled']) の結果を代入する。
28. 条件'stop_points_yaml' in cfg を判定し、真なら内部処理を行う。
29. self._load_stops(...) を実行する。

代表コード断片

```
def _apply_config(self, cfg: dict):
    params = []
    for key in ['v_min_kmh', 'v_max_kmh', 'v_ref_kmh', 'w_fuel', 'w_speed', 'w_dv', 'w_stop', 'dv_max_kmhps']:
        if key in cfg:
            params.append(Parameter(key, value=cfg[key]))
    if 'dv_max_kmh_per_s' in cfg and 'dv_max_kmhps' not in cfg:
        params.append(Parameter('dv_max_kmhps', value=cfg['dv_max_kmh_per_s']))
    if 'horizon_steps' in cfg:
        params.append(Parameter('passo_horizon_steps', value=cfg['horizon_steps']))
    if 'dt_control' in cfg:
        params.append(Parameter('passo_dt', value=cfg['dt_control']))
    if params:
        self.set_parameters(params)
    if 'horizon_steps' in cfg:
        self.Np = int(cfg['horizon_steps'])
    if 'dt_control' in cfg:
        self.dt = float(cfg['dt_control'])
    if 'model_a0' in cfg:
        self.model_coeffs[0] = float(cfg['model_a0'])
    if 'model_a1' in cfg:
        self.model_coeffs[1] = float(cfg['model_a1'])
    if 'model_a2' in cfg:
        self.model_coeffs[2] = float(cfg['model_a2'])
    if 'model_a3' in cfg:
        self.model_coeffs[3] = float(cfg['model_a3'])
    if 'model_a4' in cfg:
        self.model_coeffs[4] = float(cfg['model_a4'])
    if 'online_id_enabled' in cfg:
        self.online_id_enabled = bool(cfg['online_id_enabled'])
    if 'stop_points_yaml' in cfg:
        self._load_stops(str(cfg['stop_points_yaml']))
```

L2778 関数 MPCNode._stop_penalty_passo

- 行範囲: L2778–L2790
- 所属: MPCNode
- 定義: _stop_penalty_passo(self, s_km:float, v_kmh:float)->float
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: abs, float, get, get_parameter, max

- 戻り値の要点: pen
- 主なローカル代入名: dwell_s, pen, s_stop, st, v_ms, vmax_kmh, vmax_ms, w_stop, width_km
- 読み取る主なインスタンス属性: self.get_parameter, self.stops
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. v_ms に v_kmh / 3.6 の結果を代入する。
2. vmax_kmh に float(self.get_parameter('v_max_kmh').value) の結果を代入する。
3. vmax_ms に vmax_kmh / 3.6 の結果を代入する。
4. pen に 0.0 の結果を代入する。
5. w_stop に float(self.get_parameter('w_stop').value) の結果を代入する。
6. self.stops を順に走査し、各要素を st に入れて処理する。
7. s_stop に float(st.get('s_km', 0.0)) の結果を代入する。
8. dwell_s に float(st.get('dwell_s', 0.0)) の結果を代入する。
9. width_km に max(0.05, dwell_s * vmax_ms / 1000.0 * 0.5) の結果を代入する。
10. 条件 abs(s_km - s_stop) <= width_km を判定し、真なら内部処理を行う。
11. pen を Add で更新する。
12. pen を返す。

代表コード断片

```
def _stop_penalty_passo(self, s_km: float, v_kmh: float) -> float:
    v_ms = v_kmh / 3.6
    vmax_kmh = float(self.get_parameter('v_max_kmh').value)
    vmax_ms = vmax_kmh / 3.6
    pen = 0.0
    w_stop = float(self.get_parameter('w_stop').value)
    for st in self.stops:
        s_stop = float(st.get('s_km', 0.0))
        dwell_s = float(st.get('dwell_s', 0.0))
        width_km = max(0.05, (dwell_s * vmax_ms) / 1000.0 * 0.5)
        if abs(s_km - s_stop) <= width_km:
            pen += w_stop * (v_ms ** 2)
    return pen
```

L2792 関数 MPCNode._fuel_model_lph

- 行範囲: L2792–L2798
- 所属: MPCNode
- 定義: `_fuel_model_lph(self, v_kmh:float, acc_kmhps:float, grade:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`, `max`
- 戻り値の要点: `max(0.0, float(fuel))`
- 主なローカル代入名: `a0`, `a0_eff`, `a1`, `a2`, `a3`, `a4`, `fuel`
- 読み取る主なインスタンス属性: `self.idle_fuel_lph`, `self.model_coeffs`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `(a0, a1, a2, a3, a4)` に `self.model_coeffs` の結果を代入する。
2. `a0_eff` に `a0` の結果を代入する。
3. 条件 `np.isfinite(self.idle_fuel_lph)` を判定し、真なら内部処理を行う。
4. `a0_eff` に `float(self.idle_fuel_lph)` の結果を代入する。
5. `fuel` に `a0_eff + a1 * v_kmh + a2 * v_kmh ** 2 + a3 * acc_kmhps ** 2 + a4 * grade * v_kmh` の結果を代入する。
6. `max(0.0, float(fuel))` を返す。

代表コード断片

```
def _fuel_model_lph(self, v_kmh: float, acc_kmhps: float, grade: float) -> float:
    a0, a1, a2, a3, a4 = self.model_coeffs
    a0_eff = a0
    if np.isfinite(self.idle_fuel_lph):
        a0_eff = float(self.idle_fuel_lph)
    fuel = a0_eff + a1 * v_kmh + a2 * (v_kmh ** 2) + a3 * (acc_kmhps ** 2) + a4 * grade
    * v_kmh
    return max(0.0, float(fuel))
```


L2800 関数 MPCNode._update_identification

- 行範囲: L2800–L2834
- 所属: MPCNode
- 定義: `_update_identification(self, now_sec:float, acc_kmhps:float)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `array`, `column_stack`, `float`, `isfinite`, `len`, `list`, `lstsq`, `maximum`, `mean`, `ones_like`, `popleft`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `X`, `_`, `a`, `alpha`, `coeffs`, `g`, `grade`, `r2`, `resid`, `rmse`, `s`, `samples`, `ss_res`, `ss_tot`, `v`, `y`, `y_hat`
- 読み取る主なインスタンス属性: `self.fuel_rate_lph`, `self.grade`, `self.id_ema_alpha`, `self.id_min_samples`, `self.id_samples`, `self.id_window_sec`, `self.idle_fuel_lph`, `self.model_coeffs`, `self.obd_ok`, `self.online_id_enabled`, `self.v_now`
- 更新する主なインスタンス属性: `self.id_r2`, `self.id_rmse`, `self.model_coeffs`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `not self.online_id_enabled` を判定し、真なら内部処理を行う。
2. を返す。
3. 条件 `self.obd_ok < 0.5` を判定し、真なら内部処理を行う。
4. を返す。
5. 条件 `not np.isfinite(self.v_now) or not np.isfinite(self.fuel_rate_lph)` を判定し、真なら内部処理を行う。
6. を返す。
7. 条件 `self.v_now <= 5.0` を判定し、真なら内部処理を行う。
8. を返す。
9. `grade` に `float(self.grade)` if `np.isfinite(self.grade)` else `0.0` の結果を代入する。
10. `self.id_samples.append(...)` を実行する。
11. 条件 `self.id_samples and now_sec - self.id_samples[0][0] > self.id_window_sec` が成り立つ間くり返す。
12. `self.id_samples.popleft(...)` を実行する。
13. 条件 `len(self.id_samples) < self.id_min_samples` を判定し、真なら内部処理を行う。
14. を返す。

15. `samples` に `list(self.id_samples)` の結果を代入する。
16. `v` に `np.array([s[1] for s in samples], dtype=float)` の結果を代入する。
17. `a` に `np.array([s[2] for s in samples], dtype=float)` の結果を代入する。
18. `g` に `np.array([s[3] for s in samples], dtype=float)` の結果を代入する。
19. `y` に `np.array([s[4] for s in samples], dtype=float)` の結果を代入する。
20. `X` に `np.column_stack([np.ones_like(v), v, v ** 2, a ** 2, g * v])` の結果を代入する。
21. `(coeffs, _, _, _)` に `np.linalg.lstsq(X, y, rcond=None)` の結果を代入する。
22. `coeffs` に `np.maximum(coeffs, 0.0)` の結果を代入する。
23. `alpha` に `self.id_ema_alpha` の結果を代入する。
24. `self.model_coeffs` に `(1.0 - alpha) * self.model_coeffs + alpha * coeffs` の結果を代入する。
25. 条件 `np.isfinite(self.idle_fuel_lph)` を判定し、真なら内部処理を行う。
26. `self.model_coeffs[0]` に `float(self.idle_fuel_lph)` の結果を代入する。
27. `y_hat` に `X @ coeffs` の結果を代入する。
28. `resid` に `y - y_hat` の結果を代入する。
29. `rmse` に `float(np.sqrt(np.mean(resid ** 2))) if len(resid) else math.nan` の結果を代入する。
30. `ss_tot` に `float(np.sum((y - np.mean(y)) ** 2)) if len(y) else 0.0` の結果を代入する。
31. `ss_res` に `float(np.sum(resid ** 2))` の結果を代入する。
32. `r2` に `1.0 - ss_res / ss_tot if ss_tot > 0.0 else math.nan` の結果を代入する。
33. `self.id_rmse` に `rmse` の結果を代入する。
34. `self.id_r2` に `r2` の結果を代入する。

代表コード断片

```
def _update_identification(self, now_sec: float, acc_kmhps: float):
    if not self.online_id_enabled:
        return
    if self.obd_ok < 0.5:
        return
    if not np.isfinite(self.v_now) or not np.isfinite(self.fuel_rate_lph):
        return
    if self.v_now <= 5.0:
        return
    grade = float(self.grade) if np.isfinite(self.grade) else 0.0
    self.id_samples.append((now_sec, float(self.v_now), float(acc_kmhps), grade, float(
self.fuel_rate_lph)))
    while self.id_samples and (now_sec - self.id_samples[0][0]) > self.id_window_sec:
        self.id_samples.popleft()
    if len(self.id_samples) < self.id_min_samples:
        return
    samples = list(self.id_samples)
    v = np.array([s[1] for s in samples], dtype=float)
    a = np.array([s[2] for s in samples], dtype=float)
    g = np.array([s[3] for s in samples], dtype=float)
    y = np.array([s[4] for s in samples], dtype=float)
    X = np.column_stack([np.ones_like(v), v, v ** 2, a ** 2, g * v])
    coeffs, _, _, _ = np.linalg.lstsq(X, y, rcond=None)
    coeffs = np.maximum(coeffs, 0.0)
```

```

alpha = self.id_ema_alpha
self.model_coeffs = (1.0 - alpha) * self.model_coeffs + alpha * coeffs
if np.isfinite(self.idle_fuel_lph):
    self.model_coeffs[0] = float(self.idle_fuel_lph)
y_hat = X @ coeffs
resid = y - y_hat
rmse = float(np.sqrt(np.mean(resid ** 2))) if len(resid) else math.nan
ss_tot = float(np.sum((y - np.mean(y)) ** 2)) if len(y) else 0.0
ss_res = float(np.sum(resid ** 2))
r2 = 1.0 - ss_res / ss_tot if ss_tot > 0.0 else math.nan
self.id_rmse = rmse
self.id_r2 = r2

```

L2836 関数 MPCNode._solve_passo_mpc

- 行範囲: L2836–L2877
- 所属: MPCNode
- 定義: `_solve_passo_mpc(self, w_fuel_override=None) -> np.ndarray`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_fuel_model_lph`, `_stop_penalty_passo`, `abs`, `all`, `array`, `clip`, `dict`, `float`, `get_parameter`, `isfinite`, `max`, `minimize`
- 戻り値の要点: `np.array(res.x, dtype=float) / J / x0`
- 主なローカル代入名: `J`, `Np`, `bounds`, `dv`, `dv_max`, `fuel_l`, `fuel_lph`, `grade`, `k`, `res`, `s_km`, `v0`, `v_k`, `v_max`, `v_min`, `v_prev`, `v_ref`, `w_dv`, `w_dv_limit`, `w_fuel`
- 読み取る主なインスタンス属性: `self.Np`, `self._fuel_model_lph`, `self._stop_penalty_passo`, `self.dt`, `self.get_parameter`, `self.grade`, `self.s_km`, `self.v_now`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `v_min` に `float(self.get_parameter('v_min_kmh').value)` の結果を代入する。
2. `v_max` に `float(self.get_parameter('v_max_kmh').value)` の結果を代入する。
3. `v_ref` に `float(self.get_parameter('v_ref_kmh').value)` の結果を代入する。
4. `w_fuel` に `float(self.get_parameter('w_fuel').value)` の結果を代入する。
5. 条件 `w_fuel_override is not None` を判定し、真なら内部処理を行う。
6. `w_fuel` に `float(w_fuel_override)` の結果を代入する。

7. `w_speed` に `float(self.get_parameter('w_speed').value)` の結果を代入する。
8. `w_dv` に `float(self.get_parameter('w_dv').value)` の結果を代入する。
9. `w_dv_limit` に `float(self.get_parameter('w_dv_limit').value)` の結果を代入する。
10. `dv_max` に `float(self.get_parameter('dv_max_kmhps').value)` の結果を代入する。
11. `Np` に `max(1, self.Np)` の結果を代入する。
12. `v0` に `v_ref if not np.isfinite(self.v_now) else float(np.clip(self.v_now, v_min, v_max))` の結果を代入する。
13. `x0` に `np.array([v0] * Np, dtype=float)` の結果を代入する。
14. `bounds` に `[(v_min, v_max)] * Np` の結果を代入する。
15. `grade` に `float(self.grade) if np.isfinite(self.grade) else 0.0` の結果を代入する。
16. 関数 `cost` を定義する。
17. `res` に `minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=120))` の結果を代入する。
18. 条件 `not np.all(np.isfinite(res.x))` を判定し、真なら内部処理を行う。
19. `x0` を返す。
20. `np.array(res.x, dtype=float)` を返す。

代表コード断片

```
def _solve_passo_mpc(self, w_fuel_override=None) -> np.ndarray:
    v_min = float(self.get_parameter('v_min_kmh').value)
    v_max = float(self.get_parameter('v_max_kmh').value)
    v_ref = float(self.get_parameter('v_ref_kmh').value)
    w_fuel = float(self.get_parameter('w_fuel').value)
    if w_fuel_override is not None:
        w_fuel = float(w_fuel_override)
    w_speed = float(self.get_parameter('w_speed').value)
    w_dv = float(self.get_parameter('w_dv').value)
    w_dv_limit = float(self.get_parameter('w_dv_limit').value)
    dv_max = float(self.get_parameter('dv_max_kmhps').value)

    Np = max(1, self.Np)
    v0 = v_ref if not np.isfinite(self.v_now) else float(np.clip(self.v_now, v_min,
v_max))
    x0 = np.array([v0] * Np, dtype=float)
    bounds = [(v_min, v_max)] * Np

    grade = float(self.grade) if np.isfinite(self.grade) else 0.0

    def cost(v_vec):
        s_km = float(self.s_km)
        v_prev = v0
        J = 0.0
        for k in range(Np):
            v_k = float(v_vec[k])
            dv = (v_k - v_prev) / max(self.dt, 1.0e-3)
            fuel_lph = self._fuel_model_lph(v_k, dv, grade)
            fuel_l = fuel_lph * (self.dt / 3600.0)
            J += w_fuel * fuel_l
            J += w_speed * (v_k - v_ref) ** 2
            J += w_dv * (dv ** 2)
            if dv_max > 0.0:
                J += w_dv_limit * max(0.0, abs(dv) - dv_max) ** 2
```

```
s_km += v_k * (self.dt / 3600.0)
J += self._stop_penalty_passo(s_km, v_k)
```

...

L2855 関数 MPCNode._solve_passo_mpc.cost

- 行範囲: L2855–L2872
- 所属: MPCNode._solve_passo_mpc
- 定義: cost(v_vec)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _fuel_model_lph, _stop_penalty_passo, abs, float, max, range
- 戻り値の要点: J
- 主なローカル代入名: J, dv, fuel_l, fuel_lph, k, s_km, v_k, v_prev
- 読み取る主なインスタンス属性: self._fuel_model_lph, self._stop_penalty_passo, self.dt, self.s_km
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. s_km に float(self.s_km) の結果を代入する。
2. v_prev に v0 の結果を代入する。
3. J に 0.0 の結果を代入する。
4. range(Np) を順に走査し、各要素を k に入れて処理する。
5. v_k に float(v_vec[k]) の結果を代入する。
6. dv に (v_k - v_prev) / max(self.dt, 0.001) の結果を代入する。
7. fuel_lph に self._fuel_model_lph(v_k, dv, grade) の結果を代入する。
8. fuel_l に fuel_lph * (self.dt / 3600.0) の結果を代入する。
9. J を Add で更新する。
10. J を Add で更新する。
11. J を Add で更新する。
12. 条件 dv_max > 0.0 を判定し、真なら内部処理を行う。
13. J を Add で更新する。
14. s_km を Add で更新する。

15. J を Add で更新する。
16. v_prev に v_k の結果を代入する。
17. J を返す。

代表コード断片

```
def cost(v_vec):
    s_km = float(self.s_km)
    v_prev = v0
    J = 0.0
    for k in range(Np):
        v_k = float(v_vec[k])
        dv = (v_k - v_prev) / max(self.dt, 1.0e-3)
        fuel_lph = self._fuel_model_lph(v_k, dv, grade)
        fuel_l = fuel_lph * (self.dt / 3600.0)
        J += w_fuel * fuel_l
        J += w_speed * (v_k - v_ref) ** 2
        J += w_dv * (dv ** 2)
        if dv_max > 0.0:
            J += w_dv_limit * max(0.0, abs(dv) - dv_max) ** 2
        s_km += v_k * (self.dt / 3600.0)
        J += self._stop_penalty_passo(s_km, v_k)
        v_prev = v_k
    return J
```

L2879 関数 MPCNode._publish_trajectory_passo

- 行範囲: L2879–L2892
- 所属: MPCNode
- 定義: `_publish_trajectory_passo(self, v_seq: np.ndarray)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `PoseStamped`, `append`, `enumerate`, `float`, `get_clock`, `now`, `publish`, `to_msg`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `k`, `path`, `pose`, `s_tmp`, `v_k`
- 読み取る主なインスタンス属性: `self.dt`, `self.get_clock`, `self.pub_path`, `self.s_km`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. path に Path() の結果を代入する。
2. path.header.stamp に self.get_clock().now().to_msg() の結果を代入する。
3. path.header.frame_id に 'map' の結果を代入する。
4. s_tmp に float(self.s_km) の結果を代入する。
5. enumerate(v_seq) を順に走査し、各要素を (k, v_k) に入れて処理する。
6. s_tmp を Add で更新する。
7. pose に PoseStamped() の結果を代入する。
8. pose.header に path.header の結果を代入する。
9. pose.pose.position.x に float(k) の結果を代入する。
10. pose.pose.position.y に float(v_k) の結果を代入する。
11. pose.pose.position.z に s_tmp の結果を代入する。
12. path.poses.append(...) を実行する。
13. self.pub_path.publish(...) を実行する。

代表コード断片

```
def _publish_trajectory_passo(self, v_seq: np.ndarray):
    path = Path()
    path.header.stamp = self.get_clock().now().to_msg()
    path.header.frame_id = 'map'
    s_tmp = float(self.s_km)
    for k, v_k in enumerate(v_seq):
        s_tmp += float(v_k) * (self.dt / 3600.0)
        pose = PoseStamped()
        pose.header = path.header
        pose.pose.position.x = float(k)
        pose.pose.position.y = float(v_k)
        pose.pose.position.z = s_tmp
        path.poses.append(pose)
    self.pub_path.publish(path)
```

L2894 関数 MPCNode._step_passo

- 行範囲: L2894–L2962
- 所属: MPCNode
- 定義: _step_passo(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: Float32, Float32MultiArray, String, _fuel_model_lph, _publish_trajectory_passo, _solve_passo_mpc, _update_identification, array, float, get_clock, get_parameter, isfinite
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: acc_kmhps, dt, dt_step, fuel_pred_lph, fuel_valid, grade, msg, now_sec, obd_valid, run_ready, run_ready_sec, speed_valid, status, v_ref, v_seq

- 読み取る主なインスタンス属性: `self.Np`, `self._fuel_model_lph`, `self._publish_trajectory_passo`, `self._solve_passo_mpc`, `self._update_identification`, `self.config_ready`, `self.fuel_rate_lph`, `self.get_clock`, `self.get_parameter`, `self.grade`, `self.id_r2`, `self.id_rmse`, `self.last_step_time`, `self.max_acc_dt_sec`, `self.mpc_state`, `self.obd_ok`, `self.prev_speed_kmh`, `self.prev_speed_time`, `self.pub_mpc_state`, `self.pub_speed`, `self.pub_status`, `self.s_kmh`, `self.v_cmd`, `self.v_now`
- 更新する主なインスタンス属性: `self.last_step_time`, `self.mpc_state`, `self.prev_speed_kmh`, `self.prev_speed_time`, `self.v_cmd`, `self.valid_fuel_sec`, `self.valid_obd_sec`, `self.valid_speed_sec`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 8、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `now_sec` に `self.get_clock().now().nanoseconds / 1000000000.0` の結果を代入する。
2. 条件 `self.last_step_time is None` を判定し、真なら内部処理を行う。
3. `dt_step` に `0.0` の結果を代入する。
4. 上の条件が偽の場合:
5. `dt_step` に `max(0.0, min(5.0, now_sec - self.last_step_time))` の結果を代入する。
6. `self.last_step_time` に `now_sec` の結果を代入する。
7. `acc_kmhps` に `0.0` の結果を代入する。
8. 条件 `np.isfinite(self.v_now)` を判定し、真なら内部処理を行う。
9. 条件 `self.prev_speed_time is not None` を判定し、真なら内部処理を行う。
10. `dt` に `now_sec - self.prev_speed_time` の結果を代入する。
11. 条件 `0.0 < dt <= self.max_acc_dt_sec and np.isfinite(self.prev_speed_kmh)` を判定し、真なら内部処理を行う。
12. `acc_kmhps` に `(float(self.v_now) - float(self.prev_speed_kmh)) / dt` の結果を代入する。
13. `self.prev_speed_time` に `now_sec` の結果を代入する。
14. `self.prev_speed_kmh` に `float(self.v_now)` の結果を代入する。
15. `v_ref` に `float(self.get_parameter('v_ref_kmh').value)` の結果を代入する。
16. `speed_valid` に `np.isfinite(self.v_now)` の結果を代入する。
17. `fuel_valid` に `np.isfinite(self.fuel_rate_lph)` の結果を代入する。
18. `obd_valid` に `self.obd_ok > 0.5` の結果を代入する。
19. 条件 `dt_step > 0.0` を判定し、真なら内部処理を行う。
20. `self.valid_obd_sec` に `self.valid_obd_sec + dt_step if obd_valid else 0.0` の結果を代入する。
21. `self.valid_speed_sec` に `self.valid_speed_sec + dt_step if speed_valid else 0.0` の結果を代入する。
22. `self.valid_fuel_sec` に `self.valid_fuel_sec + dt_step if fuel_valid else 0.0` の結果を代入する。

23. `run_ready_sec` に `float(self.get_parameter('run_ready_sec').value)` の結果を代入する。
24. `run_ready` に `self.valid_obd_sec >= run_ready_sec and self.valid_speed_sec >= run_ready_sec and (self.valid_fuel_sec >= run_ready_sec)` の結果を代入する。
25. 条件 `not self.config_ready` を判定し、真なら内部処理を行う。
26. `self.mpc_state` に 'IDLE' の結果を代入する。
27. `self.v_cmd` に `v_ref` の結果を代入する。
28. `v_seq` に `np.array([self.v_cmd] * max(1, self.Np), dtype=float)` の結果を代入する。
29. 上の条件が偽の場合:
30. 条件 `not run_ready` を判定し、真なら内部処理を行う。
31. `self.mpc_state` に 'DEGRADED_RUN' の結果を代入する。
32. `self.v_cmd` に `float(self.v_now) if speed_valid else v_ref` の結果を代入する。
33. `v_seq` に `np.array([self.v_cmd] * max(1, self.Np), dtype=float)` の結果を代入する。
34. 上の条件が偽の場合:
35. `self.mpc_state` に 'RUN' の結果を代入する。
36. `self._update_identification(...)` を実行する。
37. `v_seq` に `self._solve_passo_mpc()` の結果を代入する。
38. `self.v_cmd` に `float(v_seq[0])` の結果を代入する。
39. `msg` に `Float32()` の結果を代入する。
40. `msg.data` に `float(self.v_cmd)` の結果を代入する。
41. `self.pub_speed.publish(...)` を実行する。
42. `self._publish_trajectory_passo(...)` を実行する。
43. `fuel_pred_lph` に `math.nan` の結果を代入する。
44. 条件 `np.isfinite(self.v_cmd)` を判定し、真なら内部処理を行う。
45. `grade` に `float(self.grade) if np.isfinite(self.grade) else 0.0` の結果を代入する。
46. `fuel_pred_lph` に `self._fuel_model_lph(float(self.v_cmd), 0.0, grade)` の結果を代入する。
47. `status` に `Float32MultiArray()` の結果を代入する。
48. `status.data` に `[float(self.fuel_rate_lph) if np.isfinite(self.fuel_rate_lph) else math.nan, float(fuel_pred_lph) if np.isfinite(fuel_pred_lph) else math.nan, float(self.v_now) if np.isfinite(self.v_now) else math.nan, float(self.v_cmd), float(self.s_km), float(self.id_rmse) if np.isfinite(self.id_rmse) else math.nan, float(self.id_r2) if np.isfinite(self.id_r2) else math.nan]` の結果を代入する。
49. `self.pub_status.publish(...)` を実行する。
50. `self.pub_mpc_state.publish(...)` を実行する。

代表コード断片

```
def _step_passo(self):
    now_sec = self.get_clock().now().nanoseconds / 1e9
    if self.last_step_time is None:
        dt_step = 0.0
    else:
        dt_step = max(0.0, min(5.0, now_sec - self.last_step_time))
    self.last_step_time = now_sec

    acc_kmhps = 0.0
    if np.isfinite(self.v_now):
        if self.prev_speed_time is not None:
            dt = now_sec - self.prev_speed_time
            if 0.0 < dt <= self.max_acc_dt_sec and np.isfinite(self.prev_speed_kmh):
                acc_kmhps = (float(self.v_now) - float(self.prev_speed_kmh)) / dt
            self.prev_speed_time = now_sec
            self.prev_speed_kmh = float(self.v_now)

    v_ref = float(self.get_parameter('v_ref_kmh').value)
    speed_valid = np.isfinite(self.v_now)
    fuel_valid = np.isfinite(self.fuel_rate_lph)
    obd_valid = self.obd_ok > 0.5

    if dt_step > 0.0:
        self.valid_obd_sec = self.valid_obd_sec + dt_step if obd_valid else 0.0
        self.valid_speed_sec = self.valid_speed_sec + dt_step if speed_valid else 0.0
        self.valid_fuel_sec = self.valid_fuel_sec + dt_step if fuel_valid else 0.0

    run_ready_sec = float(self.get_parameter('run_ready_sec').value)
    run_ready = (self.valid_obd_sec >= run_ready_sec and
                 self.valid_speed_sec >= run_ready_sec and
                 self.valid_fuel_sec >= run_ready_sec)

    if not self.config_ready:
        self.mpc_state = 'IDLE'
        self.v_cmd = v_ref

...
```

L2965 関数 main

- 行範囲: L2965–L2975
- 所属: module 直下
- 定義: main()
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: MPCNode, MultiThreadedExecutor, add_node, destroy_node, init, shutdown, spin
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: executor, node
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. rclpy.init(...) を実行する。
2. node に MPCNode() の結果を代入する。
3. executor に MultiThreadedExecutor(num_threads=4) の結果を代入する。
4. executor.add_node(...) を実行する。
5. 例外処理を伴う try ブロックを実行する。
6. executor.spin(...) を実行する。
7. 成否にかかわらず finally で:
8. executor.shutdown(...) を実行する。
9. node.destroy_node(...) を実行する。
10. rclpy.shutdown(...) を実行する。

代表コード断片

```
def main():
    rclpy.init()
    node = MPCNode()
    executor = MultiThreadedExecutor(num_threads=4)
    executor.add_node(node)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()
```

設定値と入力パラメータ

行	名前	既定値・補足
58	passo_mode	False
292	forecast_csv	inputs/forecast_10min.csv
293	maps_dir	maps
294	drive_eff_map	
295	regen_eff_map	
296	rint_map	
297	drive_map_eco	
298	drive_map_power	
299	regen_map_eco	
300	regen_map_power	
301	panel_eff_map	
302	mppt_eff_map	
303	ocv_soc_map	

304	dt	600.0
305	horizon_steps	9
306	v_max_kmh	110.0
307	terminal_soc_min	0.1
308	stop_yaml	inputs/stop_points.yaml
309	forecast_time_mode	auto
310	forecast_time_tz	UTC
311	forecast_start_time_utc	
312	forecast_time_offset_sec	0.0
313	forecast_reload_sec	60.0
314	replan_on_forecast_reload	False
315	params_yaml	
316	profile_runtime_mode	
317	route_profile_csv	
318	speed_profile_csv	
319	drive_schedule_yaml	
320	initial_upper_policy_csv	
321	use_measured_s	True
322	use_measured_speed	True
323	soc0	0.95
324	Tb0	30.0
325	s0_km	0.0
326	speed_meas_timeout_sec	3.0
327	distance_meas_timeout_sec	5.0
328	battery_meas_timeout_sec	15.0
329	speed_meas_filter_tau_sec	0.6
330	speed_meas_max_accel_kmhps	12.0
331	speed_meas_max_decel_kmhps	20.0
332	distance_meas_max_rate_kmps	0.06
333	distance_meas_max_backtrack_km	0.02
334	battery_meas_filter_tau_sec	1.0
335	w_dv	0.05
336	w_dv_limit	2.0
337	dv_max_kmhps	5.0
338	w_T	5.0
339	w_speed_limit	50.0
340	w_drive_window	100000.0

ROS topic I/O

行	種別	topic	callback / 補足
737	publisher	/planner/speed_cmd	publisher
738	publisher	/planner/upper_speed_cmd	publisher
739	publisher	/planner/throttle_cmd_pct	publisher
740	publisher	/planner/drive_mode	publisher
741	publisher	/planner/trajectory	publisher

742	publisher	/planner/upper_plan	publisher
743	publisher	/planner/lower_plan	publisher
744	publisher	/planner/env	publisher
745	publisher	/planner/metrics	publisher
746	publisher	/planner/status	publisher
2700	publisher	/planner/speed_cmd	publisher
2701	publisher	/planner/trajectory	publisher
2702	publisher	/planner/status	publisher
2703	publisher	/system/mpc_state	publisher
749	subscription	/vehicle/s_km	self._on_s_km_solar
750	subscription	/vehicle/speed_kmh	self._on_speed_solar
751	subscription	/vehicle/batt_soc	self._on_soc_solar
752	subscription	/vehicle/batt_temp_c	self._on_tb_solar
753	subscription	/vehicle/batt_current_a	self._on_i_solar
754	subscription	/vehicle/batt_voltage_v	self._on_v_solar
755	subscription	/calib/solar_gain	self._on_calib_solar_gain
756	subscription	/calib/drive_power_gain	self._on_calib_drive_power_gain
757	subscription	/calib/aux_power_w	self._on_calib_aux_power
2688	subscription	/vehicle/s_km	self._on_s_km
2689	subscription	/vehicle/speed_kmh	self._on_speed
2690	subscription	/vehicle/fuel_rate_lph	self._on_fuel
2691	subscription	/vehicle/throttle_pct	self._on_throttle
2692	subscription	/vehicle/obd_ok	self._on_obd_ok
2693	subscription	/vehicle/grade	self._on_grade
2694	subscription	/vehicle/idle_fuel_lph	self._on_idle_fuel
2695	subscription	/system/config	self._on_config
2696	subscription	/system/config_ready	self._on_config_ready
2697	subscription	/system/state	self._on_system_state

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/mpc_node.py` を **runtime core** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。